



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática — Ingeniería del Software

Sistema de Gestión de Entregables Académicos

**Realizado por
Manuel María Calderón Rodríguez
José Manuel Márquez Gutiérrez**

**Dirigido por
Antonio Manuel Gutiérrez Fernández**

**Departamento
Escuela Técnica Superior de Ingeniería Informática**

Sevilla, Mayo de 2026

Resumen

Contexto y problema: en la operativa académica habitual, la gestión de entregables suele fragmentarse entre herramientas generales, con fricciones en trazabilidad, seguimiento de versiones y evaluación docente. Este TFG aborda el problema de disponer de un flujo integral, desde la definición del entregable hasta la publicación de feedback y notas, adaptado a perfiles de administración, profesorado y alumnado.

Relevancia: la necesidad es especialmente significativa por su impacto directo en tiempo de corrección, coherencia de criterios de evaluación, calidad de experiencia del estudiante y sostenibilidad del sistema cuando aumenta la carga docente o el número de grupos.

Solución propuesta y método: se diseña e implementa una plataforma web con arquitectura cliente-servidor desacoplada, frontend en React + TypeScript y backend en Spring Boot sobre Java 21, persistencia relacional y migraciones versionadas con Flyway. La seguridad combina autenticación JWT con renovación de sesión, autorización por rol y validaciones por contexto académico. El desarrollo se ejecuta por iteraciones, con trazabilidad entre requisitos, backlog, implementación y pruebas.

Resultados y por qué la solución es adecuada: el sistema cubre el ciclo funcional completo (cursos, actividades, entregables, entregas, evaluación y feedback), con una estrategia de calidad en capas (pruebas unitarias, integración, E2E y mutación), automatización CI/CD y despliegue cloud reproducible. El resultado es una base funcional, mantenible y escalable, con menor riesgo de regresiones y mayor trazabilidad técnica, alineada con los objetivos definidos para el TFG.

Resumen complementario

Contexto y problema: los flujos de trabajo de entregas académicas suelen estar fragmentados entre herramientas de propósito general, lo que genera fricciones en trazabilidad, control de versiones de entregas y consistencia de calificaciones. Esta memoria aborda la necesidad de un flujo integral de extremo a extremo, desde la definición del entregable hasta la publicación de retroalimentación y calificaciones, adaptado a administradores, docentes y estudiantes.

Relevancia: el problema tiene impacto directo en la carga de trabajo docente, la experiencia del estudiante y la mantenibilidad a largo plazo a medida que aumenta el número de cursos y grupos.

Solución propuesta y método: el proyecto diseña e implementa una plataforma web desacoplada, con frontend en React + TypeScript y backend en Spring Boot sobre Java 21, persistencia relacional y migraciones de esquema versionadas con Flyway. La seguridad se basa en autenticación JWT con renovación de sesión, autorización por rol y validación de acceso contextualizada. El desarrollo sigue incrementos iterativos con trazabilidad explícita entre requisitos, backlog, implementación y pruebas.

Resultados y adecuación: la plataforma resultante cubre el ciclo funcional completo (cursos, actividades, entregables, entregas, calificación y retroalimentación), y aplica una estrategia de calidad en capas (pruebas unitarias, de integración, de extremo a extremo y de mutación), además de automatización CI/CD y despliegue en cloud. El resultado es una solución funcional, mantenible y evolutiva, con menor riesgo de regresiones y mayor trazabilidad técnica.

Agradecimientos

Queremos expresar nuestro agradecimiento al tutor del Trabajo Fin de Grado, por la orientación técnica y metodológica aportada durante el desarrollo del proyecto, y por su seguimiento continuo en las distintas fases de análisis, implementación y validación.

Asimismo, agradecemos a la Escuela Técnica Superior de Ingeniería Informática de la Universidad de Sevilla el marco formativo y los recursos que han hecho posible la realización de este trabajo.

Finalmente, reconocemos el apoyo de nuestras familias y compañeros, cuya ayuda ha sido clave para completar este proyecto con el nivel de exigencia requerido.

Índice general

Índice general	IV
Índice de cuadros	X
Índice de figuras	XII
1 Definición de objetivos	1
1.1 Contexto y motivación	1
1.2 Problema de ingeniería a resolver	1
1.3 Objetivo general	2
1.4 Metodología para definir los objetivos	2
1.5 Objetivos específicos	2
1.6 Priorización de objetivos (criterio MoSCoW)	3
1.7 Alcance funcional del trabajo	3
1.8 Delimitación explícita del alcance	3
1.9 Criterios de cumplimiento y evidencia de logro	4
1.10 Trazabilidad objetivos-requisitos-entregables	4
1.11 Riesgos estratégicos asociados a los objetivos	4
1.12 Conclusiones del capítulo	5
2 Análisis de requisitos, diseño e implementación	6
2.1 Introducción	6
2.2 Ecosistema de desarrollo y operación	6
2.2.1 Lenguajes y herramientas utilizadas	6
2.2.2 Política de gestión del código fuente	6
2.2.3 Estrategia de integración continua y despliegue	6
2.2.4 Uso de asistentes de Inteligencia Artificial	7
2.3 Análisis de requisitos	7
2.3.1 Fuentes de requisitos	7
2.3.2 Estructura del catálogo	7
2.3.3 Requisitos funcionales	8
2.3.4 Reglas de negocio	8
2.3.5 Requisitos no funcionales	8
2.3.6 Priorización y alcance	9
2.3.7 Trazabilidad requisito-backlog-código	9
2.3.8 Contexto documental de partida (ERS y DAS)	9
2.3.9 Situación actual y necesidades de negocio (ERS)	10
2.3.10 Descomposición en subsistemas (ERS)	10
2.3.11 Catálogo exhaustivo de requisitos del ERS	10
2.3.12 Reutilización de diagramas y mockups del DAS	11

2.3.13	Lectura técnica de los diagramas estructurales del DAS	11
2.3.14	Flujos operativos del DAS y su encaje en la solución	17
2.3.15	Catálogo operativo de casos de uso y cobertura funcional	17
2.3.16	Matriz ampliada de trazabilidad ERS-DAS-implementación	22
2.3.17	Cobertura documental alcanzada	22
2.4	Diseño del sistema	25
2.4.1	Arquitectura de alto nivel	25
2.4.2	Vista de despliegue	25
2.4.3	Diseño de dominio y datos	26
2.4.4	Diseño de API	27
2.4.5	Diseño de seguridad	27
2.4.6	Diseño del frontend	28
2.5	Implementación	29
2.5.1	Implementación backend	29
2.5.2	Implementación de flujos críticos	29
2.5.3	Implementación frontend	30
2.5.4	Persistencia, migraciones e integraciones	30
2.5.5	Verificación de implementación	31
2.6	Discusión técnica	31
2.6.1	Decisiones y compromisos	31
2.6.2	Limitaciones y trabajo futuro	31
2.7	Conclusiones del capítulo	32
3	Análisis de antecedentes y aportación realizada	33
3.1	Introducción	33
3.2	Metodología de análisis de antecedentes	33
3.3	Situación actual (escenario <i>actual</i>)	33
3.3.1	Actores y procesos principales	33
3.3.2	Fortalezas del entorno de partida	34
3.3.3	Debilidades estructurales detectadas	34
3.3.4	Análisis de causas raíz	34
3.4	Brechas entre escenario actual y necesidades objetivo	34
3.5	Entorno tecnológico de partida	35
3.6	Análisis de competidores y brechas sectoriales	35
3.6.1	Tipologías de competidores analizados	35
3.6.2	LMS generalistas	35
3.6.3	Herramientas de evaluación técnica	35
3.6.4	Suites colaborativas y almacenamiento cloud	35
3.6.5	Gestores de entrega simples	36
3.6.6	Brechas sectoriales detectadas	36
3.6.7	Implicaciones para el diseño del TFG	36
3.7	Aportación realizada (escenario <i>objetivo</i>)	36
3.7.1	Aportación funcional	36
3.7.2	Aportación arquitectónica y tecnológica	37
3.7.3	Aportación en calidad y operación	37
3.8	Matriz de mapeo debilidades-aportaciones	37
3.9	Comparativa sintética	37
3.10	Alineación con objetivos de negocio	38
3.11	Limitaciones residuales y continuidad	38
3.12	Conclusiones del capítulo	38

4	Análisis temporal y costes de desarrollo	39
4.1	Análisis temporal	39
4.1.1	Metodología aplicada y roles del proyecto	39
4.1.2	Herramientas de gestión y seguimiento	39
4.1.3	Marco de planificación	40
4.1.4	Hipótesis de capacidad y calendario base	40
4.1.5	Planificación macro por sprints	40
4.1.6	Planificación micro y distribución del esfuerzo	40
4.1.7	Hitos de control del proyecto	41
4.1.8	Seguimiento del avance y métricas de control	41
4.1.9	Control de desviaciones con valor ganado simplificado	42
4.1.10	Desviaciones y replanificación	42
4.1.11	Lecciones aprendidas de planificación	43
4.1.12	Riesgos temporales y mitigación	43
4.1.13	Conclusión temporal	43
4.2	Costes de desarrollo	43
4.2.1	Criterio de estimación económica	43
4.2.2	Coste de personal	44
4.2.3	Coste de personal por iteración	44
4.2.4	Coste asociado a aseguramiento de calidad	44
4.2.5	Costes de infraestructura y operación	45
4.2.6	Coste total del proyecto	45
4.2.7	Análisis de sensibilidad económica	45
4.2.8	Interpretación económica y sostenibilidad	45
4.2.9	Conclusión del análisis de costes	46
4.3	Alternativas de arquitectura y stack	46
4.3.1	Arquitectura de aplicación	46
4.3.2	Backend framework	47
4.3.3	Frontend framework	47
4.4	Alternativas de persistencia y almacenamiento	47
4.4.1	Base de datos relacional	47
4.4.2	Estrategia de esquema	48
4.4.3	Almacenamiento de archivos	48
4.5	Alternativas de seguridad y sesión	48
4.5.1	Modelo de autenticación	48
4.5.2	Extensiones de seguridad	48
4.6	Alternativas investigadas y descartadas	48
4.6.1	SharePoint para OneDrive corporativo	48
4.6.2	Registro de usuarios con OAuth	49
4.7	Alternativas de despliegue y operación	49
4.7.1	Estrategia de despliegue	49
4.7.2	CI/CD y calidad	49
4.8	Comparativa multicriterio	49
4.9	Decisiones adoptadas y justificación final	49
4.10	Conclusiones del capítulo	50
5	Justificación detallada de decisiones	51
5.1	Introducción	51
5.2	Marco de toma de decisiones	51
5.2.1	Criterios utilizados	51
5.2.2	Ponderación de criterios	51
5.3	Decisiones de alcance funcional	51

5.3.1	Decisión D1: definir un MVP centrado en flujo académico nuclear	51
5.3.2	Decisión D2: diseñar experiencia por rol desde el inicio	52
5.4	Decisiones de arquitectura	52
5.4.1	Decisión D3: arquitectura SPA + API desacoplada	52
5.4.2	Decisión D4: arquitectura backend por capas	53
5.4.3	Decisión D5: uso de DTOs y mapper dedicado	53
5.5	Decisiones de stack tecnológico	53
5.5.1	Decisión D6: backend en Spring Boot + Java 21	53
5.5.2	Decisión D7: frontend en React + TypeScript + Vite	53
5.6	Decisiones de seguridad	54
5.6.1	Decisión D8: autenticación JWT con refresh token	54
5.6.2	Decisión D9: autorización por rol + contexto académico	54
5.7	Decisiones de datos e integraciones	54
5.7.1	Decisión D10: PostgreSQL como base de datos principal	54
5.7.2	Decisión D11: Flyway como autoridad de esquema	54
5.7.3	Decisión D12: estrategia híbrida de almacenamiento de archivos	54
5.8	Decisiones de calidad y verificación	55
5.8.1	Decisión D13: testing en capas	55
5.8.2	Decisión D14: incluir mutación (PIT/Stryker)	55
5.9	Decisiones de operación y despliegue	55
5.9.1	Decisión D15: GitHub Actions como columna vertebral CI/CD	55
5.9.2	Decisión D16: despliegue en PaaS gestionado	55
5.10	Matriz consolidada de decisiones	56
5.11	Análisis coste-beneficio por bloques de decisiones	56
5.11.1	Bloque de arquitectura	56
5.11.2	Bloque de stack tecnológico	56
5.11.3	Bloque de seguridad	56
5.11.4	Bloque de calidad y pruebas	57
5.11.5	Bloque de despliegue y operación	57
5.12	Riesgos asociados a decisiones y medidas de contingencia	57
5.13	Trazabilidad de decisiones con requisitos y evidencia	57
5.14	Decisiones descartadas y por qué no se adoptaron en esta fase	58
5.14.1	Autenticación federada completa en MVP	58
5.14.2	2FA integral desde primera versión	58
5.14.3	Orquestación avanzada de infraestructura	58
5.15	Conclusiones del capítulo	58
6	Pruebas	59
6.1	Introducción	59
6.2	Objetivos de verificación	59
6.3	Estrategia de pruebas en capas	59
6.4	Inventario cuantitativo de pruebas realizadas	60
6.5	Pruebas en frontend	60
6.5.1	Pruebas unitarias y cobertura	60
6.5.2	Módulos testeados en frontend	61
6.5.3	Pruebas E2E	61
6.5.4	Mutación en frontend	61
6.6	Pruebas en backend	62
6.6.1	Pruebas unitarias e integración	62
6.6.2	Módulos testeados en backend	62
6.6.3	Cobertura backend	62
6.6.4	Mutación en backend	62

6.7	Automatización en CI/CD	63
6.8	Resultados verificados	63
6.9	Criterios de aceptación	64
6.10	Limitaciones y riesgos	64
6.11	Conclusiones del capítulo	64
7	Manual de usuario	65
7.1	Introducción	65
7.2	Acceso al sistema	65
7.2.1	Requisitos previos de uso	65
7.2.2	Pantallas de acceso	65
7.2.3	Ciclo de sesión	66
7.2.4	Recuperación de contraseña	66
7.3	Perfiles y permisos	66
7.4	Vistas comunes a todos los perfiles	66
7.5	Guía de uso para profesorado	66
7.5.1	Vistas principales	66
7.5.2	Crear y configurar una actividad	66
7.5.3	Crear y editar entregables	74
7.5.4	Revisar entregas y evaluar	74
7.5.5	Flujo docente recomendado de extremo a extremo	75
7.6	Guía de uso para estudiantes	75
7.6.1	Vistas principales	75
7.6.2	Consultar actividades y entregables	75
7.6.3	Realizar una entrega	78
7.6.4	Recomendaciones para entregas sin incidencias	78
7.6.5	Consultar historial y feedback	78
7.7	Guía de uso para administración	78
7.7.1	Vistas principales	78
7.7.2	Panel de administración	78
7.7.3	Operaciones destacadas	85
7.7.4	Políticas recomendadas de gobierno de usuarios	85
7.8	Mensajes de error y resolución de incidencias	85
7.8.1	Matriz de incidencias operativas	86
7.9	Seguridad y protección de la información en el uso diario	86
7.10	Checklist operativo por perfil	86
7.11	Buenas prácticas de uso	87
7.12	Alternativa o complemento a la enseñanza virtual	87
7.13	Aspectos novedosos	87
7.14	Desafíos técnicos encontrados en el proyecto	87
7.15	Análisis de la evolución del proyecto	88
7.16	Conclusiones del capítulo	88
8	Conclusiones y desarrollos futuros	89
8.1	Síntesis final del trabajo	89
8.2	Grado de cumplimiento de objetivos	89
8.3	Aportaciones principales	89
8.4	Resultados y evidencia	90
8.5	Evaluación del impacto por tipo de usuario	90
8.6	Validación del enfoque de ingeniería adoptado	91
8.7	Limitaciones del trabajo	91
8.8	Lecciones aprendidas	91

8.9	Riesgos residuales y deuda técnica controlada	91
8.10	Desarrollos futuros	92
8.10.1	Corto plazo	92
8.10.2	Medio plazo	92
8.10.3	Largo plazo	92
8.11	Hoja de ruta priorizada de continuidad	92
8.12	Reflexión final	92
8.13	Cierre	93
A	Apéndices	94
A.1	Introducción	94
A.2	Anexo A. Manual de instalación	94
A.2.1	Requisitos previos	94
A.2.2	Preparación del entorno	94
A.2.3	Base de datos	94
A.2.4	Arranque del backend	94
A.2.5	Arranque del frontend	95
A.2.6	Ejecución con contenedores	95
A.2.7	Build de producción	95
A.3	Anexo B. Configuración por entornos	95
A.3.1	Variables críticas de backend	95
A.3.2	Variables críticas de frontend	95
A.4	Anexo C. Operación y verificación	96
A.4.1	Comprobaciones de salud	96
A.4.2	Validación post-despliegue	96
A.5	Anexo D. Declaración de uso de IA generativa	96
A.5.1	Alcance y criterios de uso	96
A.5.2	Proceso de control y verificación	97
A.5.3	Ejemplos concretos de consultas y respuestas	97
A.6	Anexo E. Documentación complementaria disponible	97
A.7	Anexo F. Referencias técnicas de apoyo	98
A.8	Cierre de apéndices	98
	Referencias	99

Índice de cuadros

1.1	Priorización de objetivos conforme a estrategia MoSCoW	3
1.2	Criterios de evaluación del cumplimiento de objetivos	4
1.3	Trazabilidad entre objetivos, requisitos y resultados técnicos	4
2.1	Requisitos no funcionales representativos	8
2.2	Ejemplo de trazabilidad entre requisitos, backlog e implementación	9
2.3	Subsistemas definidos en ERS y reutilizados en el diseño del TFG	10
2.4	Inventario de requisitos generales (RQG) en ERS	11
2.5	Inventario de requisitos de información (RQI) en ERS	11
2.6	Inventario de requisitos funcionales (RQF-001 a RQF-012)	17
2.7	Inventario de requisitos funcionales (RQF-013 a RQF-025)	22
2.8	Inventario de reglas de negocio (REGN) en ERS	25
2.9	Inventario de requisitos no funcionales (RQNF) en ERS	26
2.10	Diagramas principales reutilizados desde DAS	26
2.11	Inventario de mockups de interfaz reutilizados desde DAS	27
2.12	Interpretación técnica de los diagramas estructurales del DAS	28
2.13	Cobertura operativa por rol con evidencia de DAS y requisitos de ERS	28
2.14	Cobertura por familias de casos de uso del DAS dentro del alcance del MVP	29
2.15	Trazabilidad ampliada entre ERS, DAS e implementación efectiva	30
2.16	Vista de despliegue simplificada del sistema	31
3.1	Brechas identificadas entre el escenario de partida y el objetivo	34
3.2	Relación entre debilidades detectadas y aportación implementada	37
3.3	Comparativa entre escenario de partida y propuesta desarrollada	38
4.1	Planificación temporal macro del proyecto	41
4.2	Seguimiento temporal con valor ganado simplificado	42
4.3	Riesgos temporales y acciones de mitigación	43
4.4	Coste de personal para 660 horas de proyecto	44
4.5	Coste por sprint según escenario de tarifa	44
4.6	Coste estimado del aseguramiento de calidad	45
4.7	Comparativa entre coste real y coste operativo orientativo	45
4.8	Coste total estimado del proyecto	45
4.9	Sensibilidad del coste total ante variación de tarifa horaria	46
4.10	Comparativa global (escala 1-5, mayor es mejor)	50
5.1	Ponderación orientativa para selección de decisiones	52
5.2	Resumen de decisiones clave y justificación dominante	56
5.3	Riesgos derivados de decisiones y medidas de contingencia	57
5.4	Trazabilidad entre decisiones, requisitos y evidencia	58

6.1	Matriz de niveles y herramientas de prueba	60
6.2	Distribución de casos unitarios por módulo en frontend	61
6.3	Distribución de casos de prueba por módulo en backend	62
6.4	Resumen de resultados de verificación	64
7.1	Capacidades por perfil de usuario	66
7.2	Matriz de incidencias frecuentes y resolución recomendada	86
8.1	Hoja de ruta priorizada para evolución del sistema	92

Índice de figuras

2.1	Arquitectura lógica del sistema (DAS)	12
2.2	Modelo de clases del sistema (DAS)	12
2.3	Diagrama de estado de actividades (DAS)	13
2.4	Navegabilidad principal de la interfaz (DAS)	13
2.5	Secuencia de autenticación y acceso (DAS)	14
2.6	Secuencia de realización de entrega (DAS)	14
2.7	Mockup de panel principal de estudiante (DAS)	15
2.8	Mockup de panel principal de profesor (DAS)	16
2.9	Secuencia de creación de actividad en backend (DAS)	18
2.10	Secuencia de evaluación y feedback (DAS)	18
2.11	Mockup operativo: estudiante dentro de una asignatura (DAS)	19
2.12	Mockup operativo: detalle de calificación como profesor (DAS)	20
2.13	Mockup de entrega con archivo adjunto (DAS)	21
2.14	Caso de uso representativo CU-1 (DAS)	23
2.15	Caso de uso representativo CU-2 (DAS)	23
2.16	Caso de uso representativo CU-3 (DAS)	24
2.17	Caso de uso representativo CU-4 (DAS)	24
7.1	Pantalla de inicio de sesión. Vista compartida por todos los perfiles.	67
7.2	Sección de perfil (1/2). Vista compartida por todos los perfiles.	67
7.3	Sección de perfil (2/2). Vista compartida por todos los perfiles.	67
7.4	Dashboard docente. Secciones visibles: dashboard, actividades, perfil y evaluaciones.	68
7.5	Sección de actividades por curso.	68
7.6	Detalle de asignatura con profesorado asignado.	69
7.7	Creación de actividad (1/2).	69
7.8	Creación de actividad (2/2).	69
7.9	Eliminación de actividad desde la asignatura.	70
7.10	Detalle de actividad y entregables asociados.	70
7.11	Creación de entregables (1/2).	70
7.12	Creación de entregables (2/2).	71
7.13	Edición de actividad (1/3).	71
7.14	Edición de actividad (2/3).	72
7.15	Edición de actividad (3/3).	72
7.16	Detalle de entregable para revisiones.	73
7.17	Bandeja de evaluaciones (1/2).	73
7.18	Bandeja de evaluaciones (2/2).	73
7.19	Evaluación de la actividad de un alumno.	74
7.20	Dashboard de estudiante. Secciones visibles: panel principal, actividades, perfil y calificaciones.	75
7.21	Detalle de entregable desde la visión de estudiante.	76

7.22	Formulario de entrega (1/2).	76
7.23	Formulario de entrega (2/2).	76
7.24	Entrega registrada correctamente.	77
7.25	Consulta de calificación por estudiante.	77
7.26	Panel de administración. Secciones visibles: usuarios, cursos y grupos.	79
7.27	Creación de usuarios.	79
7.28	Modificación de usuarios.	80
7.29	Eliminación de usuarios.	80
7.30	Gestión de cursos.	81
7.31	Creación de curso (1/2).	81
7.32	Creación de curso (2/2).	82
7.33	Edición de curso.	82
7.34	Eliminación de curso.	83
7.35	Gestión de grupos.	83
7.36	Creación de grupo.	83
7.37	Edición de grupo.	84
7.38	Eliminación de grupo.	84
7.39	Asignación de profesorado a cursos.	84
7.40	Asignación de usuarios en grupo.	85

Definición de objetivos

1.1– Contexto y motivación

El presente Trabajo Fin de Grado se desarrolla en el ámbito de la gestión académica universitaria, centrado en el ciclo de vida completo de las actividades evaluables: definición de actividades, publicación de entregables, realización de entregas, evaluación y retroalimentación.

La motivación principal se fundamenta en una realidad operativa detectada en el análisis previo: en muchos contextos docentes, el flujo de entrega y corrección combina una plataforma institucional de propósito general con tareas manuales de clasificación, validación y descarga de archivos. Esa combinación introduce ineficiencias en tres dimensiones críticas:

- **Esfuerzo docente:** se consume tiempo en operaciones administrativas de bajo valor.
- **Experiencia del estudiante:** la entrega puede resultar poco guiada y con validación tardía.
- **Trazabilidad de evaluación:** la relación entrega-version-feedback no siempre queda consolidada en un mismo flujo.

Por tanto, el proyecto no persigue replicar una plataforma LMS generalista, sino resolver de forma especializada el problema operativo de la gestión de entregables, apoyándose en los artefactos de requisitos y análisis del sistema (ERS y DAS), y en la planificación ejecutiva definida en backlog y plan de sprints.

1.2– Problema de ingeniería a resolver

Desde una perspectiva de ingeniería del software, el problema se formula como la necesidad de diseñar un sistema que, con recursos acotados de tiempo y equipo, maximice simultáneamente:

1. cobertura funcional del ciclo de entregables,
2. robustez técnica y seguridad por rol,
3. capacidad de evolución sin deuda estructural excesiva.

Este problema impone una restricción de diseño clave: no basta con construir funcionalidades aisladas, sino que es necesario garantizar coherencia extremo a extremo entre requisitos, arquitectura, implementación, pruebas y despliegue.

1.3– Objetivo general

Diseñar, implementar y validar un sistema web para la gestión integral de entregables académicos, que permita a profesores y estudiantes operar de forma centralizada, segura y trazable, reduciendo tareas manuales y mejorando la eficiencia del proceso de evaluación continua.

1.4– Metodología para definir los objetivos

La definición de objetivos no se ha realizado de forma declarativa aislada, sino mediante un proceso iterativo de convergencia entre:

- objetivos de negocio (OBJN) definidos en el análisis inicial,
- requisitos funcionales y no funcionales del ERS,
- decisiones arquitectónicas del DAS,
- prioridades MoSCoW del Product Backlog,
- capacidad temporal real (660 horas en 6 iteraciones).

Este enfoque permite que cada objetivo sea evaluable, relacionable con entregables técnicos concretos y verificable mediante evidencia objetiva de implementación y pruebas.

1.5– Objetivos específicos

Para alcanzar el objetivo general se establecen los siguientes objetivos específicos, ordenados por su contribución al MVP:

1. **Centralizar la estructura académica del sistema.** Modelar cursos, grupos, actividades, entregables y entregas de forma coherente con la organización docente real.
2. **Proporcionar flexibilidad en la configuración de actividades y entregables.** Permitir al profesorado definir condiciones de entrega (fechas, visibilidad, reenvío, tipo de material y parámetros de evaluación).
3. **Optimizar el proceso de entrega para el alumnado.** Implementar flujos de subida y consulta de historial de versiones, con validaciones y mensajes de estado claros.
4. **Mejorar la evaluación y la retroalimentación.** Facilitar la revisión de entregas por parte del profesorado, incorporando calificación y feedback en el mismo flujo de trabajo.
5. **Garantizar seguridad y control de acceso por roles y contexto.** Aplicar autenticación y autorización de forma consistente, diferenciando permisos de administrador, profesor y estudiante.
6. **Diseñar una arquitectura modular e interoperable.** Mantener separación clara entre frontend y backend mediante API REST, facilitando integraciones y evolución por componentes.
7. **Asegurar calidad técnica mediante validación automatizada.** Verificar el comportamiento funcional y la robustez del sistema con pruebas unitarias, de integración, E2E y de mutación.
8. **Establecer un despliegue reproducible y mantenible.** Integrar prácticas de CI/CD y contenedorización para sostener entregas frecuentes, trazables y auditables.

1.6– Priorización de objetivos (criterio MoSCoW)

Para alinear expectativas y alcance, los objetivos se priorizan con criterio MoSCoW, consistente con el backlog del proyecto.

Prioridad	Objetivos asociados	Justificación
Must Have	O1, O2, O3, O4, O5	Sin estos objetivos no existe flujo académico completo de entrega y evaluación.
Should Have	O6, O7	Aumentan mantenibilidad y calidad, y reducen riesgo de regresión.
Could Have	Extensiones analíticas y observabilidad avanzada	Aportan valor, pero no condicionan la validez del MVP.
Won't Have (MVP)	inicio de sesión federado (OAuth2/OIDC), 2FA integral, notificaciones avanzadas	Se reservan para evolución por coste de implementación y validación.

Cuadro 1.1: Priorización de objetivos conforme a estrategia MoSCoW



1.7– Alcance funcional del trabajo

El alcance funcional cubierto en este TFG incluye:

- gestión de usuarios y control de acceso por rol;
- gestión de cursos, grupos y actividades académicas;
- gestión de entregables con configuración de plazos y visibilidad;
- realización de entregas, historial de versiones y descarga;
- evaluación y retroalimentación docente;
- automatización de validación y despliegue en entorno CI/CD.

1.8– Delimitación explícita del alcance

Para mantener viabilidad temporal y calidad de salida, se delimita de forma explícita lo que queda fuera del MVP, aunque exista su consideración en el análisis inicial:

- inicio de sesión federado (OAuth2/OIDC institucional),
- doble factor de autenticación integral (2FA),
- notificaciones avanzadas desacopladas,
- funcionalidades avanzadas de analítica y monitorización.

Estas exclusiones se refieren al inicio de sesión federado; la autorización OAuth2 necesaria para conectar OneDrive sí está implementada.

Esta delimitación no representa una carencia metodológica, sino una decisión de ingeniería para proteger el objetivo principal: entregar un sistema funcional, verificable y defendible en plazo.

Obj.	Indicador principal	Meta de cumplimiento	Evidencia
O1–O4	Cobertura del flujo funcional extremo a extremo	Flujo completo operativo por rol	Endpoints y pantallas de curso/actividad/entrega/evaluación
O5	Restricciones de acceso aplicadas	Operaciones protegidas por rol/contexto	Configuración de seguridad, filtros y tests de autorización
O6	Separación modular de componentes	Bajo acoplamiento entre capas	Arquitectura frontend-backend + DTOs + servicios
O7	Calidad técnica cuantificable	Suites en verde y métricas altas	Cobertura, E2E y mutación reportadas
O8	Reproducibilidad de entrega	Pipeline estable para build/test/deploy	Workflows CI/CD y despliegue trazable

Cuadro 1.2: Criterios de evaluación del cumplimiento de objetivos

1.9– Criterios de cumplimiento y evidencia de logro

El cumplimiento de objetivos se evalúa mediante evidencia verificable, evitando formulaciones subjetivas.

1.10– Trazabilidad objetivos-requisitos-entregables

La trazabilidad constituye un criterio central de rigor académico. Cada objetivo se conecta con requisitos del ERS y con entregables técnicos ejecutados durante los sprints.

Objetivo	Requisitos relacionados	Backlog/Sprint	Entregable técnico
O1	RQF-005 a RQF-017	EP-04, EP-05 (S1)	Módulos de cursos, grupos y actividades
O2–O3	RQF-018 a RQF-026	EP-06, EP-07 (S1-S2)	Módulos de entregables y entregas
O4	RQF-027 a RQF-029	EP-09 (S1-S3)	Módulo de evaluación y feedback
O5	RQNF-009, REGN-001	EP-12 (S1)	JWT, control de roles y validación contextual
O7	RQNF-001, RQNF-004	EP-13 (S1-S4)	Testing en capas + mutación
O8	RQNF-008, RQNF-004	EP-14 (S1)	CI/CD y despliegue automatizado

Cuadro 1.3: Trazabilidad entre objetivos, requisitos y resultados técnicos

1.11– Riesgos estratégicos asociados a los objetivos

Al definir objetivos de forma ambiciosa, es necesario explicitar riesgos y medidas de mitigación.

- **Riesgo de sobrealcance funcional:** incorporar más funcionalidades de las asumibles en 660 horas. **Mitigación:** priorización MoSCoW y control por hitos.

- **Riesgo de deuda de calidad:** posponer testing para la fase final. **Mitigación:** estrategia de pruebas integrada en cada sprint.
- **Riesgo de incoherencia documental:** divergencia entre memoria y estado real del producto. **Mitigación:** redacción iterativa con trazabilidad objetiva.

1.12– Conclusiones del capítulo

La definición de objetivos presentada no se limita a una enumeración formal, sino que establece un marco de ejecución y validación completo.

En consecuencia, este capítulo fija la base de todo el desarrollo posterior: qué se pretende conseguir, por qué se prioriza de ese modo, qué evidencia demostrará su cumplimiento y cuáles son los límites deliberados del alcance en el contexto del TFG.

Análisis de requisitos, diseño e implementación

2.1– Introducción

Este capítulo integra tres perspectivas que en un desarrollo profesional no deben tratarse de forma aislada: la especificación de requisitos, el diseño técnico y la implementación efectiva del sistema.

El objetivo es demostrar trazabilidad de extremo a extremo: desde la necesidad identificada en el análisis inicial hasta su materialización en componentes software, persistencia, seguridad, pruebas y despliegue.

2.2– Ecosistema de desarrollo y operación

2.2.1. Lenguajes y herramientas utilizadas

El ecosistema técnico del proyecto se ha definido para garantizar coherencia entre desarrollo, validación y despliegue:

- **Implementación backend:** Java 21, Spring Boot, Spring Security, JPA/Hibernate, Maven y Flyway.
- **Implementación frontend:** React, TypeScript, Vite, React Router y Axios.
- **Documentación:** memoria en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ y documentación técnica auxiliar en Markdown.
- **Gestión de proyecto:** backlog y planificación iterativa versionados en el propio repositorio.

2.2.2. Política de gestión del código fuente

La gestión del código se realiza en repositorio Git con enfoque de ramas temáticas, integración mediante *pull requests* y validaciones automáticas previas a fusión. Esta política reduce riesgo de regresión, mejora trazabilidad de cambios y permite relacionar requisitos con commits, pruebas y despliegues.

2.2.3. Estrategia de integración continua y despliegue

La estrategia CI/CD se apoya en workflows de GitHub Actions, con etapas separadas para backend, frontend y validación conjunta:

- ejecución de compilación, pruebas y lint en cada cambio relevante,

- análisis de calidad y seguridad sobre artefactos de código,
- construcción de imágenes Docker y publicación automatizada,
- despliegue en PaaS con configuración por entorno.

El objetivo operativo es que cada versión candidata sea reproducible, auditable y desplegable con esfuerzo mínimo, manteniendo una línea de entrega continua compatible con el calendario del TFG.

2.2.4. Uso de asistentes de Inteligencia Artificial

El proyecto adopta un uso **limitado, transparente y verificable** de herramientas de IA generativa, siempre como apoyo a tareas de redacción preliminar, revisión o aceleración operativa, nunca como sustituto del criterio de ingeniería del autor. El alcance y las salvaguardas aplicadas se detallan en el Anexo D de esta memoria.

2.3– Análisis de requisitos

2.3.1. Fuentes de requisitos

La definición del catálogo de requisitos se ha construido a partir de las siguientes fuentes:

- Documento ERS (Especificación de Requisitos del Sistema).
- Documento DAS (Documento de Análisis del Sistema).
- Modelado estructurado y versionado en Proteus.
- Product Backlog y Sprint Backlog utilizados en la ejecución iterativa.
- Evidencia empírica obtenida durante pruebas funcionales y de integración.

Este planteamiento evita una especificación estática: los requisitos se formalizan, se priorizan, se implementan por incrementos y se vuelven a validar con el producto funcionando.

2.3.2. Estructura del catálogo

El catálogo se organiza en cuatro bloques complementarios:

- Requisitos funcionales de información y operación.
- Reglas de negocio que acotan el comportamiento permitido.
- Requisitos no funcionales de calidad del sistema.
- Restricciones e integraciones técnicas por entorno.

En Proteus se distinguen explícitamente requisitos funcionales (por ejemplo, RQF-001), reglas de negocio (REGN-001) y no funcionales (RQNF-001 a RQNF-011), lo que facilita su seguimiento durante la implementación.

2.3.3. Requisitos funcionales

El ERS define un abanico funcional amplio, del que se ha priorizado el núcleo necesario para el MVP. A nivel de dominio, la funcionalidad se agrupa en:

1. **Identidad y acceso:** autenticación de usuarios y control por rol.
2. **Estructura académica:** cursos, grupos, actividades y vinculaciones.
3. **Gestión de entregables:** configuración de fechas, visibilidad, reenvíos y estructura esperada.
4. **Gestión de entregas:** subida, historial, descarga y trazabilidad.
5. **Evaluación y feedback:** calificación, comentarios y publicación de notas.

Como ejemplos representativos:

- **RQF-001:** creación de usuarios por administrador.
- **RQF-014–RQF-017:** operaciones de entrega y versionado.
- **RQF-018–RQF-025:** evaluación, visibilidad e integraciones.

2.3.4. Reglas de negocio

Las reglas de negocio restringen de forma explícita qué acciones son admisibles. Una regla clave es **REGN-001**, que establece que solo el profesor autorizado puede asignar una calificación.

Esta norma se implementa en validaciones de servicio y controlador, evitando tanto la elevación de privilegios como la evaluación fuera de contexto (p. ej., docente no vinculado al grupo o actividad).

2.3.5. Requisitos no funcionales

Los requisitos no funcionales aportan condiciones de calidad del producto. La Tabla 2.1 resume los más relevantes para el alcance actual.

Código	Categoría	Aplicación en el sistema
RQNF-001	Fiabilidad	Mitigar pérdida de datos en operaciones críticas de entrega/evaluación.
RQNF-002	Usabilidad	Flujos entendibles para usuarios no técnicos, con navegación guiada por rol.
RQNF-004	Mantenibilidad	Bajo acoplamiento entre capas y separación estricta de responsabilidades.
RQNF-006	Eficiencia	Respuesta ágil en consultas frecuentes de panel, actividad y entregables.
RQNF-008	Portabilidad	Compatibilidad web en navegadores modernos de uso general.
RQNF-009	Seguridad	Exposición de datos condicionada por autenticación y autorización.

Cuadro 2.1: Requisitos no funcionales representativos

2.3.6. Priorización y alcance

La priorización se realizó con criterio MoSCoW y validación continua con el backlog. El foco del MVP se situó en los procesos con mayor impacto docente:

- gestión de actividades y entregables,
- flujo completo de entrega y evaluación,
- seguridad por rol,
- persistencia y trazabilidad de cambios.

Funcionalidades avanzadas (integraciones extendidas, autenticación federada completa, observabilidad más profunda) se mantienen en backlog de evolución y no comprometen la viabilidad del alcance de TFG.

2.3.7. Trazabilidad requisito-backlog-código

La trazabilidad se gestiona en tres niveles:

- **Nivel de análisis:** identificador en ERS/Proteus.
- **Nivel de planificación:** PBI y sprint asociado.
- **Nivel de ejecución:** módulo software y prueba que lo valida.

La Tabla 2.2 muestra ejemplos representativos.

Req.	PBI / Sprint	Implementación principal	Validación
RQF-001	Usuarios y roles / S1-S2	AuthController, Usuario-Service, seguridad por rol	Tests auth + integración
RQF-018	Crear entregable / S3-S4	EntregableController, EntregableService, Entregable	Unit + API tests
RQF-023	Subir entrega / S4-S5	EntregaController, EntregaService, MaterialService	Integración + E2E
REGN-001	Evaluación docente / S5	Validaciones en servicio de entregas y evaluación	Pruebas de autorización
RQNF-009	Seguridad global / transversal	SecurityConfig, JwtAuthenticationFilter, RateLimitFilter	Tests de seguridad

Cuadro 2.2: Ejemplo de trazabilidad entre requisitos, backlog e implementación

2.3.8. Contexto documental de partida (ERS y DAS)

La especificación documental de este TFG no se apoya en descripciones ad hoc, sino en dos artefactos de análisis formal mantenidos en Proteus:

- **ERS (Especificación de Requisitos del Sistema):** define situación actual, necesidades de negocio, subsistemas y catálogo de requisitos (generales, de información, funcionales, no funcionales y reglas de negocio).

- **DAS (Documento de Análisis del Sistema):** define arquitectura lógica, modelo de clases, diagramas de estado, diagramas de secuencia, navegabilidad e inventario de mockups de interfaz.

La reutilización de ambos documentos en esta memoria persigue dos objetivos: primero, preservar trazabilidad respecto al trabajo original de Ingeniería de Requisitos; segundo, demostrar que las decisiones de diseño e implementación adoptadas durante el TFG responden a una base de especificación explícita.

2.3.9. Situación actual y necesidades de negocio (ERS)

El ERS identifica un escenario de partida con fricción operativa notable: creación de actividades y recogida de entregas con alto componente manual, dificultad para mantener trazabilidad uniforme y baja capacidad de evolución cuando se pretende introducir reglas de negocio más finas.

Sobre ese diagnóstico, las necesidades de negocio priorizadas fueron:

- reducir carga administrativa en procesos docentes recurrentes,
- dar al estudiante un flujo de entrega más guiado y transparente,
- centralizar evaluación y feedback con reglas de autorización claras,
- mejorar mantenibilidad del sistema y su capacidad de evolución.

Estas necesidades se conectan con el resto de la memoria en tres planos: arquitectura (separación de capas), implementación (módulos especializados) y verificación (pruebas por flujo crítico y por requisito).

2.3.10. Descomposición en subsistemas (ERS)

El ERS descompone el sistema en cinco subsistemas. Esta descomposición ha sido reutilizada como guía de diseño modular durante el desarrollo.

Código	Subsistema	Responsabilidad principal
SUBS-001	SIU (Interacción con el Usuario)	Portal web y vistas por rol para alumno, profesor y administrador.
SUBS-002	SGUS (Usuarios y Seguridad)	Autenticación, autorización por rol y control de acceso contextual.
SUBS-003	SGA (Gestión Académica)	Cursos, actividades, entregables, evaluación y feedback.
SUBS-004	SPA (Procesamiento de Archivos)	Validación técnica de ficheros, organización y versionado de entregas.
SUBS-005	SPD (Persistencia de Datos)	Persistencia relacional y repositorio de archivos con trazabilidad.

Cuadro 2.3: Subsistemas definidos en ERS y reutilizados en el diseño del TFG

2.3.11. Catálogo exhaustivo de requisitos del ERS

Con objeto de incorporar de forma completa el contenido de ERS, se documenta el inventario de requisitos por tipología y su interpretación en el alcance real del TFG.

Código	Nombre	Descripción sintética
RQG-001	Gestión de usuarios y roles	Alta de usuarios con rol (administrador, profesor, alumno) y permisos diferenciados por perfil.
RQG-002	Gestión de actividades y entregables	Modelado de actividades evaluables/no evaluables y entregables con reglas propias.
RQG-003	Acceso de alumnos a entregables	Acceso de estudiante a entregables visibles y registro temporal de acciones.
RQG-004	Gestión de versiones y reenvíos	Control de versiones de entrega y soporte de reenvío cuando el docente lo habilita.
RQG-005	Configuración de actividades y entregables	Definición de fechas, formatos y condiciones de publicación por contexto académico.
RQG-006	Comunicación y feedback	Canal de comunicación docente-estudiante vinculado a la evaluación de entregas.
RQG-007	Seguridad y control de acceso	Restricción de visibilidad/operación por identidad, rol y pertenencia a curso-grupo.

Cuadro 2.4: Inventario de requisitos generales (RQG) en ERS

Código	Nombre	Descripción sintética
RQI-001	Información sobre actividades	Persistencia de metadatos de actividades creadas por profesorado.
RQI-002	Información sobre entregables	Persistencia de configuración de cada entregable y su vinculación con actividad.
RQI-003	Información de usuarios	Registro de identidad, rol y datos de operación por tipo de usuario.
RQI-004	Información de cursos	Persistencia de estructura académica (curso/grupo y relaciones).
RQI-005	Información de feedback	Persistencia de calificaciones, comentarios y estado de publicación.

Cuadro 2.5: Inventario de requisitos de información (RQI) en ERS

Requisitos generales (RQG)**Requisitos de información (RQI)****Requisitos funcionales (RQF)****Reglas de negocio (REGN)****Requisitos no funcionales (RQNF)****2.3.12. Reutilización de diagramas y mockups del DAS**

El DAS aporta una base visual amplia (diagramas técnicos, secuencias, navegabilidad y mockups) que se reutiliza en esta memoria para explicar de forma verificable cada decisión de diseño e implementación.

Adicionalmente, el DAS incluye un inventario de dieciséis mockups que cubre las operaciones esenciales por rol. Para mantener equilibrio entre exhaustividad y legibilidad, su trazado completo se resume en la Tabla 2.11.

2.3.13. Lectura técnica de los diagramas estructurales del DAS

La reutilización de las Figuras 2.1, 2.2, 2.3 y 2.4 no se limita a un uso ilustrativo, sino que establece una línea de diseño que condiciona decisiones de implementación en backend, frontend y datos.

En particular, la arquitectura lógica permite justificar la separación entre interacción con usuario, seguridad, gestión académica, procesamiento de archivos y persistencia. Este reparto reduce

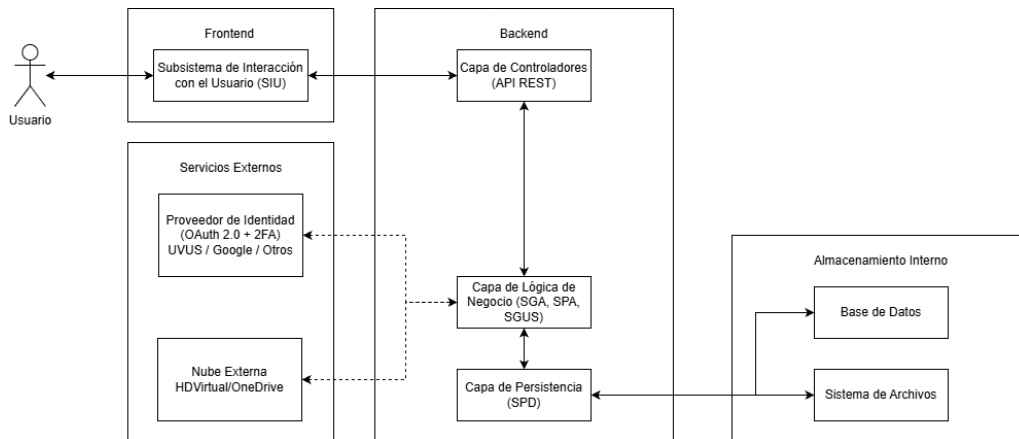


Figura 2.1: Arquitectura lógica del sistema (DAS)

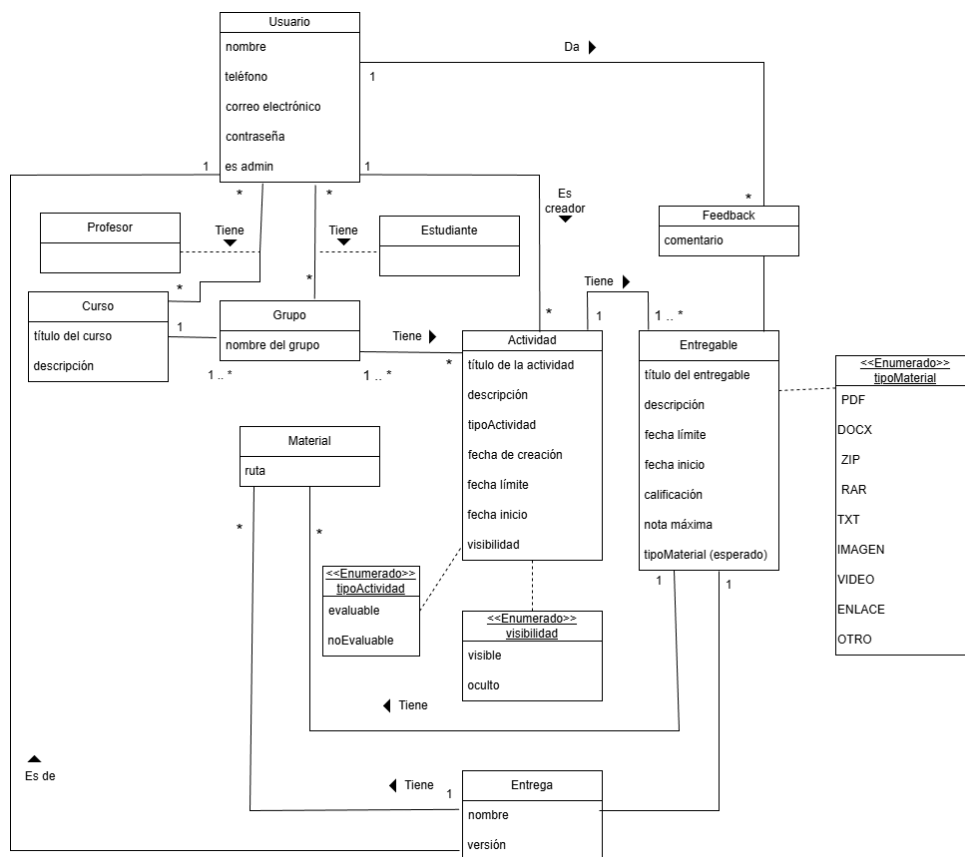


Figura 2.2: Modelo de clases del sistema (DAS)

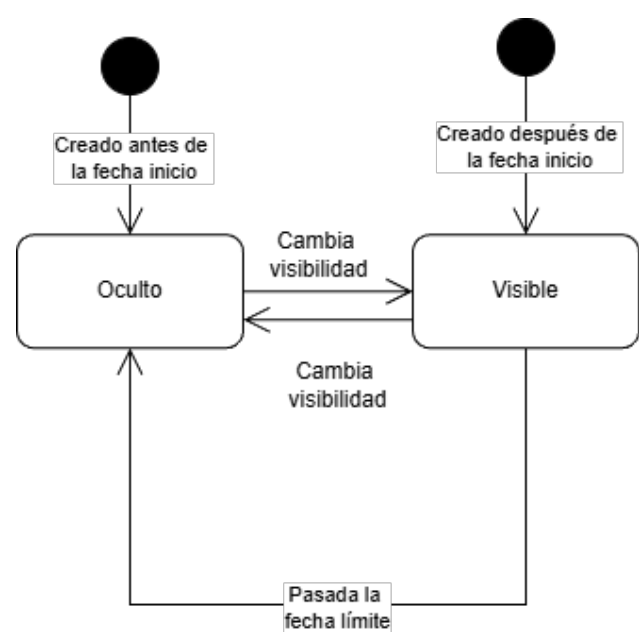


Figura 2.3: Diagrama de estado de actividades (DAS)

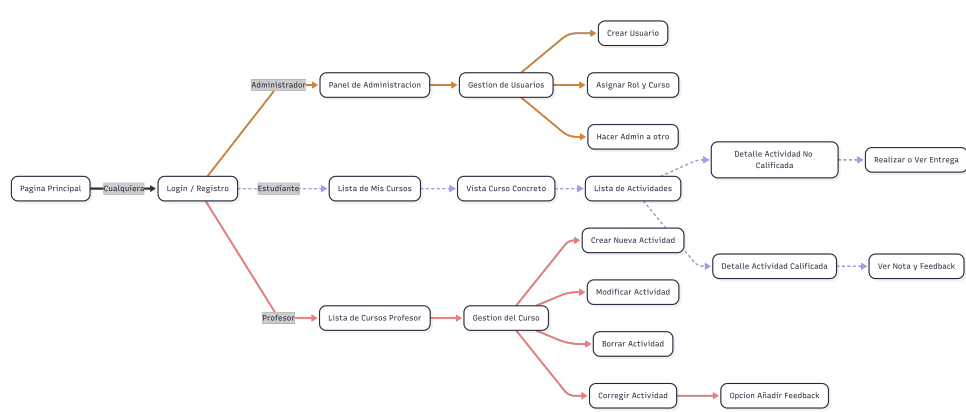


Figura 2.4: Navegabilidad principal de la interfaz (DAS)

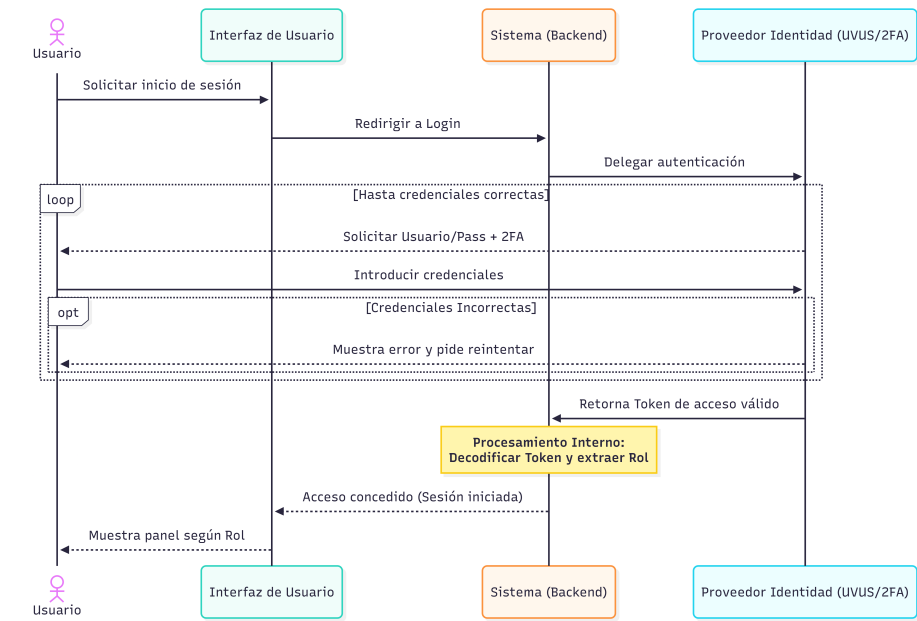


Figura 2.5: Secuencia de autenticación y acceso (DAS)

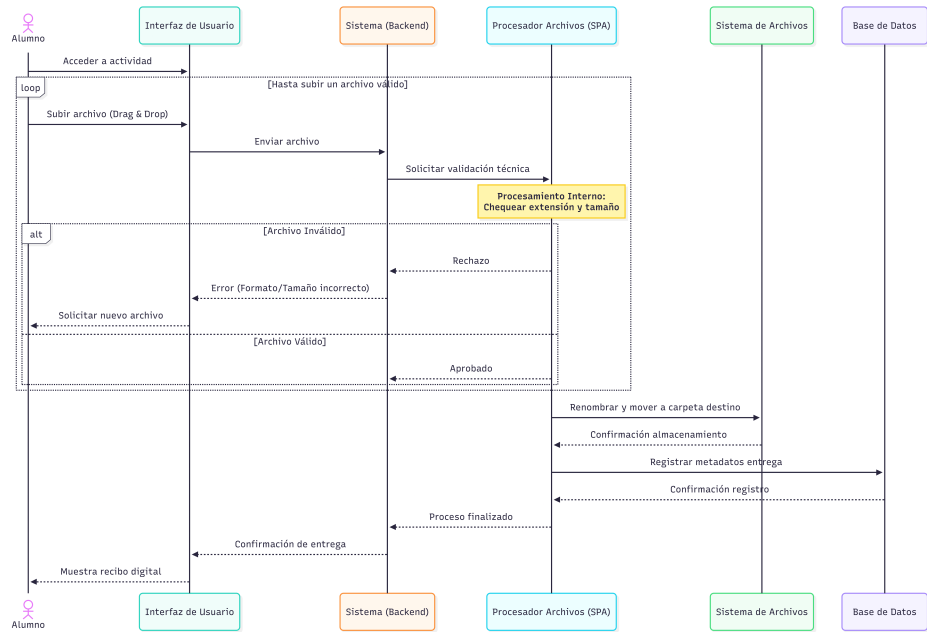


Figura 2.6: Secuencia de realización de entrega (DAS)

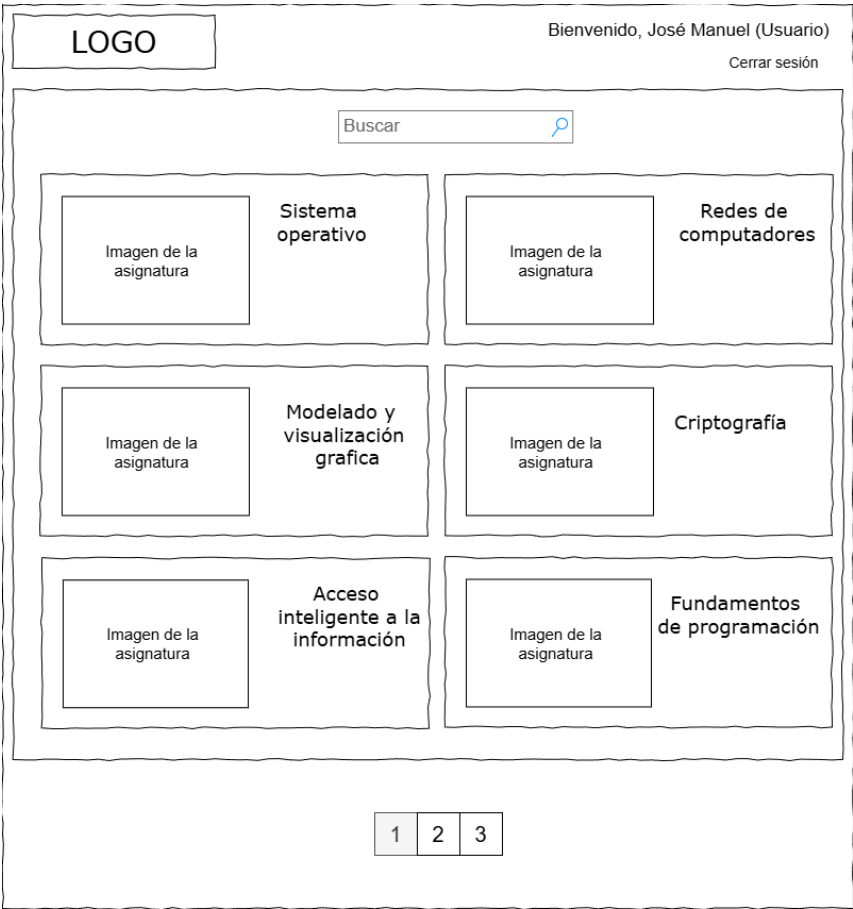


Figura 2.7: Mockup de panel principal de estudiante (DAS)

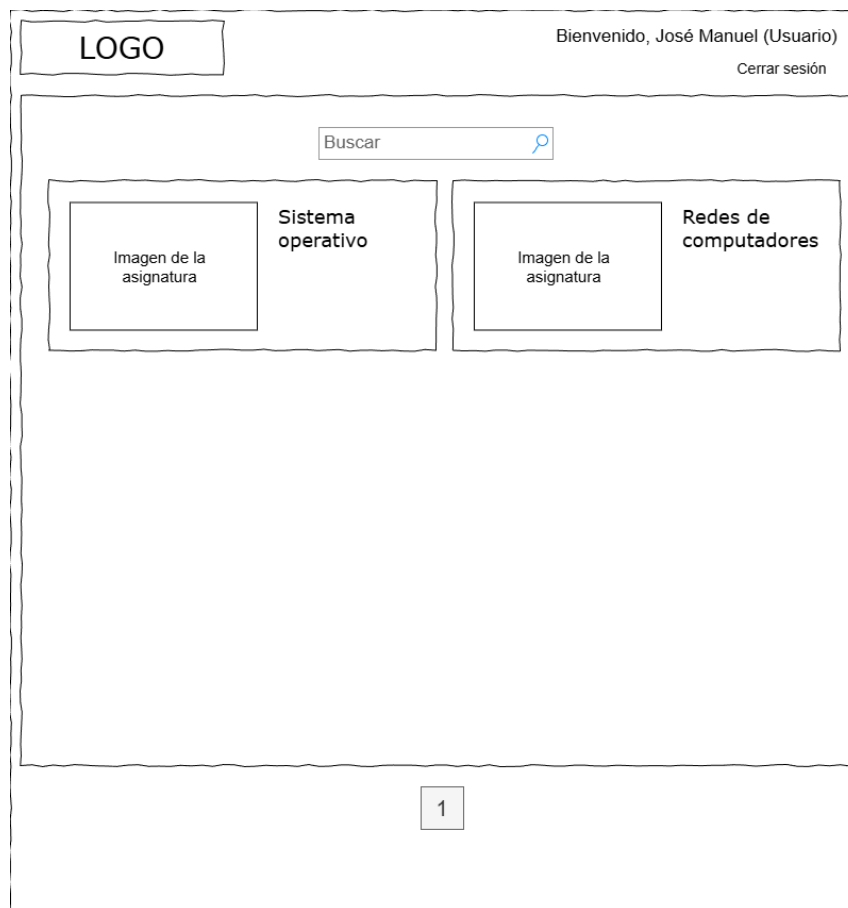


Figura 2.8: Mockup de panel principal de profesor (DAS)

Código	Nombre	Descripción sintética
RQF-001	Crear usuarios	Alta de usuarios por administrador con asignación de rol.
RQF-002	Iniciar sesión	Autenticación de usuarios en la plataforma.
RQF-003	Editar usuarios	Modificación de datos de usuario según permisos.
RQF-004	Eliminar usuarios	Baja de usuarios por perfil administrador.
RQF-005	Crear actividades	Creación de actividad académica por docente.
RQF-006	Visibilizar actividades	Publicación/ocultación de actividades para estudiantes.
RQF-007	Proporcionar adjuntos	Asociación de material de apoyo a actividad/entregable.
RQF-008	Modificar actividades	Edición de configuración de actividad por docente propietario.
RQF-009	Eliminar actividades	Eliminación de actividad según reglas de propiedad.
RQF-010	Ver actividades	Consulta de actividades por rol y contexto académico.
RQF-011	Crear entregables	Alta de entregables dentro de actividad.
RQF-012	Modificar entregables	Edición de entregable por docente autorizado.

Cuadro 2.6: Inventario de requisitos funcionales (RQF-001 a RQF-012)

acoplamiento transversal, facilita pruebas por componente y mejora mantenibilidad evolutiva.

El modelo de clases se emplea como referencia para mantener coherencia entre entidades persistentes, DTOs y contratos de API. Del mismo modo, el diagrama de estado de actividades se refleja en validaciones de servicio que evitan transiciones no permitidas (por ejemplo, publicar, ocultar o cerrar fuera de las reglas definidas).

Por último, el diagrama de navegabilidad refuerza la consistencia entre estructura funcional y experiencia de usuario por rol, evitando rutas incoherentes con las restricciones de autorización descritas en ERS.

2.3.14. Flujos operativos del DAS y su encaje en la solución

Además de los diagramas estructurales, el DAS aporta una visión de comportamiento detallada mediante secuencias y mockups operativos. Esta capa es especialmente relevante para documentar el recorrido completo de profesor y estudiante en los procesos críticos del MVP.

El flujo de creación de actividad docente se explicita en la Figura 2.9. En ella se observa la coordinación entre interfaz, validaciones de negocio y persistencia, alineada con RQF-005, RQF-008 y RQF-011.

El ciclo de evaluación y retroalimentación queda recogido en la Figura 2.10, que sirve de base para justificar la integración entre calificación, comentarios y publicación de resultados, en coherencia con RQF-018, RQF-019 y REGN-001/REGN-008.

En la dimensión de interfaz, los mockups de trabajo muestran cómo se materializa la navegación por rol en escenarios reales de uso. Las Figuras 2.11 y 2.12 capturan dos momentos clave del producto: seguimiento de actividad por el estudiante y corrección detallada por el profesorado.

Como apoyo adicional, la Figura 2.13 ilustra el estado de entrega con contenido adjunto, útil para justificar la trazabilidad de versiones y el control de evidencias de envío.

2.3.15. Catálogo operativo de casos de uso y cobertura funcional

El modelo de casos de uso del DAS se estructura como un conjunto amplio de operaciones del sistema que cubren autenticación, gestión académica, entregas, evaluación y supervisión docente. Esta descomposición se reutiliza en la memoria para demostrar que la implementación del MVP no es una selección arbitraria de pantallas, sino la materialización de un modelo operativo previo.

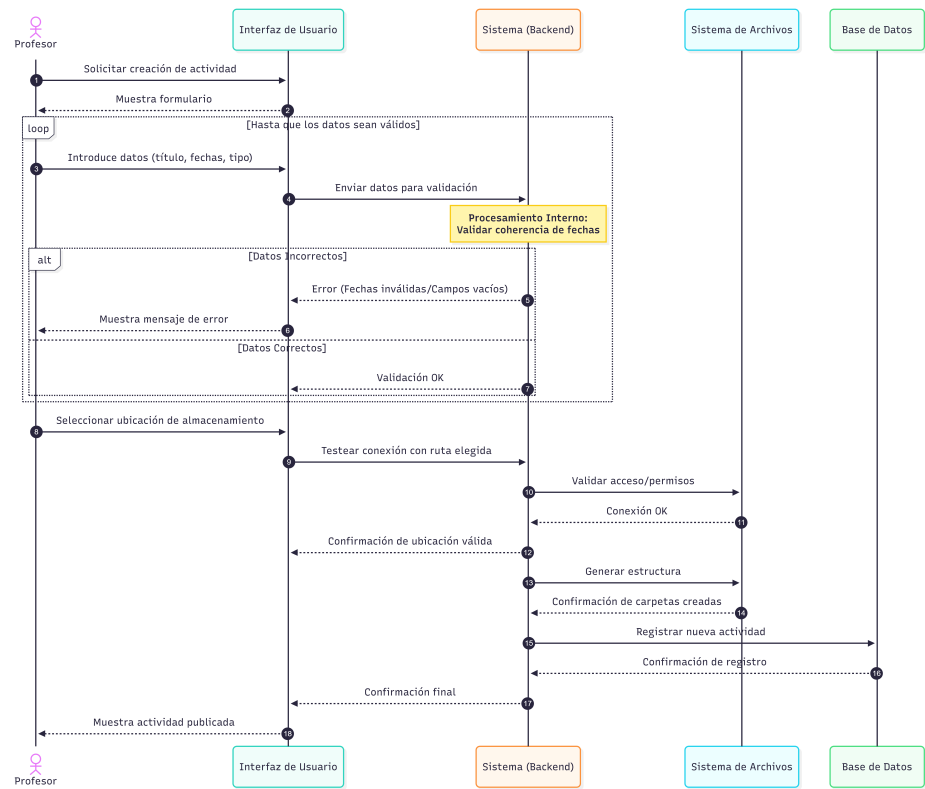


Figura 2.9: Secuencia de creación de actividad en backend (DAS)

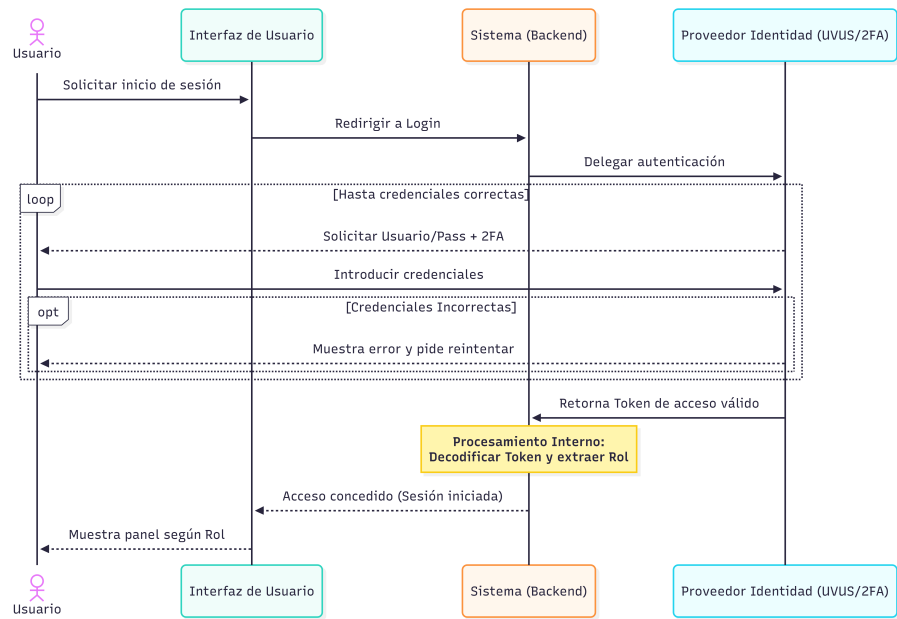


Figura 2.10: Secuencia de evaluación y feedback (DAS)

Logo

Bienvenido, José Manuel (Usuario)
Cerrar sesión

Imagen de la asignatura

Sistema operativo

Buscar

Enseñar entregables no disponibles ☐

 Prueba de comandos

Fecha de vencimiento: 7/2/2027 20:11

Iniciar entrega

No entregado

1

Entregables disponibles

Calificaciones

Figura 2.11: Mockup operativo: estudiante dentro de una asignatura (DAS)

LOGO

Bienvenido, José Manuel (Usuario)
Cerrar sesión

Imagen de la
asignatura

Sistema operativo

Prueba de comandos

Fecha de vencimiento: 7/2/2024 20:11

Detalles:

Examen para ver que conocimientos se conocen de Linux

Memoria:  Memoria_VNK5300.pdf [Descargar](#)

Una función *random* (o función aleatoria) es aquella que genera resultados que no siguen un patrón fijo, sino que varían de manera impredecible dentro de un rango definido.

En programación, suele referirse a funciones que devuelven **números pseudoaleatorios**, es decir, valores que parecen aleatorios pero en realidad son generados por un algoritmo matemático a partir de una "semilla" inicial.

Sin calificar

Calificar

Poner comentario

Asignaturas

Estudiantes

Figura 2.12: Mockup operativo: detalle de calificación como profesor (DAS)

LOGO

Bienvenido, José Manuel (Usuario)
Cerrar sesión

Imagen de la
asignatura

Sistema operativo

Prueba de comandos

Fecha de vencimiento: 7/2/2024 20:11

Detalles:

Primera entrega

Examen para ver que conocimientos se conocen de Linux

Memoria:

Memoria_VNK5300.pdf

Eliminar

Una función random (o función aleatoria) es aquella que genera resultados que no siguen un patrón fijo, sino que varían de manera impredecible dentro de un rango definido.

En programación, suele referirse a funciones que devuelven números pseudoaleatorios, es decir, valores que parecen aleatorios pero en realidad son generados por un algoritmo matemático a partir de una "semilla" inicial.

Entregar

Asignaturas

Estudiantes

Figura 2.13: Mockup de entrega con archivo adjunto (DAS)

Código	Nombre	Descripción sintética
RQF-013	Eliminar entregable	Baja de entregable por docente autorizado.
RQF-014	Realizar entrega	Subida de archivo por estudiante para un entregable habilitado.
RQF-015	Enviar actividad	Confirmación de envío de actividad completa.
RQF-016	Versionar entregables	Mantenimiento de historial de versiones por entregable.
RQF-017	Versionar actividades	Mantenimiento de versiones de resolución de actividad.
RQF-018	Dar feedback	Publicación de comentarios docentes sobre entrega/actividad.
RQF-019	Realizar calificación	Asignación de nota por profesor autorizado.
RQF-020	Ver entregables	Consulta de entregas y versiones por rol permitido.
RQF-021	Crear curso	Alta de curso/asignatura por administración.
RQF-022	Modificar curso	Edición de datos de curso por administración.
RQF-023	Vista de estudiante	Visualización por profesor de la experiencia de alumno.
RQF-024	Visibilizar entregable	Publicación/ocultación de entregables concretos.
RQF-025	Integración cloud	Uso de almacenamiento en nube según configuración de entorno.

Cuadro 2.7: Inventario de requisitos funcionales (RQF-013 a RQF-025)

Como representación visual de dicha cobertura, se incluyen cuatro diagramas de casos de uso de referencia (Figuras 2.14 a 2.17), suficientes para documentar el enlace entre el catálogo funcional y los flujos reales desarrollados.

2.3.16. Matriz ampliada de trazabilidad ERS-DAS-implementación

Para reforzar la coherencia documental, la Tabla 2.15 integra tres niveles de evidencia: requisito formal en ERS, artefacto de diseño en DAS y materialización en componentes software y pruebas del proyecto.

2.3.17. Cobertura documental alcanzada

Con la incorporación anterior, este capítulo integra de forma explícita:

- los cinco bloques completos del catálogo ERS (RQG, RQI, RQF, REGN y RQNF),
- la descomposición en subsistemas usada durante diseño e implementación,
- los diagramas estructurales y de comportamiento del DAS,
- el catálogo operativo de casos de uso y familias funcionales del sistema,
- el conjunto de mockups de interfaz como evidencia de diseño funcional,
- y una matriz ampliada de trazabilidad entre especificación, diseño, implementación y verificación.

De esta forma, la memoria no solo resume resultados: también preserva la trazabilidad documental completa entre especificación inicial, diseño técnico y comportamiento implementado.

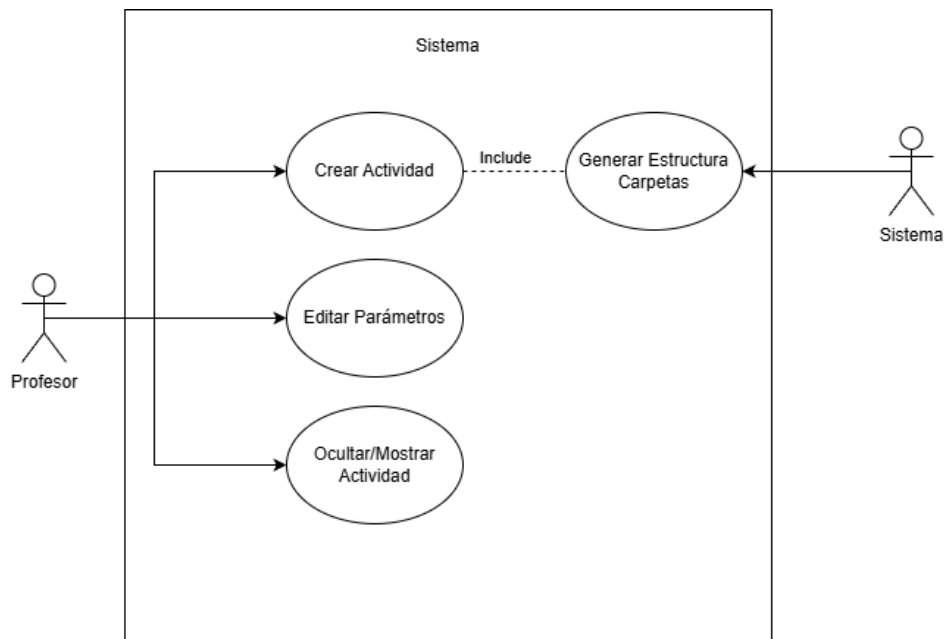


Figura 2.14: Caso de uso representativo CU-1 (DAS)

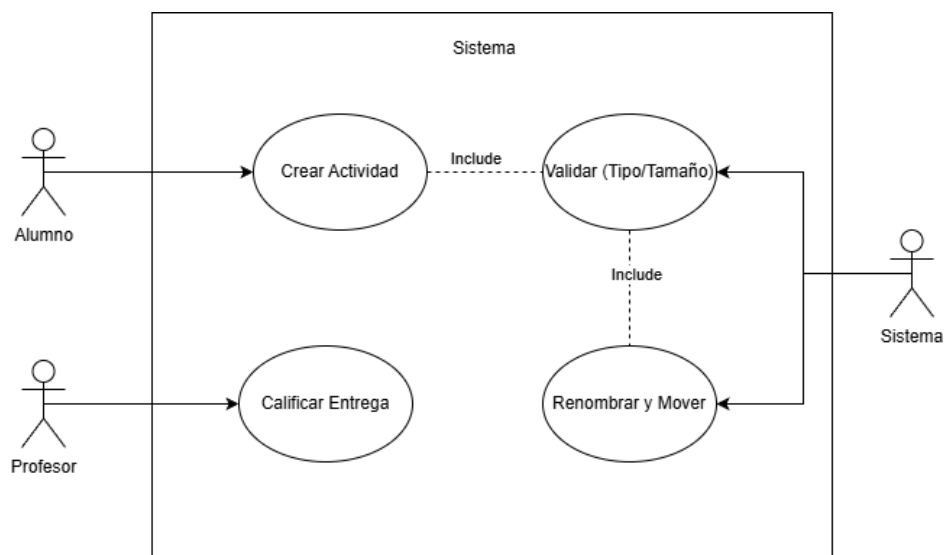


Figura 2.15: Caso de uso representativo CU-2 (DAS)

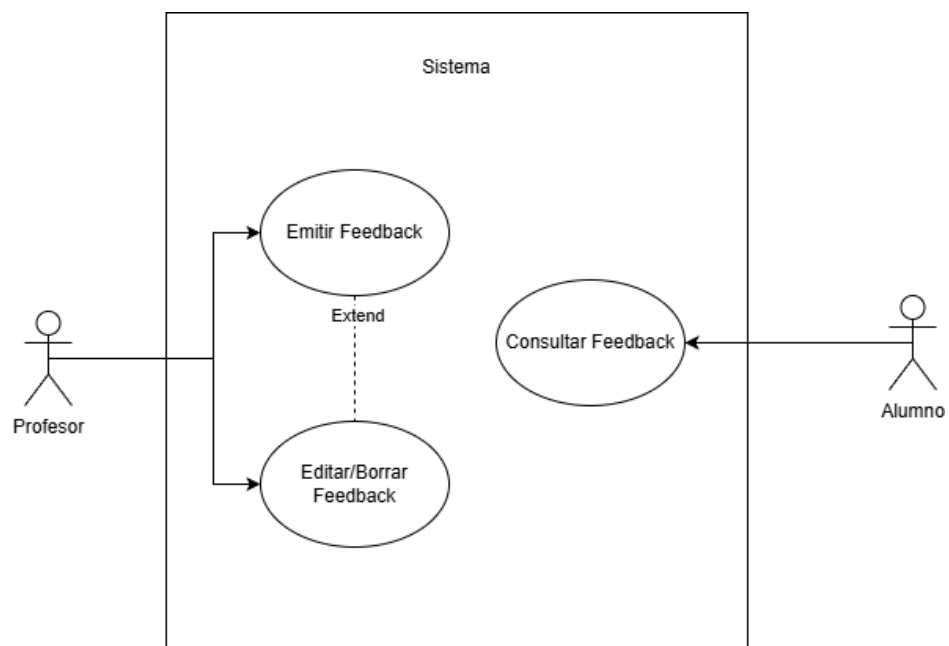


Figura 2.16: Caso de uso representativo CU-3 (DAS)

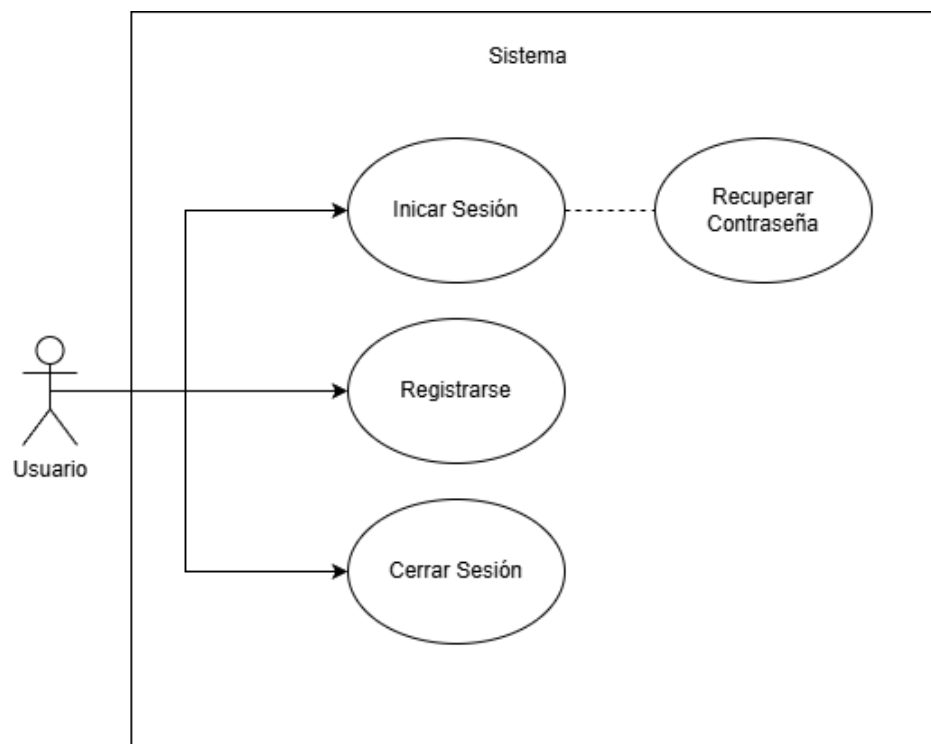


Figura 2.17: Caso de uso representativo CU-4 (DAS)

Código	Nombre	Regla de negocio
REGN-001	Corrección de entregas	Solo el profesor asignado al entregable puede emitir nota.
REGN-002	Visibilidad de entregas	El alumno solo accede a sus propias entregas/-versiones.
REGN-003	Corrección de actividades	Solo el profesor asignado a la actividad puede calificar.
REGN-004	Visibilidad de actividades	El alumno solo visualiza sus propias resoluciones de actividad.
REGN-005	Fecha límite de entrega	No se admiten envíos fuera de plazo salvo configuración explícita.
REGN-006	Recursos no visibles	Se deniega interacción del alumno con elementos ocultos.
REGN-007	Creación de recursos	Solo profesorado crea actividades y entregables.
REGN-008	Emisión de feedback	Solo el docente autorizado publica feedback evaluativo.

Cuadro 2.8: Inventario de reglas de negocio (REGN) en ERS

2.4– Diseño del sistema

2.4.1. Arquitectura de alto nivel

La arquitectura adoptada es cliente-servidor desacoplada:

- **Frontend SPA** en React + TypeScript (presentación y experiencia de usuario).
- **Backend REST** en Spring Boot (lógica de negocio y seguridad).
- **Persistencia relacional** con JPA/Hibernate y migraciones Flyway.

En backend se aplica diseño por capas (controller, service, repository y entity), que mejora mantenibilidad, testabilidad y capacidad de evolución.

2.4.2. Vista de despliegue

La vista de despliegue se organiza en cuatro bloques principales:

- **Cliente web:** navegador del usuario final (profesor, estudiante o administrador).
- **Frontend:** SPA estática servida desde plataforma PaaS (Render).
- **Backend:** API REST desplegada como servicio web Docker en Render.
- **Datos:** base de datos PostgreSQL gestionada y accesible solo desde backend.
- **Servicios externos:** almacenamiento cloud, OneDrive y correo transaccional por API HTTP.

La separación de componentes permite escalar y evolucionar cada capa sin romper el resto del sistema, manteniendo además una frontera de seguridad clara entre cliente, lógica de negocio y persistencia.

Código	Nombre	Condición de calidad
RQNF-001	Tolerancia a fallos	Persistencia local temporal ante fallos de conectividad para evitar pérdida de datos.
RQNF-002	Intuitividad	Interfaz comprensible para nuevos usuarios con patrones de uso habituales.
RQNF-003	Uniformidad	Coherencia visual y de estilo con directrices de marca institucional.
RQNF-004	Acoplamiento bajo	Separación de responsabilidades con dependencias estrictamente necesarias.
RQNF-005	Cohesión alta	Organización interna de módulos orientada a mantenimiento evolutivo.
RQNF-006	Tiempo de carga	Tiempo de transición objetivo inferior a 2 segundos en condiciones óptimas.
RQNF-007	Tiempo de búsqueda	Respuesta rápida en consultas frecuentes y minimización de búsquedas redundantes.
RQNF-008	Compatibilidad de navegadores	Soporte para navegadores modernos de uso común.
RQNF-009	Exposición de información	Visualización de datos restringida a usuarios autenticados y autorizados.
RQNF-010	Encriptación	Protección criptográfica de información personal/sensible.
RQNF-011	Portabilidad web	Compatibilidad explícita con Chrome, Firefox y Edge en escritorio/móvil.

Cuadro 2.9: Inventario de requisitos no funcionales (RQNF) en ERS

Artefacto DAS	Archivo original en assets	Uso en memoria
Arquitectura lógica	ArquitecturaTFG.drawio.png	Diseño global
Diagrama de clases	Diagrama de clases.png	Modelo de dominio
Diagrama de estado	Diagrama de estado.png	Ciclo de vida
Navegabilidad	Diagrama de navegabilidad (1).png	Flujo de interfaz
Secuencia autenticación	Autenticacion y Acceso.png	Flujo de acceso
Secuencia crear actividad	Backend Activity Creation-2026-03-18-161459.png	Flujo docente
Secuencia realizar entrega	Realizar Entrega.png	Flujo estudiante
Secuencia evaluación	Evaluacion y Feedback.png	Flujo de corrección

Cuadro 2.10: Diagramas principales reutilizados desde DAS

2.4.3. Diseño de dominio y datos

El dominio se modela alrededor de la relación docente-estudiante-actividad:

- Usuario como identidad base.
- Profesor y Estudiante como especializaciones operativas.
- Curso, Grupo y Actividad para contexto académico.
- Entregable y Entrega para el ciclo de entrega.
- Feedback, Material y entidades de soporte (tokens, notificaciones).

La persistencia evoluciona de forma versionada mediante migraciones, lo que permite incorporar nuevas capacidades sin perder consistencia histórica.

Archivo de mockup	Escenario principal representado
InterfazDeUsuarioTFG-InicioDeSesión.drawio.png	Inicio de sesión y entrada al sistema.
InterfazDeUsuarioTFG-Registrarse.drawio.png	Registro o alta inicial de usuario.
InterfazDeUsuarioTFG-LogueadoComoEstudiante.png	Panel inicial de estudiante autenticado.
VistaEstudianteDentroDeUna-Asignatura.png	Vista de asignatura desde rol estudiante.
LogueadoComoEstudiante-SeleccionandoCheckbox.png	Selección de entregables por estudiante.
LogueadoComoEstudianteEn-Calificaciones.png	Consulta de calificaciones y feedback.
LogueadoComoProfesor.png	Panel inicial de profesor autenticado.
VistaProfesorEnAsignatura.png	Vista de asignatura desde rol profesor.
Creacion de actividad.png	Creación de actividad académica.
Edición de actividad.png	Edición de actividad existente.
Realizar calificaciones.png	Proceso de calificación por docente.
DentroDeCalificacionComo-Profesor.png	Vista detallada de corrección.
EntregaSinNadaPuesto.png	Estado inicial de entrega sin adjuntos.
EntregaConAlgoPuesto.png	Estado de entrega con archivo adjunto.
PonerComentarios.png	Edición de comentarios de evaluación.
PaginaEstudiantesProfesores.png	Consulta de estudiantes y trazabilidad docente.

Cuadro 2.11: Inventario de mockups de interfaz reutilizados desde DAS

2.4.4. Diseño de API

La API sigue orientación REST y separa contratos mediante DTOs. Se definen módulos de endpoint coherentes por contexto de negocio:

- `/api/auth`: login, refresh, recuperación de contraseña.
- `/api/actividades`: gestión de actividades y consulta por usuario.
- `/api/entregables`: administración de entregables.
- `/api/entregas`: subida, revisión, evaluación y descarga de entregas.

El uso de DTOs evita exponer directamente entidades de persistencia, reduce acoplamiento y simplifica evoluciones de contrato.

2.4.5. Diseño de seguridad

El diseño de seguridad incorpora mecanismos complementarios:

- autenticación basada en JWT (token de acceso + refresh token),
- filtros de validación en la cadena de seguridad,
- autorización por rol y contexto académico,
- cifrado de contraseñas con BCrypt,
- limitación de tasa en operaciones sensibles de autenticación.

Se configura CORS de forma explícita y se diferencia entre rutas públicas y protegidas, minimizando superficie de ataque sin comprometer usabilidad.

Artefacto DAS	Decisión de diseño derivada	Materialización técnica	Requisitos ERS reforzados
Arquitectura lógica	Separación funcional en subsistemas	Frontend SPA + API REST + servicios por dominio + repositorios	SUBS-001 a SUBS-005, RQNF-004
Diagrama de clases	Normalización del dominio académico	Entidades Usuario, Curso, Actividad, Entregable, Entrega y Feedback	RQI-001 a RQI-005, RQF-005 a RQF-020
Diagrama de estado	Control explícito del ciclo de vida	Validaciones de transición y reglas temporales en servicios	REGN-005, REGN-006, RQF-006, RQF-024
Navegabilidad	Flujos de interfaz por rol y contexto	Rutas protegidas, vistas diferenciadas y control de visibilidad	RQG-001, RQG-007, RQNF-002, RQNF-009

Cuadro 2.12: Interpretación técnica de los diagramas estructurales del DAS

Actor	Operación central	Evidencia DAS reutilizada	Requisitos ERS asociados
Administrador	Alta y administración de identidades/cursos	Mockups de alta de usuario y creación de curso	RQF-001 a RQF-004, RQF-021, RQF-022
Profesor	Diseño y publicación de actividad/entregable	Fig. 2.9 y mockup de creación de actividad	RQF-005 a RQF-013, REGN-007
Estudiante	Entrega y seguimiento de estado	Fig. 2.6, Fig. 2.13	RQF-014 a RQF-017, REGN-005
Profesor evaluador	Corrección y feedback	Fig. 2.10, Fig. 2.12	RQF-018, RQF-019, REGN-001, REGN-008
Profesor y estudiante	Consulta y trazabilidad de resultados	Mockups de comentarios y vista cruzada docente-estudiante	RQF-020, RQI-005, RQNF-009

Cuadro 2.13: Cobertura operativa por rol con evidencia de DAS y requisitos de ERS

2.4.6. Diseño del frontend

El frontend se organiza en módulos `pages`, `components`, `context` y `services`, con los siguientes objetivos:

- experiencia coherente para profesor y estudiante,
- protección de rutas según sesión y rol,
- gestión centralizada de autenticación,
- manejo uniforme de errores y estados de carga.

La capa de red encapsula llamadas HTTP y gestión de tokens, incluyendo renovación automática de sesión al expirar el token de acceso.

Familia operativa	Alcance funcional en DA-S/ERS	Actor dominante	Cobertura en MVP
Acceso y sesión	Inicio de sesión, control de identidad, continuidad de sesión	Todos los roles	Completa
Administración académica base	Alta/modificación de cursos, usuarios y contexto de trabajo	Administrador	Completa
Diseño docente de actividades	Creación, edición, visibilidad y configuración de entregables	Profesor	Completa
Gestión de entregas	Publicación de trabajo, versionado y consulta de historial	Estudiante	Completa
Evaluación y feedback	Calificación, comentarios y visibilidad de resultados	Mixto	Completa
Supervisión y trazabilidad	Vistas cruzadas profesor-estudiante y revisión del estado de entregas	Profesor	Parcial avanzada

Cuadro 2.14: Cobertura por familias de casos de uso del DAS dentro del alcance del MVP

2.5– Implementación

2.5.1. Implementación backend

La implementación backend se estructura en el paquete principal de la aplicación, donde destacan:

- **Controladores:** entrada HTTP y validación de contratos.
- **Servicios:** lógica de negocio y reglas del dominio.
- **Repositorios:** acceso a datos con Spring Data JPA.
- **Seguridad:** configuración central y filtros especializados.

Un punto clave es `EntregaService`, que centraliza validaciones de entrega, control de versiones, restricciones temporales y operativa de evaluación, con apoyo de servicios auxiliares para materiales y notificaciones.

2.5.2. Implementación de flujos críticos

Los flujos con mayor impacto funcional son:

1. **Autenticación y sesión:** login, emisión de tokens, refresco y cierre seguro de sesión.
2. **Publicación de entregables:** creación por docente, configuración de fechas y condiciones de entrega.
3. **Entrega de material:** subida de ficheros, verificaciones, versionado y almacenamiento de metadatos.
4. **Evaluación:** corrección por profesor autorizado, feedback y visibilidad de nota.

Cada flujo incorpora validaciones de entrada, control de errores y verificación de permisos, asegurando consistencia en escenarios concurrentes y previniendo acciones fuera de alcance.

Bloque ERS	Artefacto DAS reutilizado	Implementación principal	Evidencia de verificación
RQG-001, RQG-007	Arquitectura + navegabilidad + mockups de sesión	Configuración de seguridad, filtros JWT, control de rutas	Tests de autenticación/autorización
RQI-001 a RQI-005	Diagrama de clases + vistas por rol	Entidades JPA, DTOs y contratos API	Pruebas de persistencia e integración
RQF-005 a RQF-013	Secuencia de creación docente + mockups de actividad	Servicios de actividad/entregable y controladores asociados	Unit tests y API tests de gestión docente
RQF-014 a RQF-017	Secuencia de entrega + estados de mockup	Servicio de entregas, validaciones y versionado	Integración de entregas y E2E de alumno
RQF-018, RQF-019	Secuencia de evaluación + detalle de calificación	Flujo de feedback/calificación en backend y frontend	Pruebas funcionales de evaluación
REGN-001 a REGN-008	Casos de uso por rol + navegabilidad	Validaciones contextuales en servicios y filtros	Pruebas de reglas de negocio
RQNF-002, RQNF-003	Inventario de mockups y patrones de interfaz	Diseño de pantallas y mensajes de estado	E2E sobre flujos completos
RQNF-004, RQNF-005, RQNF-009	Arquitectura de subsistemas + separación de capas	Estructura modular backend/frontend + control de acceso	CI/CD, test suites y auditoría técnica

Cuadro 2.15: Trazabilidad ampliada entre ERS, DAS e implementación efectiva

2.5.3. Implementación frontend

La implementación del cliente cubre pantallas y operaciones necesarias para el MVP:

- acceso y gestión de sesión,
- paneles de actividades y entregables,
- formularios de entrega y seguimiento,
- vista de evaluaciones y retroalimentación.

Se han introducido medidas prácticas para mejorar usabilidad: mensajes de validación temprana, estados visuales de proceso, y comportamiento resiliente ante errores de red.

2.5.4. Persistencia, migraciones e integraciones

Se utiliza PostgreSQL en escenarios de despliegue y un perfil de desarrollo local para iteración rápida. Flyway gestiona la evolución del esquema, incluyendo ampliaciones como:

- atributos para validación estructural de entregas ZIP,
- campos para integración cloud (OneDrive/Cloudinary),
- tablas y preferencias de notificación,
- tokens de recuperación de contraseña.

Nodo	Tecnología	Responsabilidad principal
Cliente	Navegador moderno	Interacción de usuario y consumo de API sobre HTTPS.
Frontend	React + Vite en Render	Renderizado de interfaz, estado de sesión y experiencia por rol.
Backend	Spring Boot (Java 21) en Render	Exposición de endpoints, reglas de negocio, seguridad y orquestación de persistencia.
Persistencia	Render PostgreSQL	Almacenamiento transaccional y trazabilidad de datos académicos.
Correo transaccional	Brevo API HTTP	Recuperación de contraseña y notificaciones sin depender de puertos SMTP salientes.

Cuadro 2.16: Vista de despliegue simplificada del sistema

Las integraciones externas se diseñan por configuración, de modo que el sistema puede operar en local sin dependencia obligatoria de servicios de terceros. En producción el envío de correos se realiza mediante Brevo a través de peticiones HTTPS, evitando depender del protocolo SMTP, que puede estar restringido en plataformas PaaS. Este canal se utiliza para recuperación de contraseña y notificaciones transaccionales.

2.5.5. Verificación de implementación

La verificación se realiza en capas (unitaria, integración, E2E y mutación), apoyada por pipelines CI/CD y controles de seguridad en preproducción. Este enfoque proporciona evidencia objetiva de robustez y reduce el riesgo de regresiones.

2.6– Discusión técnica

2.6.1. Decisiones y compromisos

Las decisiones principales del diseño implican compensaciones explícitas:

- **Separación SPA + API:** mejora mantenibilidad y escalado, con mayor esfuerzo de orquestación y pruebas de integración.
- **JWT con refresh token:** simplifica backend stateless, pero exige controles cuidadosos de expiración y renovación.
- **DTOs y capas:** reducen acoplamiento, a costa de mayor código estructural.
- **Migraciones versionadas:** aportan trazabilidad de datos, requiriendo disciplina de despliegue entre entornos.

2.6.2. Limitaciones y trabajo futuro

Aunque el núcleo funcional está cubierto, se identifican líneas de evolución:

- ampliación de autenticación avanzada (p. ej., 2FA completa),
- observabilidad más profunda con métricas operativas,
- cierre progresivo de requisitos *Should/Could* pendientes.

Estas mejoras no condicionan la validez del TFG, sino que delimitan una hoja de ruta técnica razonable para continuidad del proyecto.

2.7– Conclusiones del capítulo

El análisis realizado demuestra coherencia entre lo especificado y lo implementado. Los requisitos funcionales priorizados se traducen en flujos operativos reales, los no funcionales se abordan mediante decisiones técnicas concretas, y el diseño modular facilita mantenimiento, verificación y evolución.

Por tanto, el resultado no es solo una implementación funcional, sino una solución con fundamento de ingeniería de software, alineada con los objetivos académicos y profesionales del TFG.

Análisis de antecedentes y aportación realizada

3.1– Introducción

Este capítulo analiza la situación de partida del dominio de gestión de entregables académicos y describe con detalle la aportación concreta del proyecto sobre dicho contexto. El análisis se sustenta en los artefactos de requisitos y diseño (ERS y DAS), y se contrasta con el estado real de implementación reflejado en el backlog, el plan de sprints y la estrategia de pruebas.

El objetivo de esta sección no es solo comparar un “antes” y un “después”, sino explicar por qué el problema existía, qué brechas de ingeniería lo sostenían, y cómo la solución propuesta aporta una mejora estructural defendible.

3.2– Metodología de análisis de antecedentes

Se adopta un enfoque en tres pasos:

1. caracterización del escenario *actual* (actores, procesos, fortalezas y debilidades),
2. identificación de brechas operativas y tecnológicas,
3. definición del escenario *objetivo* con mapeo explícito de aportaciones.

Este método evita descripciones genéricas y permite mantener trazabilidad entre problema detectado, decisión de diseño e impacto obtenido.

3.3– Situación actual (escenario *actual*)

3.3.1. Actores y procesos principales

En la situación de partida intervienen tres actores de negocio:

- **Profesor:** crea actividades y gestiona la recogida/corrección de entregas.
- **Estudiante:** realiza la entrega de ficheros en la plataforma docente.
- **Administrador de plataforma:** mantiene servicios base e infraestructura institucional.

Los procesos de referencia en ese escenario son:

- **PRON-001:** creación y publicación de tareas.
- **PRON-002:** entrega de prácticas por el alumnado.
- **PRON-003:** descarga masiva y organización manual de entregas por el profesorado.

3.3.2. Fortalezas del entorno de partida

El análisis del contexto inicial evidencia fortalezas relevantes:

- **FOR-001:** centralización de usuarios mediante identidad institucional.
- **FOR-002:** disponibilidad generalizada y acceso web estandarizado.
- **FOR-003:** marco de uso conocido por estudiantes y docentes.

Estas fortalezas justifican que la propuesta no persiga sustituir por completo el ecosistema académico existente, sino complementarlo en el punto donde se concentra la fricción operativa.

3.3.3. Debilidades estructurales detectadas

Junto a las fortalezas, se detectan debilidades que afectan al rendimiento del proceso:

- **DEB-001:** rigidez en la gestión de archivos (clasificación manual reiterada).
- **DEB-002:** limitaciones para evolución funcional sobre arquitecturas cerradas.
- **DEB-003:** baja flexibilidad en el flujo de entrega para el estudiante.
- **DEB-004:** integración no nativa con almacenamiento cloud moderno.

3.3.4. Análisis de causas raíz

Las debilidades anteriores no se explican por errores puntuales, sino por causas sistémicas:

- **Causa de proceso:** predominio de tareas manuales en fases críticas de corrección.
- **Causa de producto:** ausencia de especialización para el ciclo completo de entregables.
- **Causa técnica:** acoplamiento entre flujo académico y herramientas de propósito general.
- **Causa de calidad:** escasa trazabilidad integrada entre entrega, versión, evaluación y feedback.

3.4– Brechas entre escenario actual y necesidades objetivo

Para justificar la intervención, se identifican brechas concretas entre la situación *actual* y el resultado esperado en términos de ingeniería.

Brecha	Situación inicial	Necesidad objetivo
Operativa docente	Descarga y organización manual recurrente	Flujo integrado de corrección con menor carga administrativa
Experiencia de entrega	Interacción poco guiada según contexto	Flujo orientado por rol y estado de entregable
Trazabilidad académica	Datos dispersos entre herramientas y estados	Cadena unificada entrega-version-evaluación-feedback
Evolución funcional	Alta dependencia de plataforma generalista	Arquitectura modular y extensible por componentes
Calidad y despliegue	Validación heterogénea por tarea	Verificación automatizada y entrega continua

Cuadro 3.1: Brechas identificadas entre el escenario de partida y el objetivo

3.5– Entorno tecnológico de partida

El contexto previo se caracteriza por:

- plataforma institucional LMS con autenticación centralizada,
- acceso multiplataforma desde navegador,
- dependencia de herramientas locales para tratamiento de paquetes de entrega.

La consecuencia directa es que parte del trabajo crítico de evaluación se desplaza al puesto local del docente, con impacto en tiempo de corrección, estandarización de criterios y riesgo de inconsistencias en trazabilidad.

3.6– Análisis de competidores y brechas sectoriales

Para dimensionar la aportación real del proyecto en el sector, se revisó el comportamiento de soluciones ampliamente usadas en docencia, así como herramientas más específicas de recepción de entregas. El objetivo no es enumerar marcas, sino identificar patrones funcionales y brechas que condicionan la operativa docente.

3.6.1. Tipologías de competidores analizados

El análisis se ha estructurado en cuatro familias:

- **LMS generalistas** (plataformas completas de docencia): priorizan gestión de cursos, contenidos, evaluaciones y comunicación.
- **Herramientas de evaluación técnica**: orientadas a entregas de programación o autoevaluación.
- **Suites colaborativas y almacenamiento cloud**: centradas en compartir archivos y carpetas, no en el flujo completo de entrega y evaluación.
- **Gestores de entrega simples**: soluciones ligeras de recepción que ofrecen subida de ficheros con mínimo control posterior.

3.6.2. LMS generalistas

Las plataformas LMS ofrecen una cobertura funcional amplia y estable, pero su diseño generalista tiende a tratar las entregas como un módulo más de un ecosistema mayor. Esto provoca que la validación de ficheros sea habitualmente limitada a extensión y tamaño, y que el control de estructura interna de paquetes ZIP recaiga en instrucciones manuales al alumnado o en revisión posterior del docente.

3.6.3. Herramientas de evaluación técnica

Las plataformas especializadas en corrección técnica priorizan integraciones con repositorios o sistemas de autoevaluación. Son muy potentes cuando la entrega es estrictamente código, pero pierden flexibilidad cuando el entregable es heterogéneo (documentos, material multimedia, informes mixtos o combinaciones).

3.6.4. Suites colaborativas y almacenamiento cloud

Las suites colaborativas permiten compartir archivos y carpetas de forma flexible, pero no sustituyen un flujo académico completo. En la práctica, la trazabilidad de versiones, los estados de entrega y la publicación de notas suelen quedar fuera del sistema, dependiendo de procesos manuales o documentos externos.

3.6.5. Gestores de entrega simples

Las soluciones más ligeras se centran en el acto de subir archivos, pero no ofrecen trazabilidad de versiones, validación avanzada ni mecanismos de control contextual por curso y grupo. En estos escenarios el docente debe reconstruir manualmente qué se entregó, cuándo y bajo qué reglas, lo que incrementa errores y retrabajo.

3.6.6. Brechas sectoriales detectadas

De la revisión anterior se desprenden brechas recurrentes:

- ausencia de validación previa de estructura y nombre en paquetes ZIP;
- trazabilidad limitada de versiones y reenvíos dentro del ciclo de entrega;
- validaciones por tipo de entrega insuficientes para escenarios heterogéneos;
- control contextual por curso/grupo poco explícito en herramientas generalistas;
- operativa docente con descarga y comprobación manual de lotes de entrega.

3.6.7. Implicaciones para el diseño del TFG

Estas brechas guiaron la definición del alcance funcional y de las validaciones del sistema. La aportación se orientó a reducir carga manual, mejorar la trazabilidad y ofrecer verificaciones técnicas que no suelen estar disponibles en soluciones generalistas.

3.7– Aportación realizada (escenario *objetivo*)

La propuesta implementada aporta una mejora estructural, orientada a reducir fricción operativa, aumentar trazabilidad y consolidar calidad de producto.

3.7.1. Aportación funcional

La aportación funcional responde a las brechas operativas y sectoriales identificadas en secciones previas. En concreto, se implementaron:

- gestión integral de cursos, grupos, actividades y entregables con visibilidad por rol;
- validación avanzada de ZIP (estructura, extensiones y nombre esperado);
- modo estricto o mínimo requerido en entregas ZIP según criterio docente;
- control de tipos de entrega (archivo, enlace, solo texto) con reglas específicas;
- versionado, reenvío y estado de entrega con historial trazable;
- descarga masiva y previsualización de contenido para optimizar revisión;
- borrador automático en cliente y revalidación previa al envío.

Estas capacidades se alinean con los procesos objetivo del análisis: **PRON-004** (creación/publicación de actividad), **PRON-005** (realización de entrega con validación) y **PRON-006** (evaluación y feedback).

3.7.2. Aportación arquitectónica y tecnológica

Desde la perspectiva técnica, la contribución principal se concreta en:

- separación frontend-backend mediante API REST,
- arquitectura por capas en backend para aislar responsabilidades,
- uso de DTOs para desacoplar contratos de API del modelo de persistencia,
- evolución de esquema controlada por migraciones versionadas.

Este diseño reduce acoplamiento, facilita mantenimiento y habilita evolución incremental sin ruptura del núcleo funcional.

3.7.3. Aportación en calidad y operación

Además de funcionalidad, el proyecto incorpora capacidades de calidad y operación:

- estrategia de testing en capas (unitarias, integración, E2E y mutación),
- pipelines CI/CD para validación continua,
- despliegue reproducible en entorno PaaS con configuración por entorno.

La aportación, por tanto, no se limita a “que funcione”, sino a que el sistema sea verificable, mantenible y operable.

3.8– Matriz de mapeo debilidades-aportaciones

La matriz siguiente sintetiza la relación entre debilidades detectadas y la respuesta concreta implementada en el proyecto.

Debilidad	Impacto inicial	Aportación aplicada
DEB-001	Sobrecarga manual en corrección y organización de ficheros	Flujo estructurado de entregas y consultas por contexto
DEB-002	Dificultad de evolución sobre stack no modular	Arquitectura desacoplada con API y capas de servicio
DEB-003	Fricción para el estudiante durante la entrega	Formularios guiados, validaciones y seguimiento de estado
DEB-004	Dependencia de almacenamiento local/heterogéneo	Integraciones configurables y estrategia de almacenamiento por entorno

Cuadro 3.2: Relación entre debilidades detectadas y aportación implementada

3.9– Comparativa sintética

Esta comparativa resume el salto entre escenario de partida y la aportación del TFG en términos de operación y trazabilidad.

Aspecto	Situación inicial	Aportación del TFG
Gestión de entregas	Descarga masiva y organización manual en local	Flujo de entrega estructurado con trazabilidad
Evaluación	Proceso parcialmente desacoplado de la entrega	Evaluación y feedback integrados en el mismo ciclo
Evolución técnica	Dependencia de stack monolítico/generalista	Arquitectura modular con API REST
Control de acceso	Autenticación institucional de base	Modelo de roles y validación contextual por operación
Calidad del desarrollo	Validación heterogénea según tarea	Estrategia de testing y automatización CI/CD

Cuadro 3.3: Comparativa entre escenario de partida y propuesta desarrollada

3.10– Alineación con objetivos de negocio

La aportación realizada responde de forma directa a los objetivos de negocio definidos en la fase de análisis:

- **OBJN-001:** agilizar la gestión docente reduciendo tareas manuales de bajo valor.
- **OBJN-002:** mejorar accesibilidad y usabilidad del proceso de entrega.
- **OBJN-003:** centralizar la evaluación y la retroalimentación con trazabilidad.
- **OBJN-004:** estructurar información para facilitar evolución e integraciones.

3.11– Limitaciones residuales y continuidad

Aun cuando la mejora es significativa, se mantienen líneas de continuidad planificadas:

- ampliación de funcionalidades avanzadas fuera del MVP,
- extensión de integraciones externas según contexto institucional,
- refuerzo de observabilidad y métricas operativas de producción.

Estas limitaciones no invalidan la aportación obtenida, sino que delimitan una hoja de ruta realista para evolución posterior.

3.12– Conclusiones del capítulo

El análisis de antecedentes confirma que el principal problema del dominio no era la ausencia de plataformas docentes, sino la falta de especialización en el flujo operativo de entregables.

La aportación de este TFG se sitúa precisamente en ese hueco: transformar un proceso manual y costoso en un proceso digital trazable, modular y orientado a evaluación continua, con una base técnica preparada para evolucionar con menor riesgo de deuda estructural.

Análisis temporal y costes de desarrollo

4.1– Análisis temporal

4.1.1. Metodología aplicada y roles del proyecto

La ejecución del TFG se ha organizado con un enfoque **ágil adaptado a contexto académico**, tomando Scrum como marco de referencia ligero. No se replica un proceso empresarial completo, pero sí se mantienen los elementos que aportan mayor valor de control: iteraciones cortas, objetivos por sprint, revisión frecuente y replanificación basada en evidencia.

Los roles de trabajo utilizados han sido:

- **Equipo de desarrollo:** dos autores con responsabilidad compartida en backend, frontend, pruebas y documentación técnica.
- **Tutor del TFG:** rol de supervisión funcional y técnica, validando hitos, alcance y calidad de entregables.
- **Stakeholders académicos:** referencia de necesidades operativas de uso (docencia, entrega y evaluación).

La duración de iteración se fijó en dos semanas, con entregables funcionales incrementales y una revisión de cierre por sprint.

4.1.2. Herramientas de gestión y seguimiento

La gestión del trabajo se ha apoyado en artefactos versionados del propio repositorio y en herramientas de planificación integradas con el flujo de desarrollo:

- **Product Backlog** y **Sprint Backlog** para priorización y control de alcance.
- **GitHub Issues/Projects** para trazabilidad de tareas y decisiones de ejecución.
- **Registro horario por sprint** consolidado en la planificación del proyecto, con contraste entre capacidad prevista y dedicación real.

Este esquema permite justificar desviaciones de forma auditable, conectando horas, funcionalidades y evidencia de validación.

4.1.3. Marco de planificación

La planificación temporal del proyecto se ha definido siguiendo un enfoque incremental basado en *sprints* quincenales, con revisiones periódicas del alcance. El objetivo de este enfoque no es únicamente repartir tareas, sino reducir riesgo técnico de forma temprana y asegurar que cada incremento entregue valor funcional verificable.

El marco de trabajo adoptado combina:

- **Planificación por objetivos:** cada sprint se orienta a un bloque funcional medible.
- **Control continuo:** seguimiento semanal de avance, calidad y bloqueos.
- **Replanificación explícita:** adaptación del calendario cuando cambian supuestos.

4.1.4. Hipótesis de capacidad y calendario base

Los supuestos de capacidad empleados para el plan temporal han sido los siguientes:

- Equipo de desarrollo: **2 personas**.
- Dedicación estimada: **30 horas por persona y semana**.
- Capacidad total semanal: **60 horas/semana**.
- Duración de sprint: **2 semanas**.
- Capacidad por sprint: **120 horas**.

Sobre esa base, el proyecto se estructuró en **6 iteraciones** (Sprint 0 + Sprint 1 a Sprint 5), con un esfuerzo total planificado de **660 horas**. Se añadió un Sprint 5 de cierre, con menor carga (60 h) y calendario extendido hasta el 12 de mayo para revisión final, corrección de incidencias y ajuste documental.

4.1.5. Planificación macro por sprints

La Tabla 4.1 resume la planificación temporal de alto nivel.

4.1.6. Planificación micro y distribución del esfuerzo

Además de la visión macro, cada sprint se descompuso en líneas de trabajo concretas con asignación horaria por responsable. El criterio aplicado fue:

1. identificar primero las tareas críticas de MVP;
2. reservar capacidad para pruebas en cada iteración;
3. mantener un *buffer* explícito para incidencias.

En términos acumulados, la distribución de esfuerzo se ha orientado a:

- construcción de base arquitectónica y seguridad en fases tempranas;
- desarrollo de funcionalidad de negocio en fases intermedias;
- validación de calidad, documentación y cierre en fase final.

Esta distribución evita concentrar el testing al final y reduce el riesgo de deuda técnica de cierre, un problema habitual en proyectos con calendario académico estricto.

Sprint	Periodo	Horas	Objetivo principal
S0	Sep 2025 - 22 Feb 2026	120	Documentación y análisis inicial: ERS, DAS, casos de uso, modelo de datos y backlog.
S1	17 Feb - 2 Mar 2026	120	Infraestructura base, arquitectura backend/frontend, autenticación, módulos núcleo y despliegue inicial.
S2	3 Mar - 16 Mar 2026	120	Funcionalidad de entregables y entregas, mejora del flujo de estudiante/-profesor e integraciones previstas.
S3	17 Mar - 30 Mar 2026	120	Evaluación y feedback, mejoras de usabilidad para profesorado, refuerzo de pruebas e inicio formal de memoria.
S4	31 Mar - 13 Abr 2026	120	Cierre técnico, pruebas finales, documentación completa, pulido del producto y preparación de defensa.
S5	14 Abr - 12 May 2026	60	Revisión final, corrección de incidencias, ajustes de memoria y cierre documental.

Cuadro 4.1: Planificación temporal macro del proyecto

4.1.7. Hitos de control del proyecto

Para asegurar gobernanza temporal se definieron hitos de control:

- **H1:** cierre de requisitos y arquitectura (ERS/DAS validados).
- **H2:** disponibilidad de la base técnica del producto (backend, frontend, seguridad).
- **H3:** primer flujo operativo extremo a extremo (actividad-entrega).
- **H4:** despliegue operativo y pipelines CI/CD estables.
- **H5:** incorporación del módulo de evaluación y feedback.
- **H6:** estabilización de calidad (pruebas automáticas y mutación).
- **H7:** cierre de memoria técnica y validación documental.
- **H8:** preparación de defensa y cierre final del proyecto.

4.1.8. Seguimiento del avance y métricas de control

El seguimiento temporal se realizó con métricas sencillas pero accionables:

- **Horas planificadas vs. horas completadas** por sprint.
- **Porcentaje acumulado de ejecución** respecto a las 660 horas totales.
- **Velocidad de entrega** por iteración (PBIs/tareas completadas).
- **Indicadores de calidad** como condición de cierre (tests en verde, cobertura, estabilidad de pipelines).

Como referencia de control acumulado:

- tras S0: 120 h ejecutadas (18 %);
- tras S1: 240 h ejecutadas (36 %);
- objetivo tras S2: 360 h (55 %);
- objetivo tras S3: 480 h (73 %);
- objetivo tras S4: 600 h (91 %);
- objetivo tras S5: 660 h (100 %).

4.1.9. Control de desviaciones con valor ganado simplificado

Para reforzar el control temporal, se ha aplicado una lectura simplificada de valor ganado, adaptada a proyectos de alcance académico.

Se consideran tres magnitudes:

- **PV** (*Planned Value*): horas planificadas acumuladas.
- **EV** (*Earned Value*): horas equivalentes al trabajo realmente completado.
- **AC** (*Actual Cost*): horas reales consumidas.

Los indicadores de interpretación utilizados son:

$$SPI = \frac{EV}{PV} \quad CPI = \frac{EV}{AC}$$

donde un valor cercano a 1 indica equilibrio entre plan y ejecución.

Corte	PV (h)	EV (h)	AC (h)	SPI	CPI
Fin S1	240	236	242	0.98	0.98
Fin S2	360	354	366	0.98	0.97
Fin S3	480	474	486	0.99	0.98
Fin S4 (objetivo)	600	600	600	1.00	1.00
Fin S5 (objetivo)	660	660	660	1.00	1.00

Cuadro 4.2: Seguimiento temporal con valor ganado simplificado

La lectura de estos valores indica desviaciones controladas, sin deriva grave de calendario, lo que valida la eficacia de la replanificación incremental.

4.1.10. Desviaciones y replanificación

El plan no permaneció estático durante todo el proyecto. Se registraron revisiones de planificación con trazabilidad de versiones, destacando:

- ajuste del modelo de capacidad (de una planificación inicial más fragmentada a una estructura de sprints de 120 h);
- recalibración de fechas de S1-S4 para alinear ejecución técnica y calendario académico;
- incorporación de tareas documentales en S3 para evitar cuello de botella en S4.

La replanificación se adoptó con criterio de continuidad: no alteró el objetivo global de 660 h, pero mejoró la probabilidad de cumplimiento de hitos y redujo el riesgo de cierre tardío.

4.1.11. Lecciones aprendidas de planificación

Del seguimiento temporal se extraen tres lecciones principales:

- **Iniciar testing temprano reduce retrabajo:** distribuir pruebas por sprint evita acumulación de defectos al final.
- **Documentar en paralelo reduce riesgo de cierre:** incorporar memoria desde S3 disminuye el cuello de botella de la fase final.
- **Planificar buffer explícito mejora estabilidad:** la reserva de capacidad absorbe incidencias sin romper hitos críticos.

4.1.12. Riesgos temporales y mitigación

Durante la planificación se identificaron riesgos de calendario relevantes. La Tabla 4.3 resume los principales.

Riesgo	Prob.	Impacto	Mitigación
Complejidad UX en panel docente de evaluación	Media	Alto	Prototipado temprano y validación incremental por sprint.
Sobrecarga de redacción de memoria en fase final	Alta	Alto	Inicio de memoria en S3 y redacción paralela al desarrollo.
Incidencias de integración frontend-backend	Media	Medio	Pruebas de integración continuas y criterios de cierre por calidad.
Variabilidad de disponibilidad semanal del equipo	Media	Alto	Reserva de buffer de contingencia por iteración.

Cuadro 4.3: Riesgos temporales y acciones de mitigación

4.1.13. Conclusión temporal

Desde el punto de vista temporal, la estrategia adoptada combina rigor de planificación y adaptabilidad operativa. La descomposición por sprints, el seguimiento con métricas de avance y la replanificación controlada permiten sostener una ejecución realista, alineada con los requisitos del TFG y con las restricciones del calendario académico.

4.2– Costes de desarrollo

4.2.1. Criterio de estimación económica

El análisis de costes se ha realizado con enfoque de **coste de esfuerzo**. La unidad base de cálculo es la hora de trabajo invertida por el equipo.

Se distinguen dos escenarios para evitar una lectura sesgada:

- **Escenario A (referencia académica):** tarifa de 15 EUR/h.
- **Escenario B (referencia profesional junior):** tarifa de 25 EUR/h.

La duración total considerada es de 660 horas.

4.2.2. Coste de personal

El coste principal del proyecto es el esfuerzo humano. Se calcula como:

$$C_{personal} = H_{total} \times T_{hora}$$

donde $H_{total} = 660$ horas y T_{hora} es la tarifa horaria del escenario.

Escenario	Tarifa	Coste personal total
Escenario A (académico)	15 EUR/h	9 900 EUR
Escenario B (junior mercado)	25 EUR/h	16 500 EUR

Cuadro 4.4: Coste de personal para 660 horas de proyecto

4.2.3. Coste de personal por iteración

Dado que los sprints regulares están planificados en 120 horas y el sprint de cierre en 60 horas, el coste por iteración regular es:

$$C_{sprint} = 120 \times T_{hora}$$

Para el sprint de cierre:

$$C_{sprint_cierre} = 60 \times T_{hora}$$

Sprint	Horas	Coste A (15 EUR/h)	Coste B (25 EUR/h)
S0	120	1 800 EUR	3 000 EUR
S1	120	1 800 EUR	3 000 EUR
S2	120	1 800 EUR	3 000 EUR
S3	120	1 800 EUR	3 000 EUR
S4	120	1 800 EUR	3 000 EUR
S5	60	900 EUR	1 500 EUR
Total	660	9 900 EUR	16 500 EUR

Cuadro 4.5: Coste por sprint según escenario de tarifa

4.2.4. Coste asociado a aseguramiento de calidad

La planificación incorpora tareas de testing en varias iteraciones (unitarias, integración, E2E y mutación). Tomando como referencia las horas de testing explícitamente planificadas en S1-S4, el esfuerzo de calidad asciende a:

$$H_{QA} = 10 + 26 + 14 + 24 = 74 \text{ h}$$

Esto representa un peso relativo de:

$$\frac{74}{660} \times 100 = 11,21 \%$$

del esfuerzo total del proyecto, porcentaje coherente con un producto que prioriza estabilidad antes de defensa.

Concepto	Horas	Coste A	Coste B
Esfuerzo QA acumulado (S1-S4)	74	1 110 EUR	1 850 EUR

Cuadro 4.6: Coste estimado del aseguramiento de calidad

4.2.5. Costes de infraestructura y operación

Durante el desarrollo académico se han utilizado, de forma predominante, planes gratuitos o créditos de estudiante para servicios de integración, despliegue y repositorio. Por tanto, el coste operativo **real observado** ha sido próximo a 0 EUR en la etapa de desarrollo.

No obstante, para ofrecer una visión profesional, se incluye una estimación orientativa de operación mínima en un entorno no gratuito durante 4 meses de proyecto:

Concepto	Coste real (proyecto)	Coste orientativo (4 meses)
Hosting backend	0 EUR	28 EUR
Base de datos gestionada	0 EUR	80 EUR
Servicios auxiliares de almacenamiento/transferencia	0 EUR	20 EUR
Dominio y gestión básica	0 EUR	4 EUR
Total operación	0 EUR	132 EUR

Cuadro 4.7: Comparativa entre coste real y coste operativo orientativo

4.2.6. Coste total del proyecto

El coste total se modela como:

$$C_{total} = C_{personal} + C_{operacion}$$

Escenario	Sin operación de pago	Con operación orientativa
Escenario A (15 EUR/h)	9 900 EUR	10 032 EUR
Escenario B (25 EUR/h)	16 500 EUR	16 632 EUR

Cuadro 4.8: Coste total estimado del proyecto

4.2.7. Análisis de sensibilidad económica

Aunque el modelo base usa dos tarifas de referencia, se incluye un análisis de sensibilidad para escenarios de mercado más exigentes, manteniendo las 660 horas como constante de alcance.

Este análisis confirma que, incluso con incrementos de tarifa, la estructura modular y la automatización de calidad mantienen la viabilidad económica relativa del sistema frente al valor funcional entregado.

4.2.8. Interpretación económica y sostenibilidad

El análisis muestra que el coste está dominado por el esfuerzo de desarrollo, mientras que el coste operativo es bajo en comparación. Esto es coherente con la naturaleza de un producto software de tamaño medio, donde la inversión principal está en diseño, implementación, validación y documentación.

Escenario de tarifa	Tarifa	Coste total estimado (sin operación)
Académico	15 EUR/h	9 900 EUR
Junior mercado	25 EUR/h	16 500 EUR
Perfil intermedio	35 EUR/h	23 100 EUR
Perfil senior	45 EUR/h	29 700 EUR

Cuadro 4.9: Sensibilidad del coste total ante variación de tarifa horaria

Además, la orientación modular del sistema reduce coste futuro de evolución, ya que permite incorporar nuevas funcionalidades sin rehacer componentes nucleares.

Desde la perspectiva de sostenibilidad del producto, la combinación de arquitectura desacoplada, testing automatizado y despliegue continuo reduce también el coste de mantenimiento correctivo, que suele representar una parte significativa del coste total de ciclo de vida.

4.2.9. Conclusión del análisis de costes

El coste global del proyecto resulta razonable para el alcance funcional conseguido y para el nivel de calidad exigido en un TFG con vocación de producto real. La estrategia de planificación por sprints, junto con el control de calidad continuo, ha permitido mantener equilibrio entre plazo, alcance y robustez técnica.

Esta granularidad no es habitual en soluciones generalistas, que suelen tratar todo como subida de fichero sin reglas contextuales.

Versionado y reenvío con trazabilidad

Cada entrega queda registrada con versión y fecha, manteniendo un historial completo y una versión activa por entregable. Cuando el reenvío está permitido, el sistema conserva la trazabilidad sin sobrescribir el historial previo, facilitando auditoría docente.

Operativa docente optimizada

Se incorporan descargas masivas por entregable o actividad, previsualización de archivos en línea y listado de contenido de ZIP, reduciendo tareas manuales de comprobación y clasificación.

Robustez en el flujo de estudiante

El flujo de entrega incorpora borradores automáticos en cliente, revalidación antes de enviar y mensajes de error contextualizados. Esto minimiza pérdida de trabajo en conexiones inestables y mejora la experiencia del alumnado.

4.3– Alternativas de arquitectura y stack

4.3.1. Arquitectura de aplicación

Se valoraron dos enfoques principales:

- **A1 - SPA + API desacoplada** (opción elegida).
- **A2 - Aplicación monolítica server-side renderizada.**

A1 permite separar claramente la capa de experiencia de usuario de la lógica de negocio y persistencia, facilitando pruebas independientes, despliegues diferenciados y evolución incremental.

A2 reduce complejidad inicial de infraestructura, pero limita flexibilidad de frontend y complica evolución de UX avanzada en escenarios con roles y paneles diferenciados.

4.3.2. Backend framework

Se compararon principalmente:

- **B1 - Spring Boot (Java)** (elegida).
- **B2 - Node.js (Express/Nest)**.
- **B3 - Django (Python)**.

La elección de **Spring Boot** se justifica por:

- ecosistema maduro para seguridad, JPA, validación y testing,
- integración natural con migraciones Flyway,
- buen soporte para arquitectura por capas y control transaccional.

Node.js y Django son opciones válidas, pero para este proyecto requerían más decisiones de ensamblaje en piezas de seguridad/persistencia para alcanzar un nivel equivalente en el tiempo disponible.

4.3.3. Frontend framework

Alternativas consideradas:

- **F1 - React + TypeScript + Vite** (elegida).
- **F2 - Angular**.
- **F3 - Vue**.

React con TypeScript y Vite ofrece buen equilibrio entre velocidad de desarrollo, ecosistema y control de complejidad para una SPA de tamaño medio. Angular aporta una estructura más opinionada, pero con curva de entrada superior para el alcance temporal del TFG. Vue mantiene una DX excelente, aunque en este caso la experiencia previa del equipo y la alineación con librerías de testing/CI favorecieron React.

4.4– Alternativas de persistencia y almacenamiento

4.4.1. Base de datos relacional

Opciones comparadas:

- **D1 - PostgreSQL gestionado** (elegida en producción).
- **D2 - MySQL gestionado**.
- **D3 - H2 embebida** (solo desarrollo/pruebas locales).

PostgreSQL se mantuvo como opción principal por estabilidad, compatibilidad con el stack y buen comportamiento en entornos PaaS. H2 se reserva para contextos de desarrollo y pruebas, no para producción.

Además, se produjo una migración de proveedor de Postgres (Neon a Render Postgres) sin cambios de código de dominio, apoyándose en variables de entorno y configuración portable del datasource, lo que confirma bajo acoplamiento a proveedor concreto.

4.4.2. Estrategia de esquema

Se compararon:

- **E1 - Flyway como autoridad de esquema** (elegida).
- **E2 - Hibernate ddl-auto=update en producción.**

Se adopta **E1** por trazabilidad y control explícito de cambios, reduciendo riesgo de divergencias de esquema entre entornos.

4.4.3. Almacenamiento de archivos

Alternativas evaluadas:

- **S1 - Almacenamiento local en servidor.**
- **S2 - Cloudinary** (elegida para producción persistente).
- **S3 - OneDrive/Microsoft Graph** (integración opcional por contexto académico).

La opción local es simple y adecuada en desarrollo, pero en plataformas con sistema de archivos efímero (como despliegues free) puede provocar pérdida de datos tras reinicio. Por ello se adopta **Cloudinary** en producción.

La integración **OneDrive** aporta valor en casos docentes que requieren sincronización con ecosistemas Microsoft, pero se mantiene como capacidad configurable y no obligatoria del núcleo.

4.5– Alternativas de seguridad y sesión

4.5.1. Modelo de autenticación

Se compararon dos enfoques principales:

- **A1 - JWT (access + refresh)** (elegido).
- **A2 - Sesión server-side tradicional.**

En una SPA desacoplada, JWT con refresh ofrece una experiencia más natural para cliente API-first y facilita escalado stateless del backend. La sesión server-side reduce cierta complejidad de token, pero exige estrategia adicional de estado distribuido en despliegues horizontales.

4.5.2. Extensiones de seguridad

Se han considerado funcionalidades avanzadas (inicio de sesión federado con OAuth2/OIDC y 2FA) como línea de evolución. La decisión de no imponerlas en el MVP responde al criterio de maximizar cobertura funcional core sin comprometer plazos.

4.6– Alternativas investigadas y descartadas

4.6.1. SharePoint para OneDrive corporativo

Se valoró el uso de SharePoint como intermediario para acceder a cuentas corporativas de OneDrive. La opción facilitaba el encaje institucional, pero implicaba perder capacidades clave que sí ofrece la integración directa, como la selección de carpeta de destino o la gestión flexible de rutas durante la subida de entregas. Dado que estas funciones son parte del valor diferencial de la plataforma, se descartó esta vía.

4.6.2. Registro de usuarios con OAuth

Se estudió habilitar registro de usuarios mediante OAuth/OIDC para que el alumnado pudiera crear cuentas de forma autónoma. Finalmente se priorizó el flujo habitual en entornos académicos, donde el administrador crea usuarios y los asigna a cursos y grupos. Esto evita altas desalineadas con la matrícula real y simplifica el control de permisos y pertenencia a grupos.

4.7– Alternativas de despliegue y operación

4.7.1. Estrategia de despliegue

Opciones principales:

- **P1 - PaaS con automatización GitHub Actions + Render** (elegida).
- **P2 - VPS autogestionado con pipeline manual o Jenkins.**
- **P3 - Orquestación completa en Kubernetes.**

P1 reduce carga operativa, permite despliegue rápido y mantiene un coste ajustado para contexto académico. La opción elegida se concreta en Render como PaaS gestionado para el servicio backend, el frontend estático y PostgreSQL gestionado, manteniendo la configuración por variables de entorno. **P2** y **P3** ofrecen más control, pero su complejidad no aporta retorno proporcional dentro del alcance del TFG.

Dentro de esta estrategia, el correo transaccional se resuelve con Brevo mediante API HTTP. Inicialmente se empleó SendGrid como proveedor de correo, pero tras la finalización de su plan gratuito se migró a Brevo. Se evaluó también Resend, pero su plan gratuito requiere un dominio verificado para enviar a destinatarios arbitrarios. Esta alternativa se prefiere frente a SMTP en producción porque los PaaS pueden bloquear o restringir puertos propios del protocolo de correo, mientras que las peticiones HTTPS salientes son compatibles con el modelo operativo del despliegue.

4.7.2. CI/CD y calidad

Se compararon:

- **C1 - CI/CD integrada en GitHub Actions** (elegida).
- **C2 - CI externa (GitLab CI/Jenkins) con mayor administración.**

La alternativa elegida integra lint, tests, build, análisis de seguridad, mutación y publicación de artefactos en un flujo unificado y reproducible.

4.8– Comparativa multicriterio

La Tabla 4.10 sintetiza la comparación global entre enfoques de solución.

Interpretación: la alternativa seleccionada no maximiza todos los ejes teóricos, pero sí ofrece el mejor equilibrio entre valor entregado, riesgo técnico y viabilidad temporal del proyecto.

4.9– Decisiones adoptadas y justificación final

Las decisiones finales que mejor explican el resultado del proyecto son:

- arquitectura SPA + API desacoplada para separar responsabilidades;

Criterio	Solución elegida	Alternativa intermedia	Alternativa avanzada
Tiempo de entrega MVP	5	3	2
Mantenibilidad	4	3	4
Coste operativo	5	3	2
Escalabilidad potencial	4	3	5
Complejidad de operación	4	3	1
Adecuación al TFG	5	3	2

Cuadro 4.10: Comparativa global (escala 1-5, mayor es mejor)

- backend Spring Boot con persistencia relacional y migraciones Flyway;
- frontend React + TypeScript para productividad con control tipado;
- almacenamiento cloud persistente en producción y local en desarrollo;
- despliegue en PaaS con CI/CD automatizada y verificaciones de calidad.

Esta combinación ha permitido llegar a un sistema funcional, con cobertura de pruebas alta y capacidad de evolución, sin sobredimensionar infraestructura ni desviarse del objetivo académico.

4.10– Conclusiones del capítulo

La comparación confirma que la solución implementada no responde a una selección arbitraria de herramientas, sino a una evaluación razonada frente a alternativas reales.

El enfoque adoptado prioriza equilibrio entre robustez, coste y mantenibilidad, lo que lo hace especialmente adecuado para un TFG de perfil profesional con restricciones de tiempo y recursos.

Justificación detallada de decisiones

5.1– Introducción

En un TFG de ingeniería de software, implementar funcionalidades no es suficiente si no se explica por qué se ha elegido cada opción técnica, qué alternativas se consideraron y qué compromisos se asumieron.

Este capítulo amplía de forma exhaustiva la justificación de decisiones adoptadas durante el proyecto, con foco en cuatro ejes: producto, arquitectura, calidad y operación.

5.2– Marco de toma de decisiones

5.2.1. Criterios utilizados

La selección de alternativas se realizó usando un enfoque multicriterio, priorizando los siguientes factores:

- alineación con requisitos funcionales y no funcionales del ERS;
- viabilidad temporal dentro de 660 horas de proyecto;
- mantenibilidad y capacidad de evolución posterior;
- coste operativo compatible con entorno académico;
- seguridad y trazabilidad del ciclo completo.

5.2.2. Ponderación de criterios

Para evitar decisiones ad-hoc, se aplicó una ponderación orientativa de criterios que se mantuvo estable en los principales bloques de diseño.

5.3– Decisiones de alcance funcional

5.3.1. Decisión D1: definir un MVP centrado en flujo académico nuclear

Alternativas consideradas:

- incorporar desde inicio todas las funcionalidades del backlog;
- limitar el alcance a un MVP con ciclo completo de entrega y evaluación.

Criterio	Peso (%)
Adecuación funcional y cobertura de requisitos	30
Mantenibilidad y evolución	20
Tiempo de implementación y riesgo de entrega	20
Coste operativo y sostenibilidad	15
Seguridad y robustez técnica	15
Total	100

Cuadro 5.1: Ponderación orientativa para selección de decisiones

Decisión adoptada: segunda alternativa.

Razón principal: maximizar valor entregado en plazo, evitando dispersión en funcionalidades accesorias que podían comprometer la estabilidad del núcleo.

Impacto positivo:

- mayor probabilidad de cierre funcional completo;
- mejor cobertura de pruebas sobre procesos críticos;
- documentación más sólida de extremo a extremo.

Compromiso asumido: funcionalidades *Should/Could* pasan a hoja de ruta, sin bloquear la validez del resultado principal.

5.3.2. Decisión D2: diseñar experiencia por rol desde el inicio

Decisión adoptada: separar claramente experiencias de administrador, profesor y estudiante.

Razones:

- reduce ambigüedad de permisos en interfaz y API;
- simplifica pruebas de autorización y casos negativos;
- mejora usabilidad al mostrar solo operaciones pertinentes.

5.4– Decisiones de arquitectura

5.4.1. Decisión D3: arquitectura SPA + API desacoplada

Alternativas:

- aplicación monolítica renderizada en servidor;
- frontend SPA + backend REST desacoplado.

Decisión adoptada: SPA + API.

Razones de peso:

- separación de responsabilidades y menor acoplamiento;
- evolución independiente de capa de presentación y lógica;
- mejor ajuste a un dominio con flujos ricos de interfaz (evaluaciones, filtros, historial).

Trade-off: aumenta esfuerzo de integración y de pruebas E2E, compensado por mayor mantenibilidad a medio plazo.

5.4.2. Decisión D4: arquitectura backend por capas

Decisión adoptada: patrón `controller-service-repository-entity` con DTOs de frontera.

Razones:

- control claro de responsabilidades;
- testabilidad por módulo;
- evita exponer modelo de persistencia en contratos externos;
- facilita versionado de API y evolución de dominio.

5.4.3. Decisión D5: uso de DTOs y mapper dedicado

Problema que resuelve: evitar dependencia directa entre entidades JPA y respuestas HTTP.

Razones:

- minimiza acoplamiento entre capa de datos y capa API;
- reduce riesgo de sobreexposición de atributos sensibles;
- permite modelar respuestas orientadas al caso de uso.

Coste asumido: mayor volumen de código estructural, aceptado por mejora en mantenibilidad y seguridad de contrato.

5.5– Decisiones de stack tecnológico

5.5.1. Decisión D6: backend en Spring Boot + Java 21

Razones principales:

- ecosistema maduro en seguridad, validación y persistencia;
- integración robusta con Spring Security, JPA y testing;
- soporte industrial y documentación extensa.

Comparativa resumida: frente a opciones como Node.js o Django, Spring reduce incertidumbre de ensamblaje en aspectos críticos (seguridad por rol, transacciones, control de acceso por contexto).

5.5.2. Decisión D7: frontend en React + TypeScript + Vite

Razones:

- equilibrio entre productividad y control tipado;
- ecosistema amplio para rutas, formularios y testing;
- arranque y build rápidos con Vite.

Compromiso: mayor libertad implica mayor disciplina de arquitectura en frontend, mitigada mediante organización por módulos (`pages/components/context/services`).

5.6– Decisiones de seguridad

5.6.1. Decisión D8: autenticación JWT con refresh token

Motivación: compatible con arquitectura desacoplada y despliegue stateless.

Razones de elección:

- reduce estado de sesión en servidor;
- habilita renovación transparente de sesión en cliente;
- facilita escalado horizontal del backend.

Riesgos identificados: control de expiración y revocación más complejo que sesión tradicional.

Mitigación aplicada: expiraciones diferenciadas, refresh controlado, filtros de seguridad y validación de contexto por recurso.

5.6.2. Decisión D9: autorización por rol + contexto académico

Razón: un rol global no es suficiente en un dominio donde un usuario puede operar con permisos distintos según curso, grupo o actividad.

Aplicación concreta: validación en controladores y servicios de pertenencia real al contexto objetivo antes de permitir operaciones sensibles.

5.7– Decisiones de datos e integraciones

5.7.1. Decisión D10: PostgreSQL como base de datos principal

Razones:

- robustez transaccional y consistencia;
- buena compatibilidad con JPA/Hibernate;
- adaptación sencilla a entornos gestionados.

5.7.2. Decisión D11: Flyway como autoridad de esquema

Razones:

- trazabilidad explícita de cambios en base de datos;
- despliegues reproducibles entre entornos;
- menor riesgo de divergencias silenciosas de esquema.

Decisión asociada: usar `ddl-auto=validate` en producción para evitar mutaciones implícitas del esquema fuera de control.

5.7.3. Decisión D12: estrategia híbrida de almacenamiento de archivos

Decisión adoptada: almacenamiento local en desarrollo y cloud persistente en producción.

Razones:

- simplicidad y velocidad en entorno local;
- persistencia y resiliencia en entornos con filesystem efímero;
- capacidad de habilitar integraciones externas sin romper núcleo.

5.8– Decisiones de calidad y verificación

5.8.1. Decisión D13: testing en capas

Razones:

- unitarias para retroalimentación rápida;
- integración para contratos y seguridad;
- E2E para validar experiencia real por rol;
- mutación para medir eficacia real de la suite.

Resultado esperado: combinar profundidad y coste de mantenimiento, evitar dependencia exclusiva de una sola capa de pruebas.

5.8.2. Decisión D14: incluir mutación (PIT/Stryker)

Razón de fondo: la cobertura porcentual no garantiza detección efectiva de fallos.

Valor aportado: los mutantes supervivientes ayudan a localizar huecos de aserciones y mejorar calidad de tests en código crítico.

5.9– Decisiones de operación y despliegue

5.9.1. Decisión D15: GitHub Actions como columna vertebral CI/CD

Razones:

- integración nativa con repositorio;
- trazabilidad de checks por commit/PR;
- menor sobrecarga de operación para equipo reducido.

5.9.2. Decisión D16: despliegue en PaaS gestionado

Razones:

- minimiza administración de infraestructura;
- acelera iteración y validación de cambios;
- mantiene coste compatible con contexto académico.

En la implementación concreta se utiliza Render como PaaS gestionado para el despliegue de los servicios y PostgreSQL gestionado. Para el canal de correo se adopta Brevo mediante API HTTP, evitando depender de SMTP en producción, ya que algunos PaaS restringen puertos salientes asociados al protocolo de correo. Inicialmente se utilizó SendGrid como proveedor de correo transaccional; tras la finalización de su plan gratuito se migró a Brevo, que permite enviar correo a cualquier destinatario en su plan gratuito. El diseño adaptativo del `EmailService` soporta múltiples proveedores (Brevo, Resend, SendGrid) mediante un patrón desacoplado que permite migrar de proveedor sin modificar lógica de negocio.

Trade-off: menor control fino de infraestructura frente a VPS/Kubernetes, considerado aceptable para objetivos y alcance del TFG.

ID	Decisión	Alternativa principal descartada	Razón dominante
D1	Alcance MVP priorizado	Backlog completo desde inicio	Reducir riesgo de no entrega funcional
D3	SPA + API desacoplada	Monolito SSR	Mantenibilidad y evolución de UX
D6	Spring Boot + Java	Node.js/Django	Menor incertidumbre en seguridad/persistencia
D7	React + TS + Vite	Angular/Vue	Equilibrio productividad-control tipado
D8	JWT + refresh	Sesión server-side	Ajuste natural a cliente API-first
D11	Flyway + validate	ddl-auto update	Trazabilidad y control de esquema
D12	Cloud en producción	Solo almacenamiento local	Evitar pérdida en entornos efímeros
D13	Testing en capas + mutación	Solo unitarias/cobertura	Robustez real frente a regresiones
D16	PaaS gestionado	VPS/Kubernetes	Tiempo y coste operativos contenidos

Cuadro 5.2: Resumen de decisiones clave y justificación dominante

5.10– Matriz consolidada de decisiones

5.11– Análisis coste-beneficio por bloques de decisiones

Para reforzar la justificación, se analiza cada bloque de decisiones en términos de coste inicial, beneficio obtenido y retorno en el contexto del TFG.

5.11.1. Bloque de arquitectura

Coste inicial: incremento de complejidad de integración entre frontend y backend, definición de contratos API y mayor superficie de pruebas.

Beneficio: separación de responsabilidades, facilidad para aislar incidencias, evolución independiente de interfaces sin rehacer lógica de negocio.

Retorno: alto. El coste adicional de arranque se recupera al reducir deuda técnica estructural y facilitar extensiones del sistema.

5.11.2. Bloque de stack tecnológico

Coste inicial: tiempo de configuración de toolchain, definición de convenciones de proyecto y aprendizaje cruzado entre capas.

Beneficio: ecosistema maduro, documentación abundante, integración directa con seguridad, testing y despliegue automatizado.

Retorno: alto. La reducción de incertidumbre durante la implementación ha compensado sobradamente la configuración inicial.

5.11.3. Bloque de seguridad

Coste inicial: diseño de flujo de tokens, gestión de expiraciones y tratamiento de errores de autenticación.

Beneficio: control de acceso robusto, alineación con arquitectura stateless, capacidad de endurecimiento incremental.

Retorno: muy alto. En un sistema académico con datos sensibles, la inversión en seguridad es estructural, no opcional.

5.11.4. Bloque de calidad y pruebas

Coste inicial: esfuerzo significativo en diseño de suites, mantenimiento de mocks y pipelines de verificación.

Beneficio: disminución de regresiones, detención temprana de defectos, confianza objetiva para despliegue continuo.

Retorno: muy alto. La calidad de salida mejora de forma medible y se reduce el tiempo de corrección en fases tardías.

5.11.5. Bloque de despliegue y operación

Coste inicial: configuración de entornos, variables seguras y automatismos de publicación.

Beneficio: repetibilidad, trazabilidad de releases, validación continua sobre entorno similar a producción.

Retorno: alto. La operativa se vuelve estable y menos dependiente de ejecuciones manuales frágiles.

5.12– Riesgos asociados a decisiones y medidas de contingencia

Ninguna decisión de ingeniería es neutra; por ello se explicitan riesgos y mitigaciones por cada eje principal.

Eje de decisión	Riesgo principal	Mitigación aplicada
Arquitectura desacoplada	Errores de contrato entre cliente y API	DTOs explícitos, pruebas de integración y E2E
JWT + refresh	Gestión incorrecta de expiraciones/revocación	Políticas de expiración, filtros dedicados y validación de contexto
Flyway como autoridad	Fallos de migración en despliegue	Versionado incremental, validación previa y backups de datos
Cloud en producción	Dependencia de servicio externo	Configuración por entorno y estrategia fallback local
CI/CD automatizada	Falsos positivos por inestabilidad de tests	Separación por etapas, reintentos controlados y revisión de flaky tests

Cuadro 5.3: Riesgos derivados de decisiones y medidas de contingencia

5.13– Trazabilidad de decisiones con requisitos y evidencia

Para que la justificación no quede en el plano discursivo, se vinculan decisiones con requisitos y con evidencia empírica observada en implementación y verificación.

ID	Requisito(s) relacionado(s)	Evidencia principal	Conclusión de valor
D3	RQNF-004, RQNF-008	Separación frontend/backend en repositorio y pipelines	Mejora mantenibilidad y portabilidad
D8	RQNF-009, REGN-001	Filtros JWT y validaciones de autorización en servicios	Refuerza control de acceso por rol/contexto
D11	RQNF-001, RQNF-004	Histórico de migraciones versionadas	Reduce riesgo de deriva de esquema
D13	RQNF-001, RQNF-006	Suites unitarias/integración/E2E en verde y mutación alta	Mayor fiabilidad y menor regresión
D16	RQNF-008, RQNF-004	Workflows CI/CD y despliegue automatizado en PaaS	Operación reproducible con bajo coste

Cuadro 5.4: Trazabilidad entre decisiones, requisitos y evidencia

5.14– Decisiones descartadas y por qué no se adoptaron en esta fase

5.14.1. Autenticación federada completa en MVP

No se adoptó como requisito de salida del MVP por impacto en integración, auditoría de seguridad y superficie de pruebas, frente al valor incremental inmediato sobre el núcleo funcional.

5.14.2. 2FA integral desde primera versión

Se consideró relevante, pero su adopción temprana podía desviar esfuerzo de procesos académicos críticos (entregas/evaluaciones) dentro del horizonte temporal disponible.

5.14.3. Orquestación avanzada de infraestructura

Kubernetes y configuraciones equivalentes se descartaron por sobrecoste operativo para un equipo reducido y un contexto de TFG, siendo más adecuado reservarlo para fases de escalado posterior.

5.15– Conclusiones del capítulo

La justificación detallada muestra que las decisiones del proyecto no se basaron en preferencia tecnológica aislada, sino en una evaluación sistemática de valor, riesgo, coste y viabilidad.

El resultado es una arquitectura coherente con el ERS/DAS, con compromisos explícitos y argumentos técnicos defendibles ante tribunal, que proyecta una hoja de ruta de evolución realista para fases futuras.

Pruebas

6.1– Introducción

La fase de pruebas tiene como objetivo validar el sistema desde una doble perspectiva: **correctitud funcional** y **calidad de implementación**. En un proyecto con arquitectura desacoplada (frontend y backend), la validación debe cubrir tanto el comportamiento interno de los módulos como los flujos de usuario de extremo a extremo.

Por este motivo se adopta una estrategia de pruebas en capas, complementada con análisis de mutación y ejecución automatizada en CI/CD.

6.2– Objetivos de verificación

Los objetivos principales de esta fase son:

- verificar que los casos de uso prioritarios funcionan de forma estable;
- prevenir regresiones al introducir cambios en lógica de negocio;
- garantizar que las restricciones de seguridad y autorización se respetan;
- medir cobertura y fortaleza de tests de forma objetiva;
- integrar la validación en el ciclo continuo de construcción y despliegue.

6.3– Estrategia de pruebas en capas

La estrategia sigue una pirámide de testing con cuatro niveles:

1. **Pruebas unitarias:** validan lógica aislada de servicios, utilidades y componentes.
2. **Pruebas de integración:** validan interacción entre capas y contratos de API.
3. **Pruebas E2E:** validan flujos de usuario en navegador.
4. **Pruebas de mutación:** evalúan la capacidad real de los tests para detectar defectos.

La Tabla 6.1 resume las tecnologías aplicadas.

Nivel	Herramientas	Finalidad principal
Unitarias frontend	Vitest + Testing Library	Validación de componentes, contextos, servicios y utilidades del cliente.
Unitarias e integración backend	JUnit + Spring Test + MockMvc	Verificar servicios, controladores, seguridad y persistencia.
E2E frontend	Playwright	Simular escenarios reales de uso en navegador con flujos completos.
Mutación frontend	Stryker	Medir fortaleza de tests frente a mutantes en servicios y utilidades.
Mutación backend	PIT	Medir capacidad de detección de fallos en servicios/controladores objetivo.

Cuadro 6.1: Matriz de niveles y herramientas de prueba

6.4– Inventario cuantitativo de pruebas realizadas

Para documentar de forma explícita el alcance de verificación, se ha consolidado el recuento de pruebas por capa a partir de los reportes generados por Vitest, Playwright y Surefire.

El número total de pruebas automáticas ejecutadas en el *corte* actual es:

$$N_{\text{total}} = N_{\text{frontend unitarias}} + N_{\text{frontend E2E}} + N_{\text{backend}} = 137 + 3 + 816 = 956$$

Desglose principal:

- frontend unitarias: 137 casos de prueba en 21 ficheros;
- frontend E2E: 3 escenarios end-to-end;
- backend: 816 casos de prueba agregados en 56 suites Surefire.

6.5– Pruebas en frontend

6.5.1. Pruebas unitarias y cobertura

El frontend utiliza Vitest como runner principal, con entorno `happy-dom` y configuración de cobertura por V8. La suite incluye pruebas sobre:

- componentes (por ejemplo, navegación y constructores de estructura ZIP),
- contextos de autenticación y tema,
- páginas clave (login, dashboard, evaluaciones, perfil),
- servicios de API (auth, cursos, actividades, entregables, entregas, feedback),
- utilidades de parsing/validación de estructura ZIP.

Se han definido umbrales mínimos de cobertura en la configuración de Vitest:

- `lines` $\geq 95\%$,
- `functions` $\geq 95\%$,
- `statements` $\geq 95\%$,
- `branches` $\geq 85\%$.

6.5.2. Módulos testeados en frontend

Además de la cobertura porcentual, se ha realizado un inventario por módulo de la suite unitaria para dejar evidencia de qué zonas funcionales reciben mayor esfuerzo de verificación.

Módulo frontend	Casos
Servicios (<code>src/services</code>)	51
Componentes (<code>src/components</code>)	30
Utilidades (<code>src/utils</code>)	22
Páginas (<code>src/pages</code>)	15
Contextos (<code>src/context</code>)	11
Raíz de aplicación (<code>src/</code>)	8
Total frontend unitarias	137

Cuadro 6.2: Distribución de casos unitarios por módulo en frontend

6.5.3. Pruebas E2E

Las pruebas end-to-end se ejecutan con Playwright sobre Chromium, y validan flujos de interacción completos desde la interfaz.

El enfoque de ejecución adoptado en este proyecto es **E2E mockeada**: el navegador se levanta contra el frontend real y se simulan endpoints API en los escenarios definidos. Esto reduce inestabilidad ambiental y facilita reproducibilidad en CI.

Se validan, entre otros, los escenarios:

- autenticación correcta,
- autenticación fallida,
- flujo de evaluaciones con filtros y descarga.

6.5.4. Mutación en frontend

Stryker se aplica sobre código de producción de servicios y utilidades, excluyendo archivos de test. El objetivo es verificar que los tests no solo ejecutan líneas, sino que detectan alteraciones semánticas en el comportamiento.

La cobertura mediante mutación en frontend se ha realizado con el siguiente procedimiento:

1. selección del alcance de mutación en `src/services/**/*.ts` y `src/utils/**/*.ts`;
2. exclusión explícita de tests y archivos de infraestructura (`*.test.*`, `*.spec.*`, `api.ts`, `index.ts`);
3. ejecución de `npm run test:mutation`, generando mutantes y relanzando Vitest para cada lote;
4. análisis de mutantes `survived/timeout` para reforzar casos de prueba débiles.

Como criterio de aceptación de esta fase, Stryker aplica umbrales `high=75`, `low=60` y `break=50`, siendo este último el que determina fallo de pipeline.

6.6– Pruebas en backend

6.6.1. Pruebas unitarias e integración

En backend se emplea JUnit junto con Spring Test y MockMvc. La validación cubre:

- lógica de servicios de dominio,
- validaciones y reglas de negocio,
- contratos REST de controladores,
- seguridad y autorización por contexto,
- entidades y conversiones DTO.

Entre los tests de referencia de mayor carga funcional destaca `EntregaServiceTest`, que valida escenarios de entrega, restricciones y evolución de estado.

6.6.2. Módulos testeados en backend

El backend se ha cuantificado a partir de nodos `testcase` en reportes Surefire, lo que permite reflejar correctamente pruebas parametrizadas y escenarios anidados.

Módulo backend	Casos
Servicios (<code>service</code>)	513
Controladores (<code>controller</code>)	144
Repositorios (<code>repository</code>)	47
DTOs y validaciones (<code>dto</code>)	40
Seguridad (<code>security</code>)	31
Entidades (<code>entity</code>)	22
Manejo de excepciones (<code>exception</code>)	10
Configuración (<code>config</code>)	8
Aplicación base (<code>root</code>)	1
Total backend	816

Cuadro 6.3: Distribución de casos de prueba por módulo en backend

6.6.3. Cobertura backend

La cobertura se genera con JaCoCo durante la fase de test, produciendo informe HTML en `backend/target/site/jacoco/index.html`.

Este reporte se utiliza como soporte de revisión técnica para detectar módulos subprobados y orientar ampliaciones de suite.

6.6.4. Mutación en backend

El perfil Maven `mutation` ejecuta PIT sobre clases objetivo de servicios y controladores seleccionados. La configuración define umbrales de mutación y cobertura, y genera reportes estructurados para su análisis.

En esta implementación se mutan de forma controlada 10 clases objetivo (7 de servicio y 3 de controlador). Las clases objetivo son:

- `OneDriveService`

- ZipValidationService
- FeedbackService
- SecurityContextUserService
- CloudinaryService
- OneDriveController
- MicrosoftOAuthController
- FeedbackController

El flujo seguido para cobertura de mutación en backend ha sido:

1. ejecución del perfil con `mvn -Pmutation ... mutationCoverage`;
2. generación de mutantes con operadores de retorno, aritmética e incrementos;
3. relación mutante–test para identificar qué prueba mata cada defecto sintético;
4. revisión de mutantes supervivientes para crear pruebas adicionales o ajustar aserciones.

El perfil exige además un mínimo de mutación del 70 % y de cobertura del 75 %, reforzando que la cobertura no sea solo de líneas, sino de detección efectiva de fallos.

6.7– Automatización en CI/CD

La validación se integra en pipelines de GitHub Actions, lo que garantiza ejecución repetible por cada cambio relevante.

Workflows principales:

- `frontend-ci.yml`: lint, unitarias con cobertura, E2E y build de frontend.
- `full-stack-ci.yml`: verificación backend (`mvn verify`) y frontend completo.
- `mutation-ci.yml`: mutación backend (PIT) y frontend (Stryker), con subida de artefactos.

Esta automatización reduce el riesgo de integración tardía, y permite detectar regresiones antes de despliegues a entornos superiores.

6.8– Resultados verificados

De acuerdo con el informe de estrategia de testing del proyecto (corte de verificación final), se obtienen los siguientes resultados representativos:

Estos valores muestran una madurez de testing consistente con el nivel de exigencia de un TFG de orientación profesional.

Métrica	Resultado
E2E frontend (Playwright)	3 escenarios ejecutados, 3 en verde.
Cobertura frontend	Statements 97.77 %, Branches 90.68 %, Functions 98.19 %, Lines 98.25 %.
Suite frontend	137 tests aprobados en 21 archivos durante la ejecución del comando test:coverage.
Suite backend completa	816 casos en 56 suites Surefire, 0 fallos y 0 errores.
Total pruebas automáticas	956 ejecuciones (137 unitarias frontend + 3 E2E + 816 backend).
Mutación frontend (Stryker)	Score global 98.61 % (por encima del umbral break=50).
Mutación backend (PIT)	Mutation score 93 %, line coverage 95 %, test strength 96 %.
Test objetivo backend	EntregaServiceTest: 89 tests, 0 fallos, 0 errores.

Cuadro 6.4: Resumen de resultados de verificación

6.9– Criterios de aceptación

Como criterio operativo de calidad se establecen los siguientes mínimos:

- `lint` sin errores,
- suites unitarias/integración/E2E en verde,
- coberturas por encima de umbrales configurados,
- métricas de mutación por encima de umbral de ruptura,
- ausencia de regresiones funcionales en flujos críticos.

6.10– Limitaciones y riesgos

Aunque la estrategia es robusta, se identifican líneas de mejora:

- ampliar E2E con backend real en entorno de staging para ciertos casos,
- extender mutación a mayor superficie de clases backend,
- reforzar pruebas de carga y comportamiento en condiciones extremas.

No obstante, las limitaciones actuales no comprometen la validez de la verificación para el alcance definido del proyecto.

6.11– Conclusiones del capítulo

La estrategia de pruebas aplicada combina cobertura, profundidad y automatización, aportando evidencia objetiva de calidad funcional y técnica.

La incorporación de mutación como criterio adicional permite evaluar la eficacia real de la suite, superando una visión meramente cuantitativa de cobertura.

En conjunto, los resultados respaldan que el sistema dispone de una base de validación sólida para mantenimiento, evolución y despliegue continuo.

Manual de usuario

7.1– Introducción

Este capítulo describe el uso operativo del sistema desde la perspectiva de sus usuarios finales. El objetivo no es detallar la implementación interna, sino proporcionar una guía clara para ejecutar las tareas más frecuentes de forma correcta y segura.

El sistema dispone de tres perfiles principales: **administrador**, **profesor** y **estudiante**. Cada perfil accede a vistas y operaciones diferentes según permisos.

7.2– Acceso al sistema

7.2.1. Requisitos previos de uso

Antes de operar con normalidad en la plataforma, se recomienda verificar las siguientes condiciones:

- disponer de navegador actualizado en versión estable;
- comprobar conectividad de red durante subida de archivos de gran tamaño;
- utilizar credenciales personales y no compartidas;
- validar que el perfil asignado (administrador/profesor/estudiante) coincide con el rol esperado.

Estas precondiciones reducen incidencias de sesión, errores de permisos y fallos de subida de materiales.

7.2.2. Pantallas de acceso

El acceso se realiza desde la pantalla de inicio de sesión. Las rutas funcionales principales son:

- `/login`: inicio de sesión.
- `/forgot-password`: solicitud de recuperación de contraseña.
- `/reset-password`: establecimiento de nueva contraseña mediante token.

Tras autenticarse correctamente, el usuario es redirigido al panel principal (`/dashboard`). Si el usuario ya tiene sesión activa, la ruta de inicio redirige automáticamente al panel.

7.2.3. Ciclo de sesión

La sesión se basa en token de acceso y token de refresco. Cuando el token de acceso expira, el cliente intenta renovarlo automáticamente. Si la renovación falla, la sesión se cierra y el usuario debe autenticarse de nuevo.

7.2.4. Recuperación de contraseña

El flujo de recuperación es el siguiente:

1. El usuario solicita recuperación indicando su correo.
2. El sistema emite un token temporal y envía instrucciones de restablecimiento.
3. El usuario accede al formulario de nueva contraseña e introduce token y clave nueva.

7.3– Perfiles y permisos

La Tabla 7.1 resume capacidades por perfil.

Perfil	Operaciones principales
Administrador	Gestionar usuarios, cursos y grupos; asignar o retirar roles; mantener estructura académica.
Profesor	Crear/editar actividades; crear/editar entregables; revisar entregas; calificar; publicar/ocultar notas.
Estudiante	Consultar cursos y actividades visibles; realizar entregas; consultar historial, feedback y calificaciones según visibilidad.

Cuadro 7.1: Capacidades por perfil de usuario

7.4– Vistas comunes a todos los perfiles

Las siguientes pantallas son compartidas por los tres perfiles del sistema:

7.5– Guía de uso para profesorado

7.5.1. Vistas principales

7.5.2. Crear y configurar una actividad

Flujo recomendado:

1. Acceder a `/cursos` y seleccionar un curso.
2. Crear actividad desde el detalle del curso.
3. Definir título, descripción, fechas y visibilidad.
4. Asignar grupos destinatarios cuando aplique.

Validaciones clave:

- El título es obligatorio.
- La fecha de inicio no puede ser posterior a la fecha límite.
- La visibilidad condiciona si el alumnado puede ver la actividad.

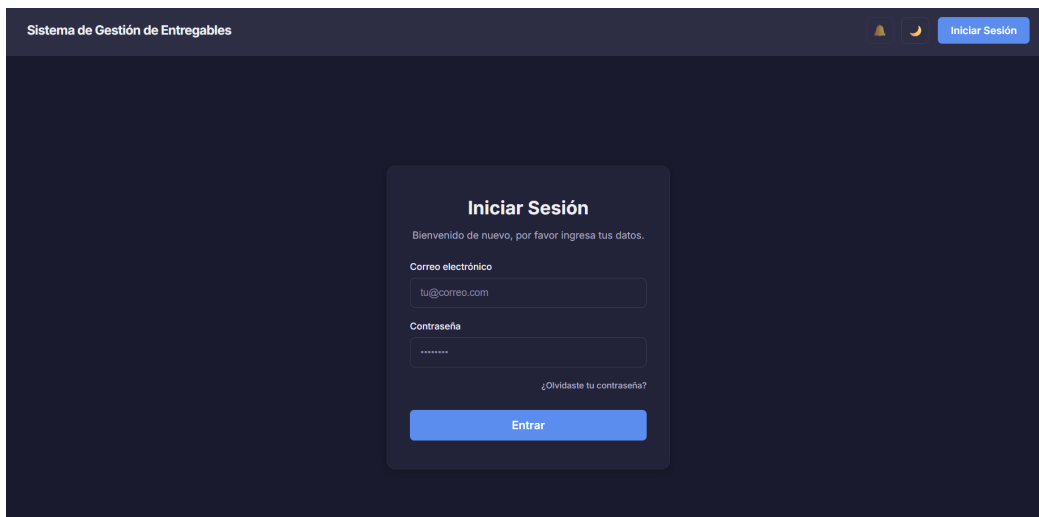


Figura 7.1: Pantalla de inicio de sesión. Vista compartida por todos los perfiles.

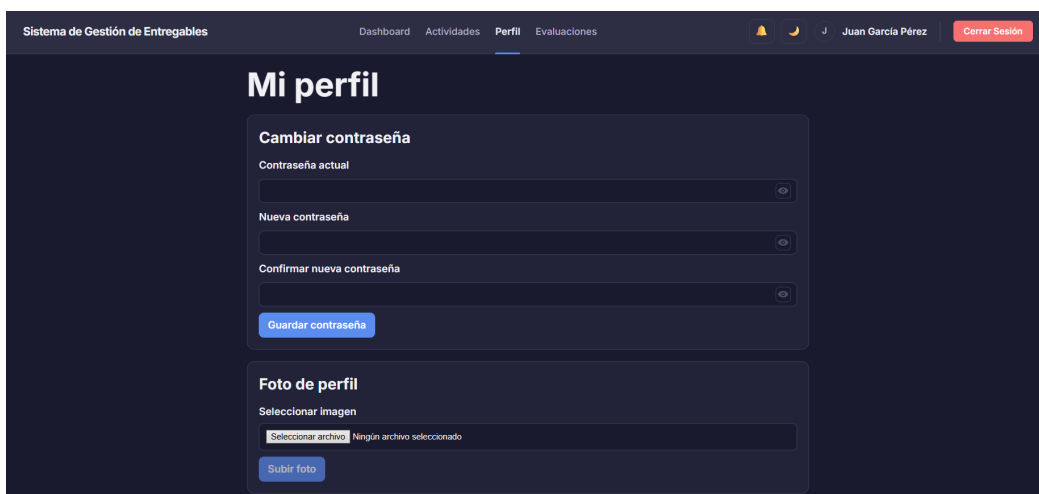


Figura 7.2: Sección de perfil (1/2). Vista compartida por todos los perfiles.



Figura 7.3: Sección de perfil (2/2). Vista compartida por todos los perfiles.

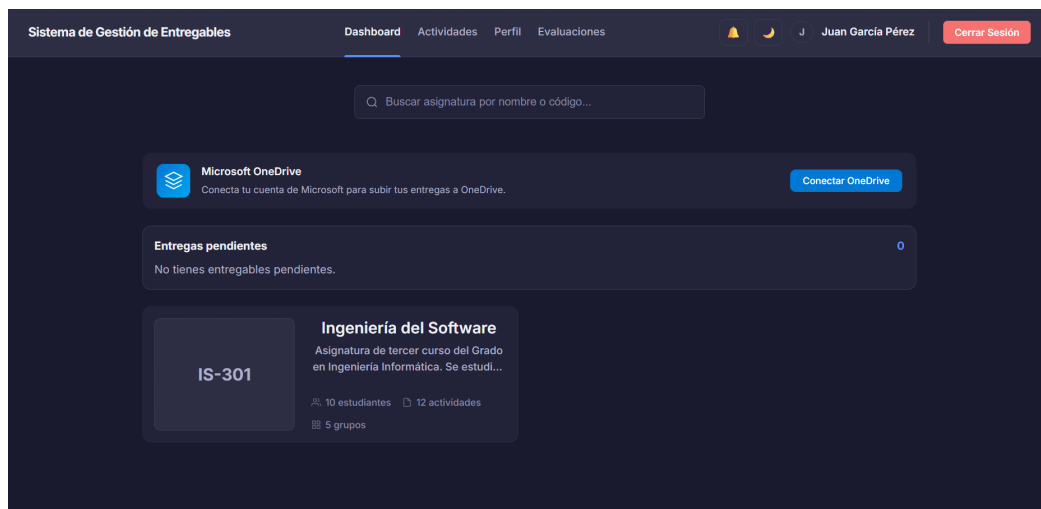


Figura 7.4: Dashboard docente. Secciones visibles: dashboard, actividades, perfil y evaluaciones.

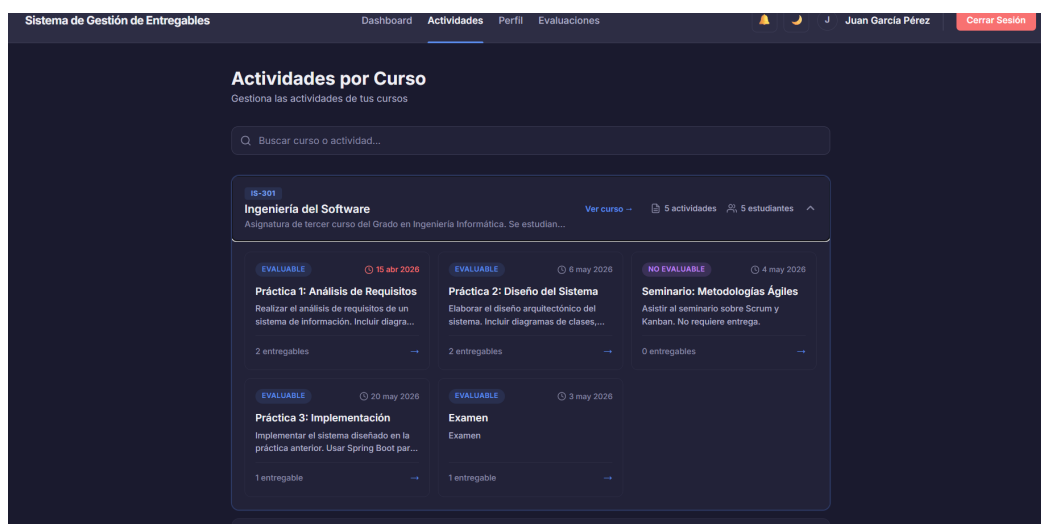


Figura 7.5: Sección de actividades por curso.

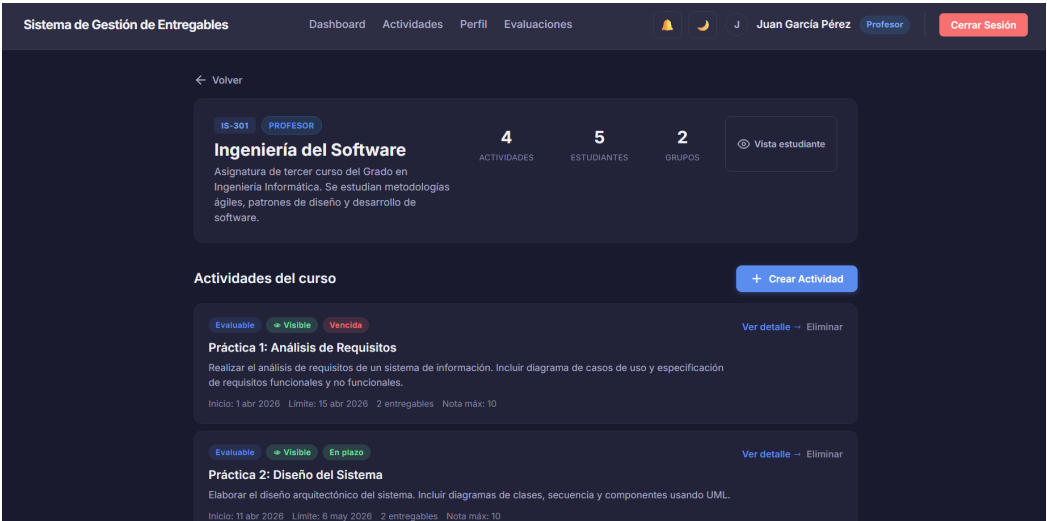


Figura 7.6: Detalle de asignatura con profesorado asignado.

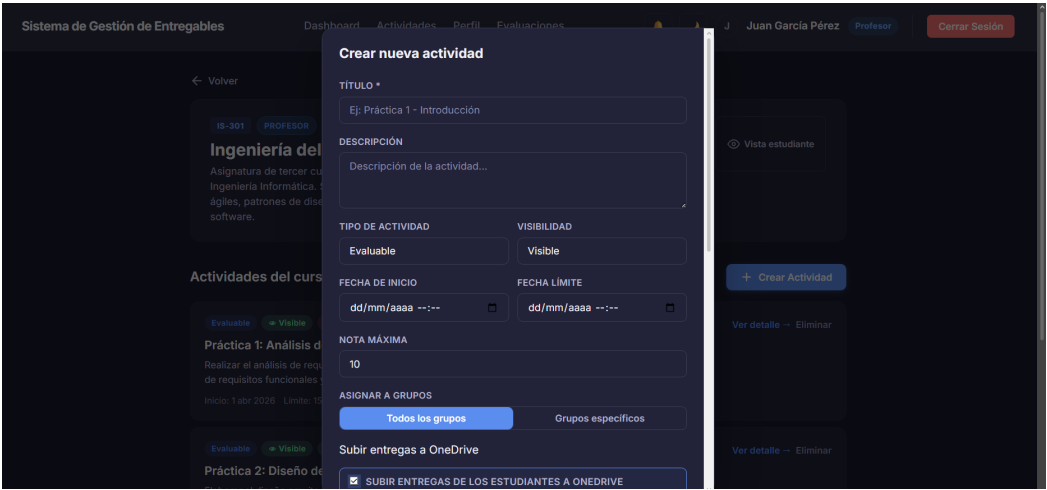


Figura 7.7: Creación de actividad (1/2).

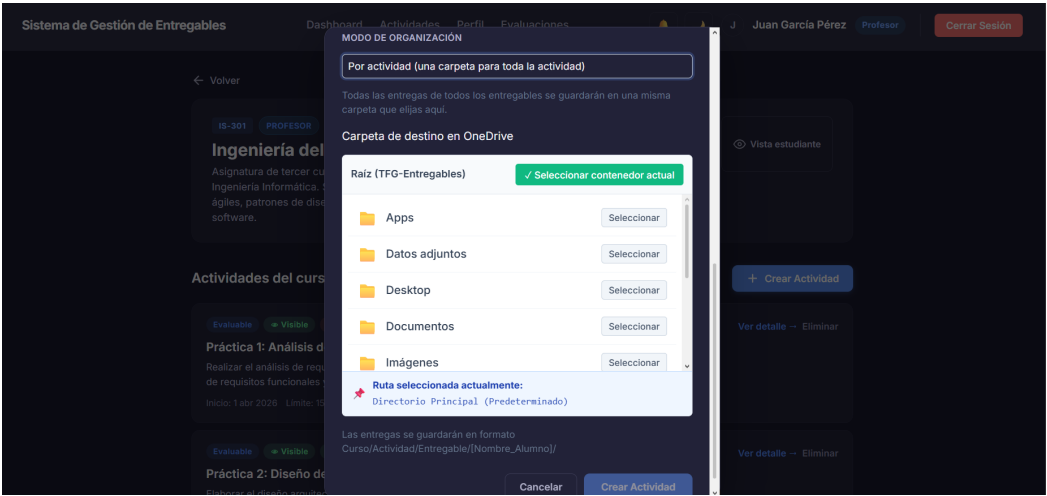


Figura 7.8: Creación de actividad (2/2).

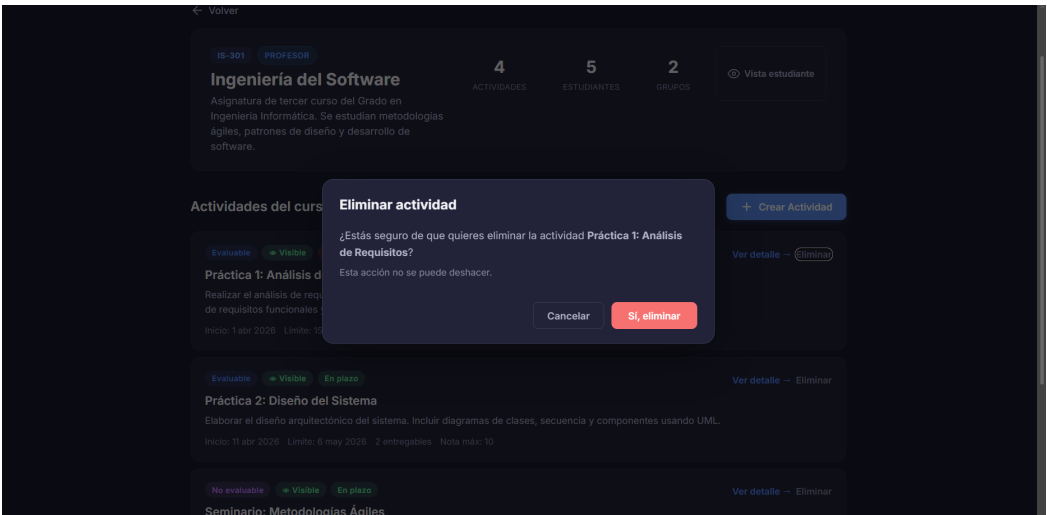


Figura 7.9: Eliminación de actividad desde la asignatura.

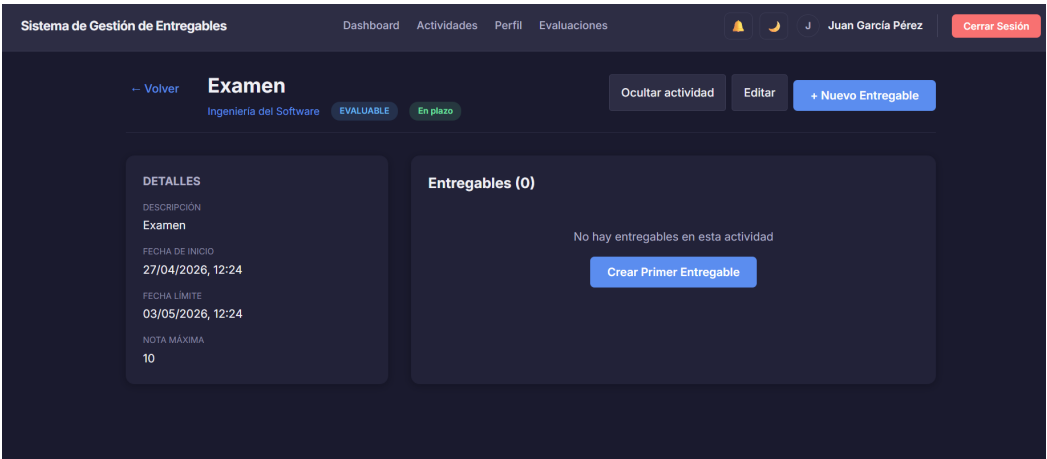


Figura 7.10: Detalle de actividad y entregables asociados.

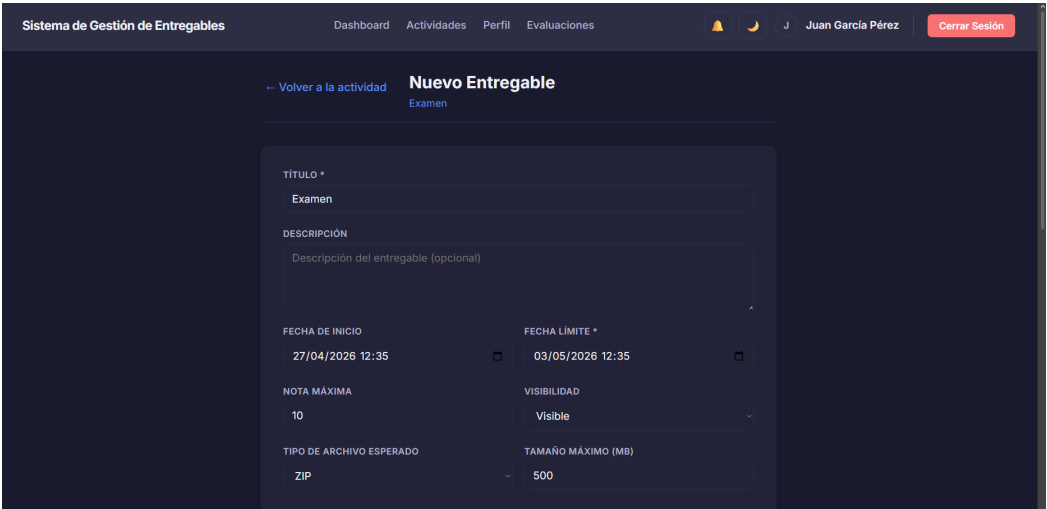


Figura 7.11: Creación de entregables (1/2).

Estructura esperada del ZIP
Define la estructura de archivos y carpetas que debe contener el ZIP. Usa * como nombre para aceptar cualquier nombre.

Importar estructura desde ZIP
Sube un ZIP de ejemplo y se extraerá su estructura automáticamente.

☒ **NOMBRES Y EXTENSIONES**
MANTIENE LOS NOMBRES DE ARCHIVOS Y SUS EXTENSIONES

☐ **SOLO NOMBRES**
MANTIENE LOS NOMBRES PERO IGNORA LAS EXTENSIONES

☐ **SOLO ESTRUCTURA**
USA COMODIN (*) PARA NOMBRES, SOLO CONSERVA EXTENSIONES

NOMBRE DEL ARCHIVO ZIP ESPERADO
Ej: proyecto_final (deja vacío o usa * para cualquier nombre) .zip
Si está vacío o es *, se acepta cualquier nombre de archivo ZIP.

☐ **MÍNIMO REQUERIDO**
EL ZIP DEBE CONTENER AL MENOS ESTOS ARCHIVOS (PUEDE TENER MÁS)

☒ **ESTRUCTURA EXACTA**
EL ZIP DEBE CONTENER EXACTAMENTE ESTOS ARCHIVOS, NI MÁS NI MENOS

☒ **Permite reenvío**
Si está activado, los estudiantes pueden enviar nuevas versiones de su entrega.

Figura 7.12: Creación de entregables (2/2).

Sistema de Gestión de Entregables Dashboard Actividades Perfil Evaluaciones Juan García Pérez Cerrar Sesión

[Volver al detalle](#) **Editar Actividad**
Ingeniería del Software

TÍTULO *
Examen

DESCRIPCIÓN
Examen

TIPO DE ACTIVIDAD * Evaluable **VISIBILIDAD** Visible

FECHA DE INICIO 27/04/2026 12:24 **FECHA LÍMITE** 03/05/2026 12:24

NOTA MÁXIMA
10

Figura 7.13: Edición de actividad (1/3).

NOTA MÁXIMA

10

GRUPOS ASIGNADOS

Todos los grupos Grupos específicos

SUBIR ENTREGAS A ONEDRIVE

☒ LAS ENTREGAS DE LOS ESTUDIANTES SE SUBIRÁN AUTOMÁTICAMENTE A TU ONEDRIVE

MODO DE ORGANIZACIÓN

Por actividad (una carpeta para toda la actividad)

Todas las entregas de todos los entregables se guardarán en una misma carpeta que elijas aquí.

CARPETA DE DESTINO EN ONEDRIVE

Raíz (TFG-Entregables) ✓ Seleccionar contenedor actual

Apps	Seleccionar
Datos adjuntos	Seleccionar
Desktop	Seleccionar
Documentos	Seleccionar
Imágenes	Seleccionar

Figura 7.14: Edición de actividad (2/3).

MODO DE ORGANIZACIÓN

Por actividad (una carpeta para toda la actividad)

Todas las entregas de todos los entregables se guardarán en una misma carpeta que elijas aquí.

CARPETA DE DESTINO EN ONEDRIVE

Raíz (TFG-Entregables) ✓ Seleccionar contenedor actual

Apps	Seleccionar
Datos adjuntos	Seleccionar
Desktop	Seleccionar
Documentos	Seleccionar
Imágenes	Seleccionar

✖ Ruta seleccionada actualmente:
Pruebas TFG/Prueba 3

Las entregas se guardarán en "Pruebas TFG/Prueba 3/[Nombre_Entregable]/[Nombre_Alumno]"

Eliminar actividad Cancelar Guardar cambios

Figura 7.15: Edición de actividad (3/3).

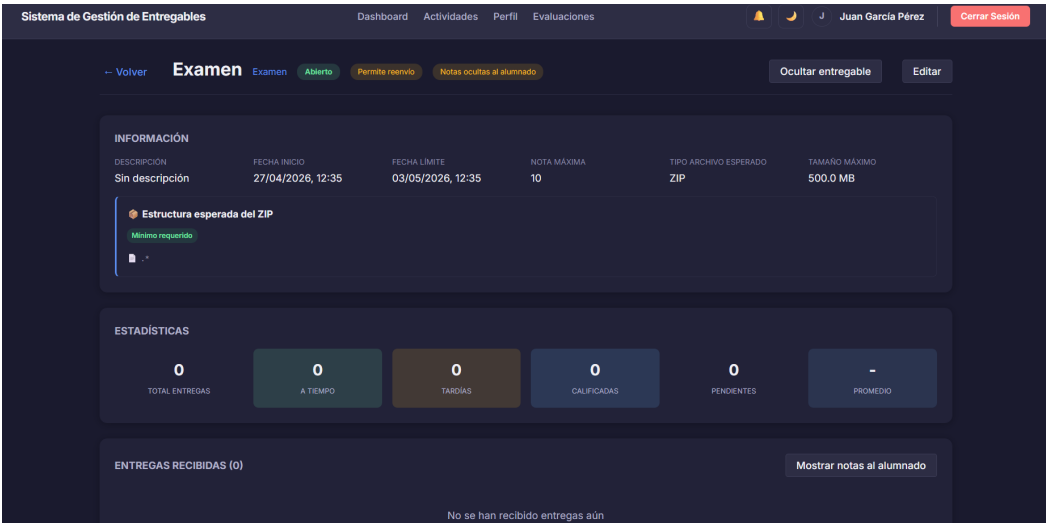


Figura 7.16: Detalle de entregable para revisiones.

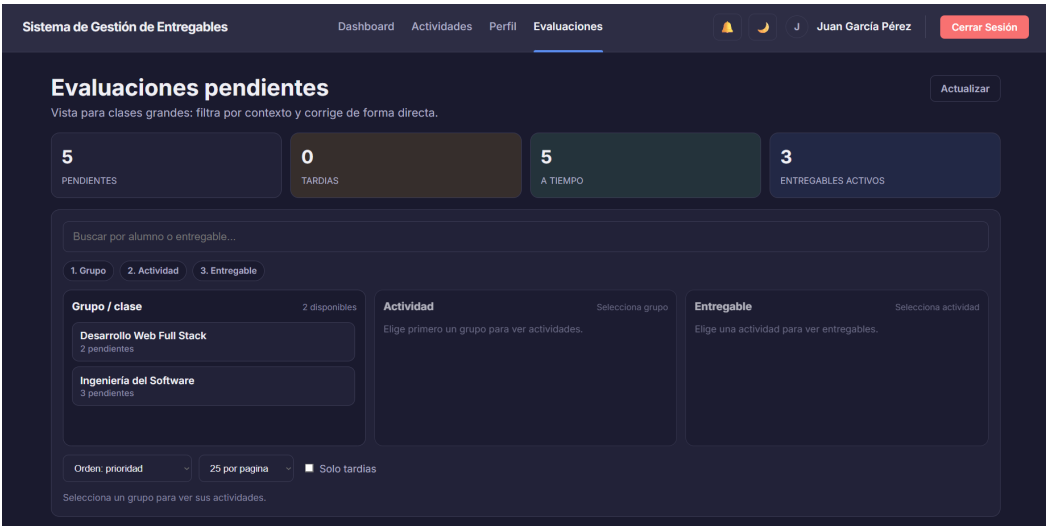


Figura 7.17: Bandeja de evaluaciones (1/2).

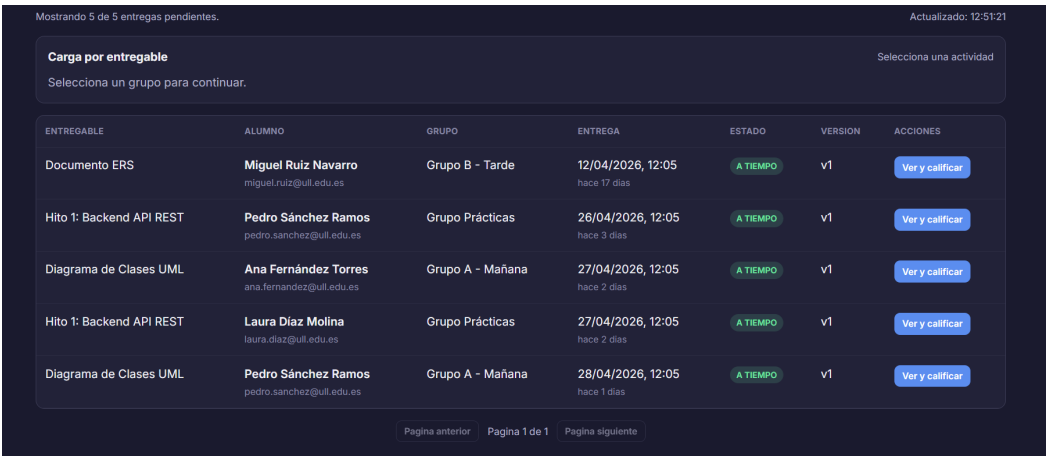


Figura 7.18: Bandeja de evaluaciones (2/2).

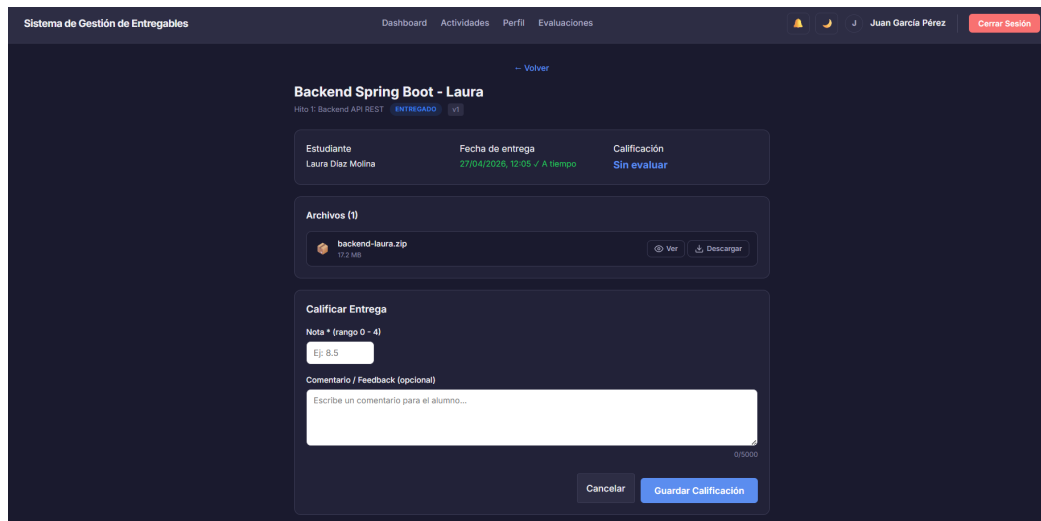


Figura 7.19: Evaluación de la actividad de un alumno.

7.5.3. Crear y editar entregables

Desde la actividad, el profesor puede crear entregables desde la vista de alta de entregables del detalle de actividad.

Parámetros relevantes de configuración:

- Título, descripción y fecha límite.
- Tipo de archivo esperado (PDF, DOCX, ZIP, imagen, video, texto, enlace, etc.).
- Tamaño máximo permitido.
- Permitir o no reenvío.
- Visibilidad de notas para estudiantes.

Cuando el tipo esperado es ZIP, puede definirse estructura esperada y validación estricta, lo que obliga a que la entrega siga el formato indicado.

7.5.4. Revisar entregas y evaluar

Las entregas pueden revisarse desde:

- `/entregables/:id` (listado por entregable),
- `/evaluaciones` (bandeja de pendientes de calificación).

Acciones habituales:

1. Filtrar por curso, actividad, entregable y estado temporal.
2. Abrir detalle de entrega (`/entregas/:id`).
3. Revisar archivos, comentario y metadatos.
4. Registrar nota y feedback.
5. Publicar/ocultar visibilidad de nota al estudiante.

El sistema permite descargar entregas de forma agregada en ZIP, tanto a nivel de entregable como de actividad completa.

7.5.5. Flujo docente recomendado de extremo a extremo

Para maximizar consistencia y reducir retrabajo, se propone la siguiente secuencia de trabajo docente:

1. crear actividad con fechas y grupos correctamente delimitados;
2. publicar entregables con requisitos de formato y tamaño explícitos;
3. revisar periódicamente estado de entregas pendientes;
4. corregir por lotes homogéneos (por grupo o por entregable);
5. registrar feedback accionable y criterio de nota transparente;
6. publicar calificaciones cuando el bloque de corrección esté validado.

Este flujo mejora la trazabilidad y evita inconsistencias entre estado de evaluación y comunicación al alumnado.

7.6– Guía de uso para estudiantes

7.6.1. Vistas principales

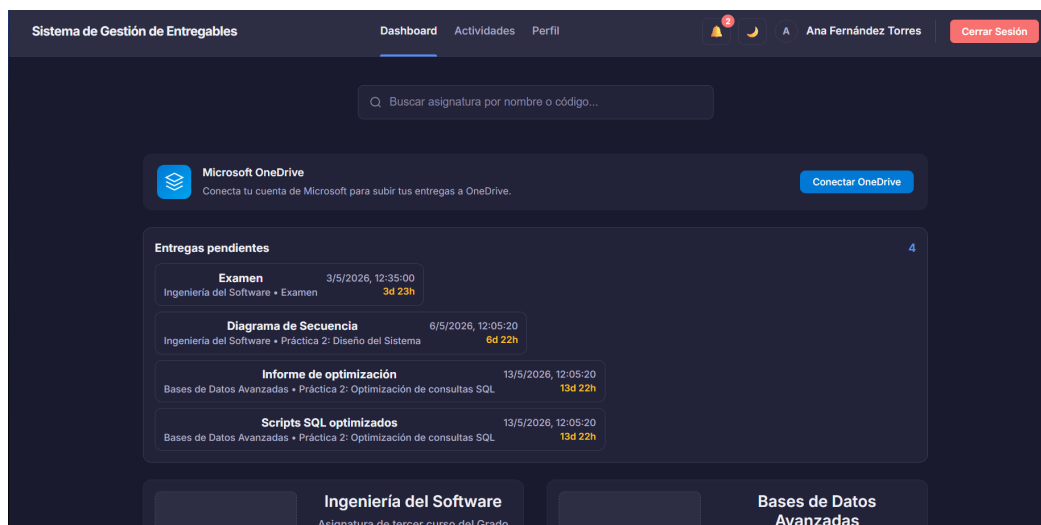


Figura 7.20: Dashboard de estudiante. Secciones visibles: panel principal, actividades, perfil y calificaciones.

7.6.2. Consultar actividades y entregables

Flujo recomendado:

1. Acceder al panel y entrar en /cursos.
2. Seleccionar curso y revisar actividades visibles.
3. Entrar en el detalle de actividad y localizar entregables disponibles.

La visibilidad de actividad/entregable depende de la configuración docente y de la pertenencia del estudiante a grupos autorizados.

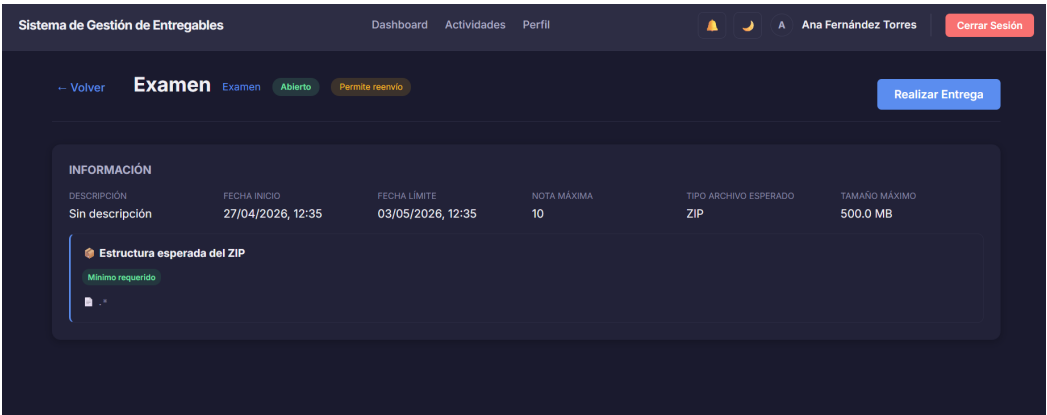


Figura 7.21: Detalle de entregable desde la visión de estudiante.

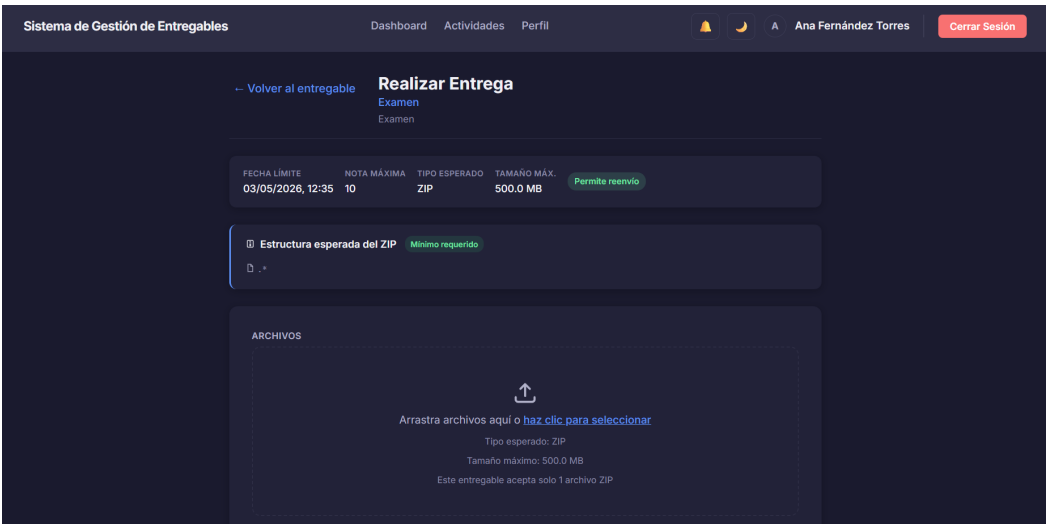


Figura 7.22: Formulario de entrega (1/2).

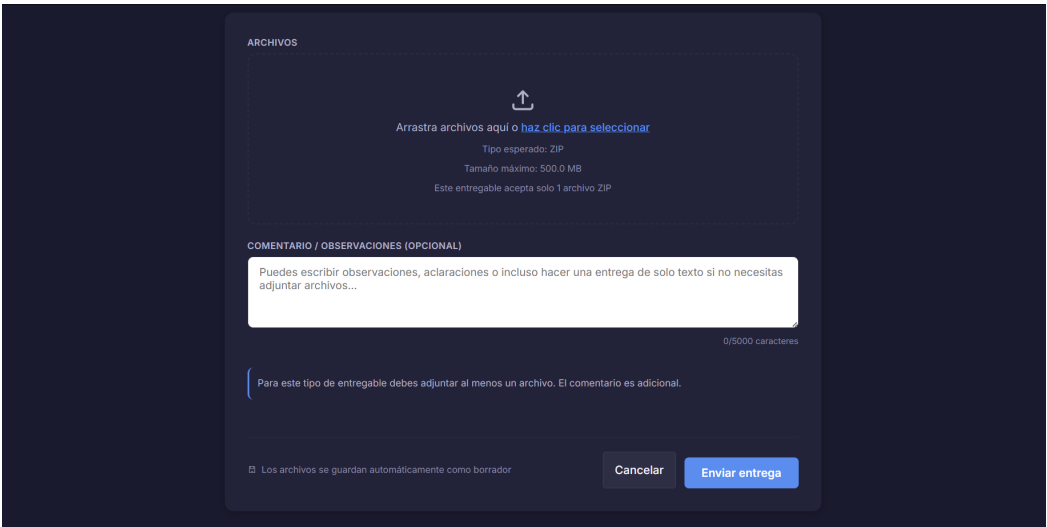


Figura 7.23: Formulario de entrega (2/2).

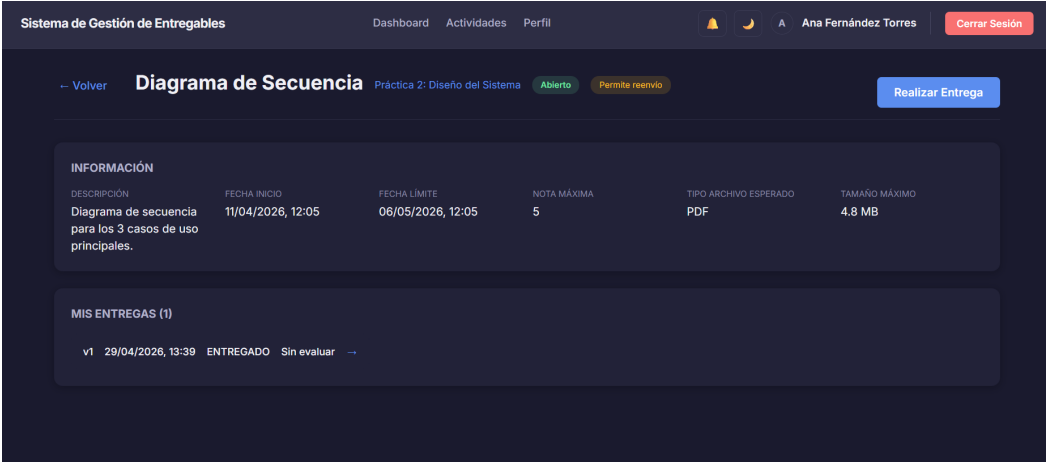


Figura 7.24: Entrega registrada correctamente.



Figura 7.25: Consulta de calificación por estudiante.

7.6.3. Realizar una entrega

La subida se realiza en `/entregables/:id/entregar`. El formulario adapta su comportamiento al tipo de entrega:

- **SOLO_TEXTO**: no admite archivos; requiere comentario.
- **ENLACE**: no admite archivos; el contenido debe ir en comentario.
- **ZIP**: exige archivo ZIP y puede validar estructura.
- **Otros tipos**: valida extensión y tamaño máximo según configuración.

Además, el formulario incluye guardado temporal en local (borrador), lo que reduce riesgo de pérdida de trabajo si se interrumpe la sesión o falla la conexión.

7.6.4. Recomendaciones para entregas sin incidencias

Para minimizar errores de validación, se recomienda al estudiante:

- revisar el tipo de entrega requerido antes de preparar el material;
- verificar nombre y extensión de archivos antes de subir;
- evitar subir archivos en el último minuto para mitigar riesgos de red;
- confirmar en la vista de detalle que la versión final ha quedado registrada.

En entregas de tipo ZIP, si existe estructura obligatoria, conviene validar localmente el contenido antes de la subida.

7.6.5. Consultar historial y feedback

En el detalle de entrega (`/entregas/:id`) el estudiante puede:

- revisar versiones entregadas,
- descargar/previsualizar materiales,
- consultar feedback y nota cuando estén publicadas.

Si la nota no es visible, la entrega aparece sin calificación pública, aunque internamente ya haya sido evaluada por el profesor.

7.7– Guía de uso para administración

7.7.1. Vistas principales

7.7.2. Panel de administración

El panel `/admin` está organizado en tres pestañas principales:

- **Usuarios**: alta, edición, eliminación y asignación de roles.
- **Cursos**: creación y mantenimiento de cursos.
- **Grupos**: creación y gestión de grupos asociados a cursos.

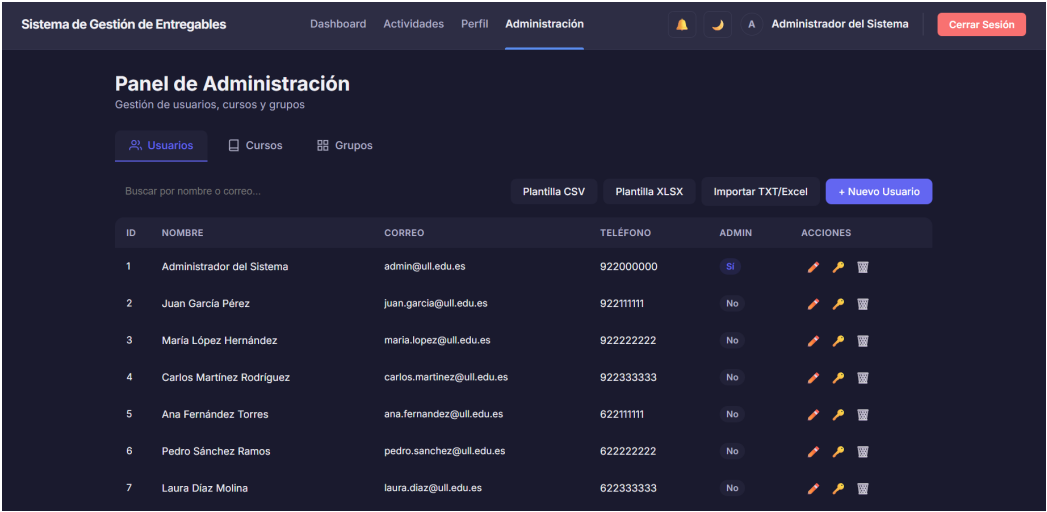


Figura 7.26: Panel de administración. Secciones visibles: usuarios, cursos y grupos.

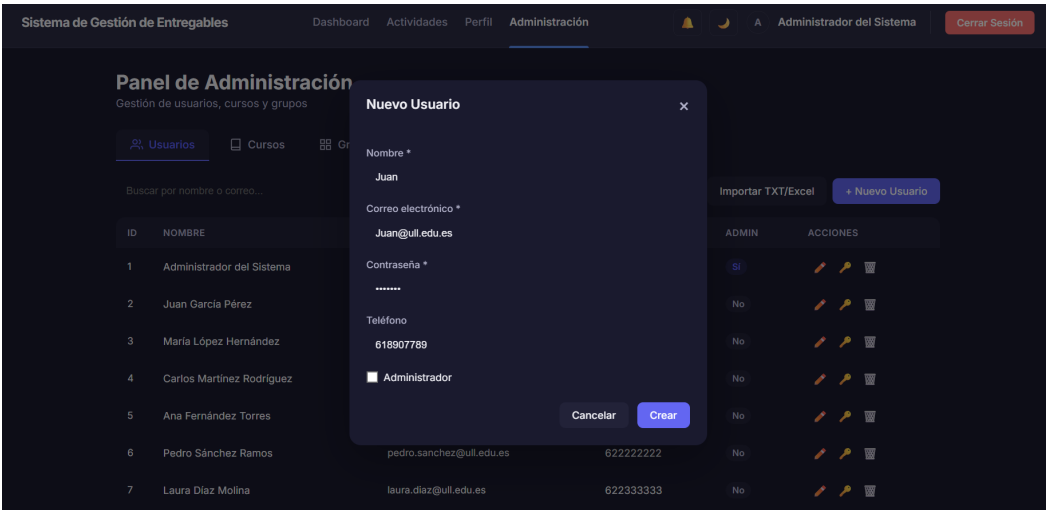


Figura 7.27: Creación de usuarios.

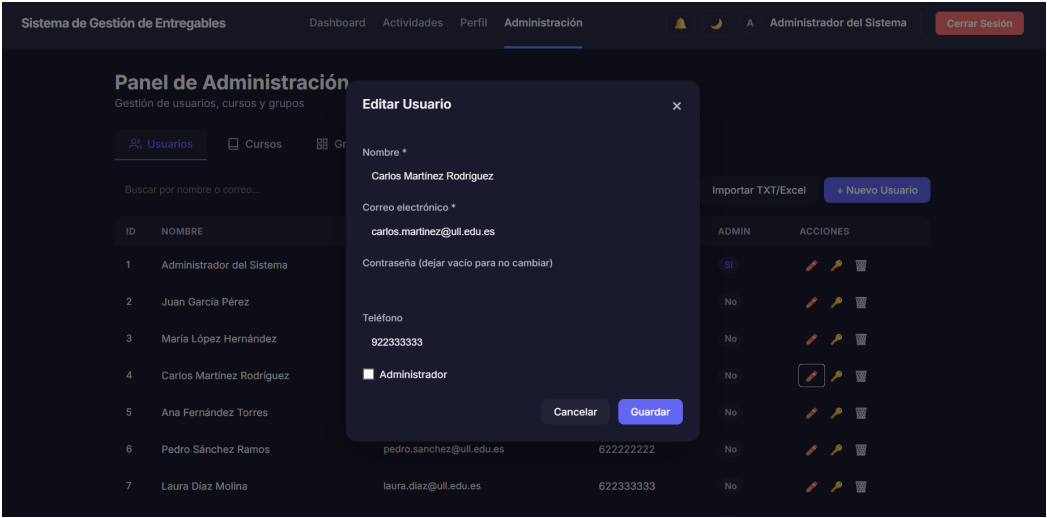


Figura 7.28: Modificación de usuarios.

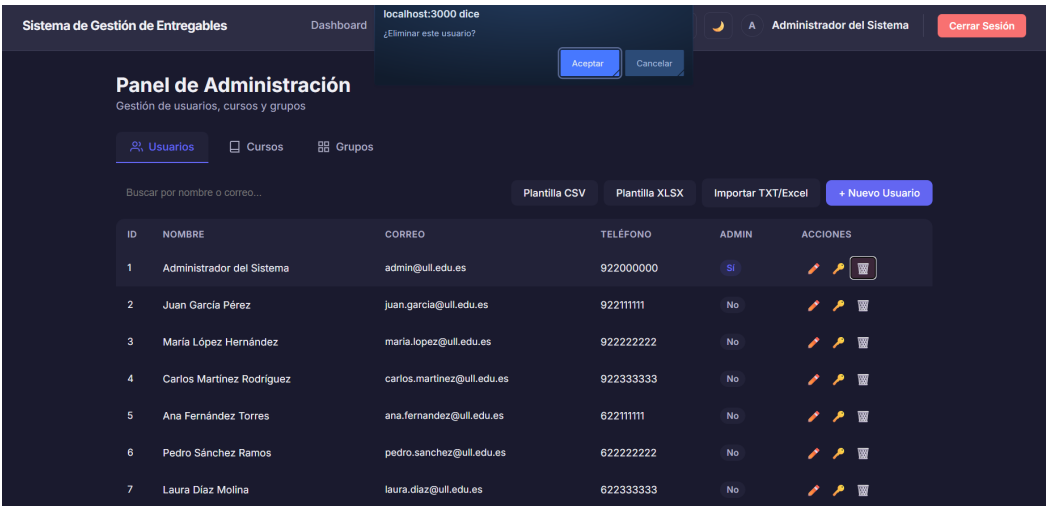


Figura 7.29: Eliminación de usuarios.

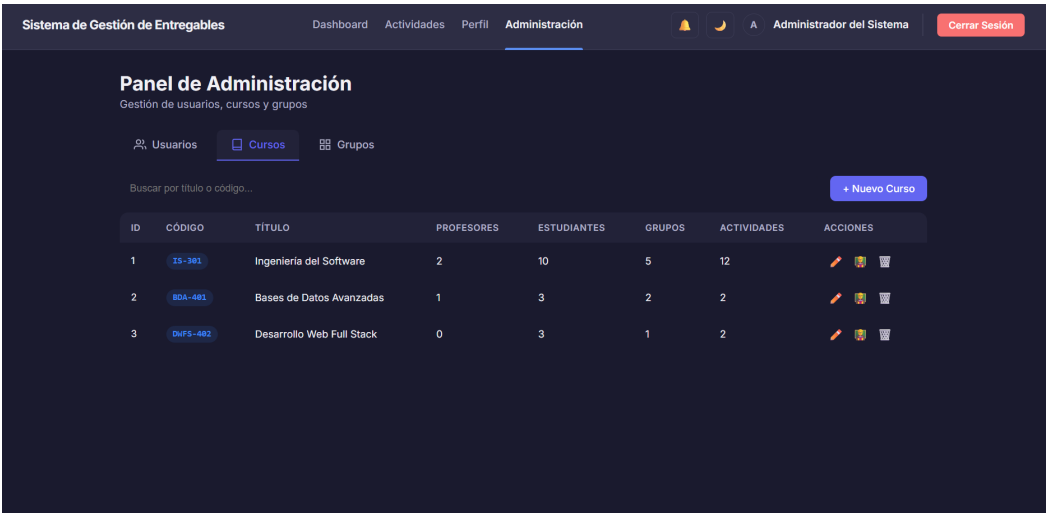


Figura 7.30: Gestión de cursos.

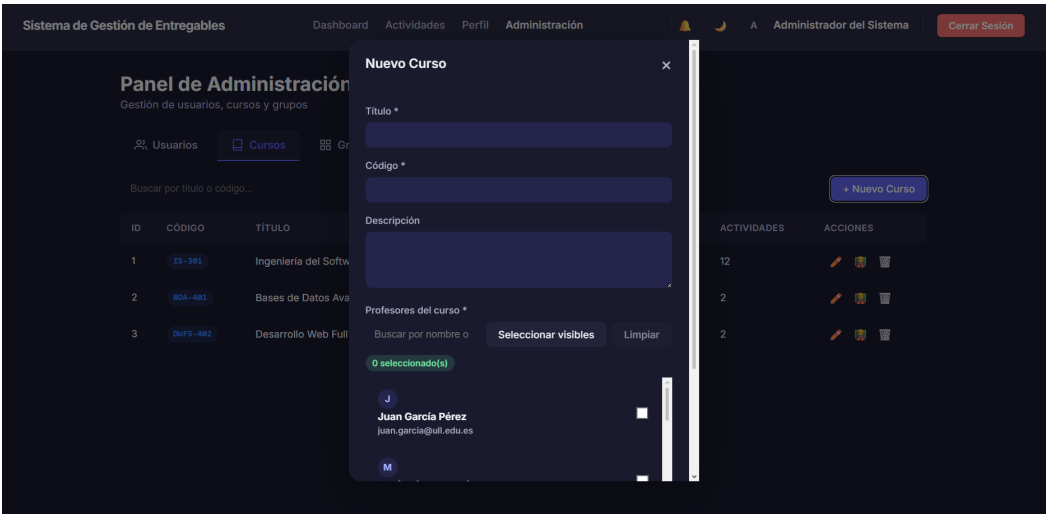


Figura 7.31: Creación de curso (1/2).

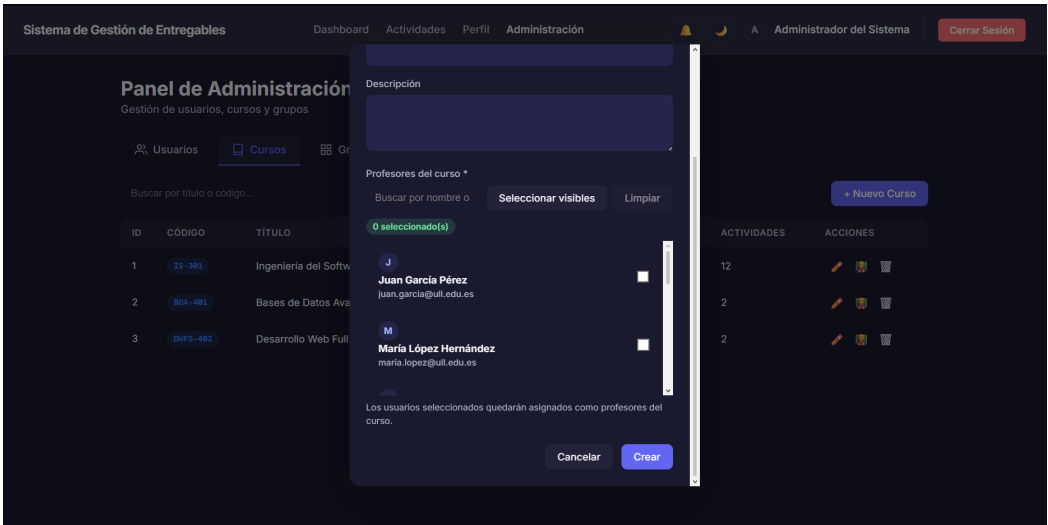


Figura 7.32: Creación de curso (2/2).

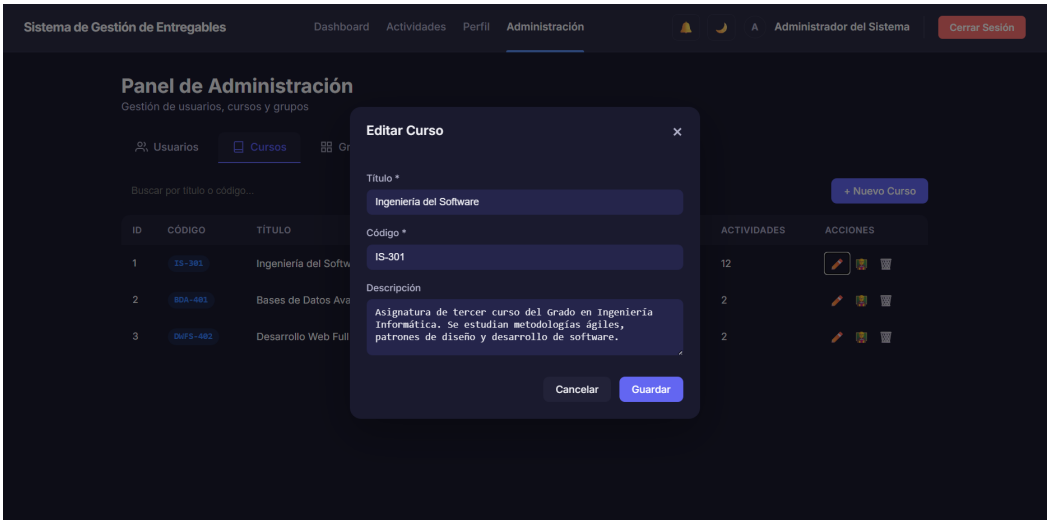


Figura 7.33: Edición de curso.

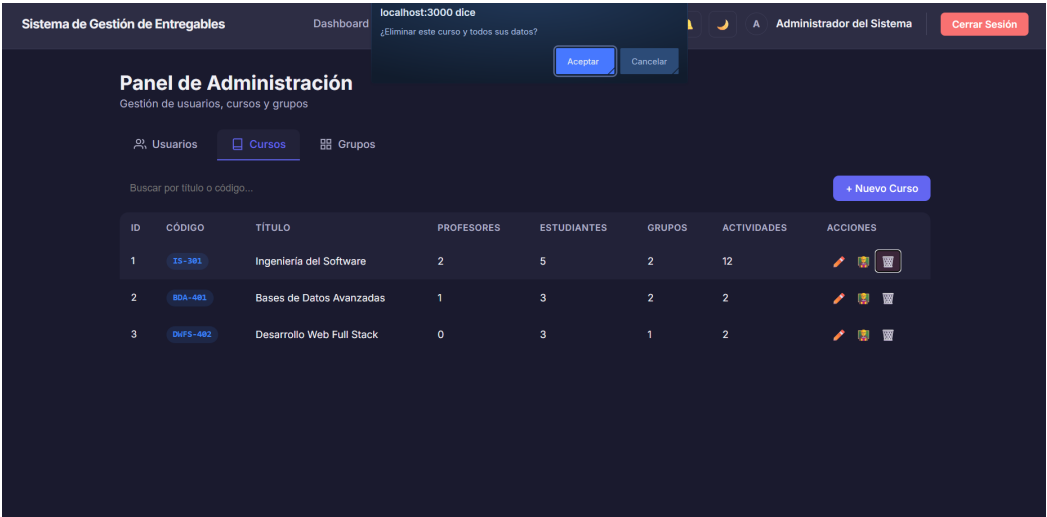


Figura 7.34: Eliminación de curso.

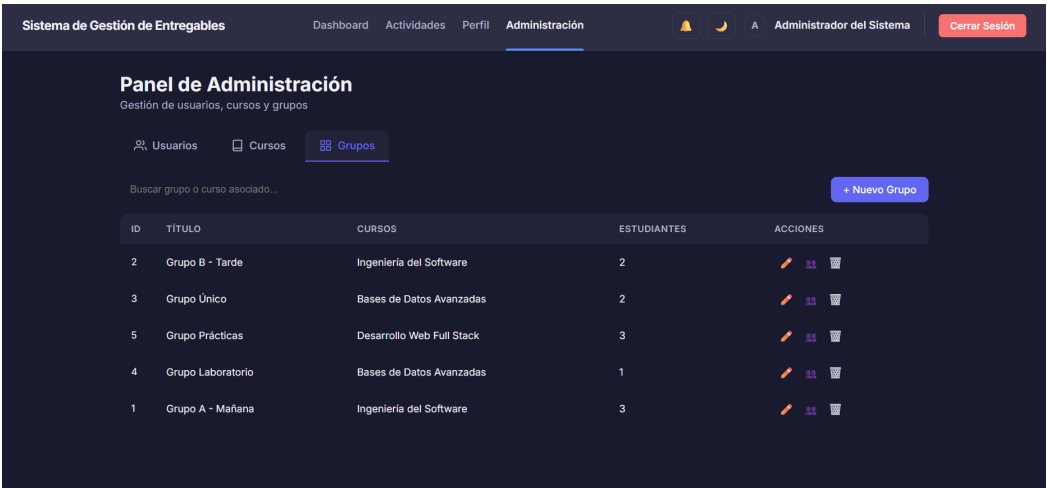


Figura 7.35: Gestión de grupos.

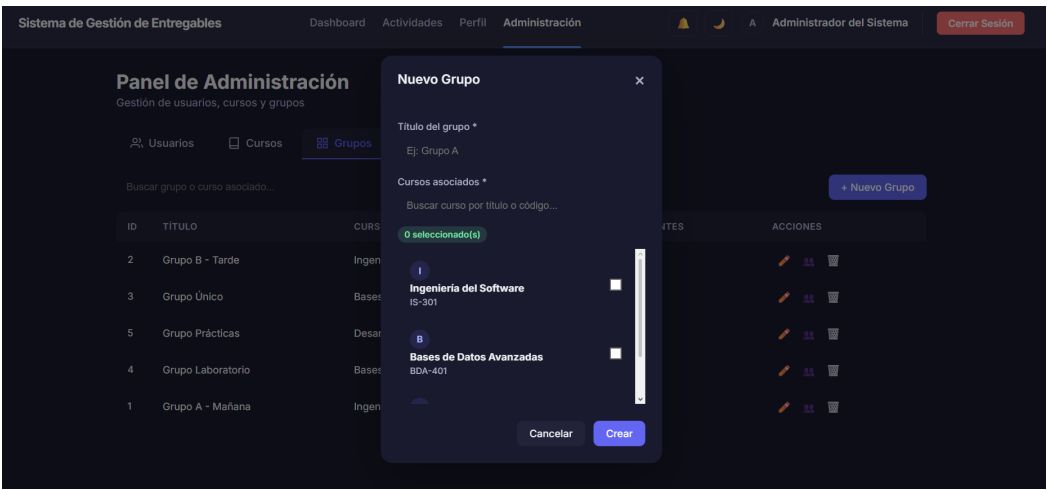


Figura 7.36: Creación de grupo.

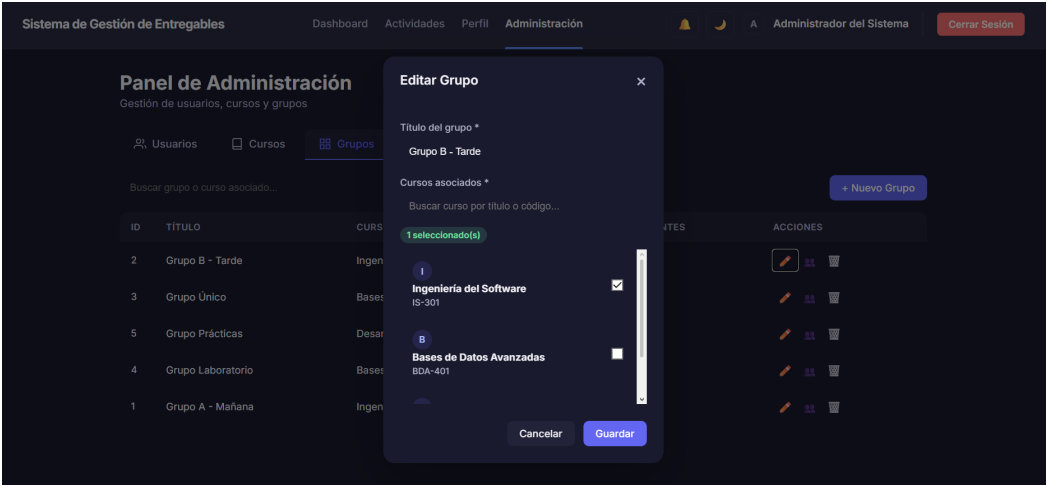


Figura 7.37: Edición de grupo.



Figura 7.38: Eliminación de grupo.

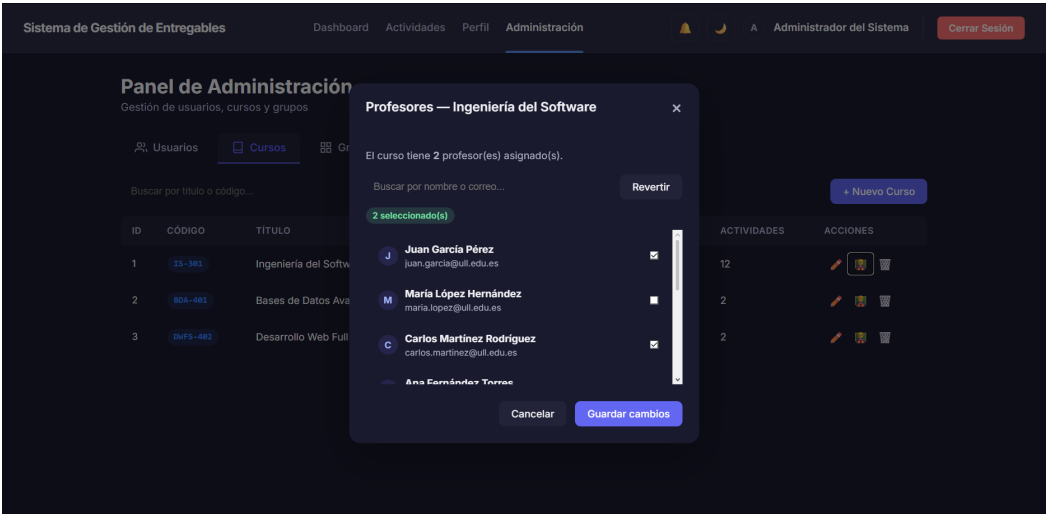


Figura 7.39: Asignación de profesorado a cursos.

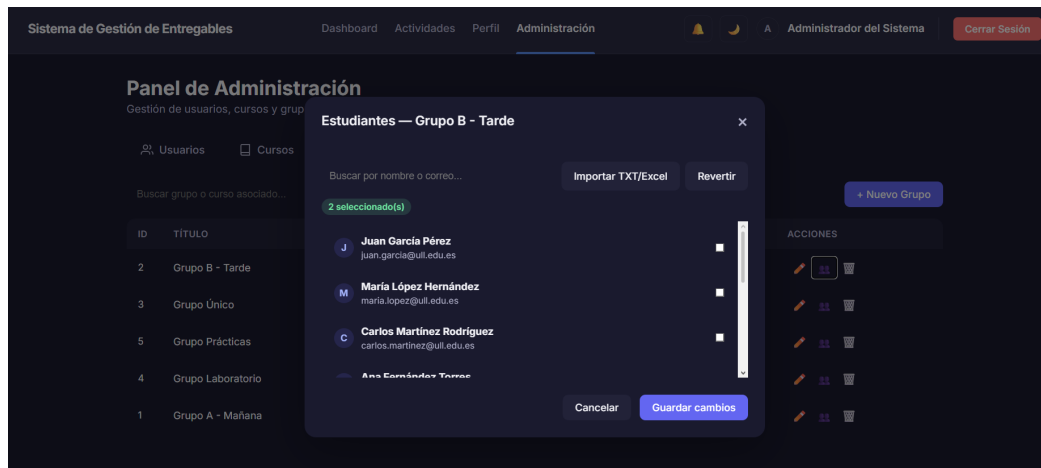


Figura 7.40: Asignación de usuarios en grupo.

7.7.3. Operaciones destacadas

El perfil administrador puede:

1. Crear usuarios individualmente o por importación de archivo.
2. Asignar rol de profesor o estudiante (con grupo en el caso de estudiante).
3. Crear cursos y vincular profesorado.
4. Estructurar grupos y reasignar alumnado.

Este perfil concentra operaciones de gobierno funcional, por lo que se recomienda limitar su uso a personal autorizado.

7.7.4. Políticas recomendadas de gobierno de usuarios

Para mantener orden operativo y seguridad, se recomienda aplicar las siguientes políticas de administración:

- principio de mínimo privilegio en asignación de roles;
- revisión periódica de cuentas inactivas o duplicadas;
- separación de cuentas de administración y cuentas de uso docente;
- registro de cambios sensibles en altas, bajas y cambios de rol.

7.8– Mensajes de error y resolución de incidencias

Las incidencias más frecuentes y su acción recomendada son:

- **Credenciales inválidas:** verificar correo y contraseña.
- **Sesión expirada:** iniciar sesión de nuevo.
- **Archivo no permitido:** revisar tipo/extensión/tamaño del entregable.

- **Sin permisos:** comprobar rol y pertenencia a curso/grupo.
- **Error al enviar entrega:** reintentar y comprobar conexión.

Como recomendación general, ante errores persistentes debe registrarse la hora, operación realizada y mensaje mostrado para facilitar diagnóstico.

7.8.1. Matriz de incidencias operativas

Incidencia	Causa habitual	Acción recomendada
No se puede iniciar sesión	Credenciales incorrectas o sesión expirada	Reintentar autenticación y, si procede, usar recuperación de contraseña
Entrega rechazada	Tipo de archivo o tamaño no permitido	Revisar configuración del entregable y ajustar archivo
No aparece actividad/entregable	Visibilidad desactivada o grupo no asignado	Verificar configuración por parte del profesorado
No se puede calificar	Falta de permisos por rol/-contexto	Confirmar vinculación docente con curso/grupo
Error en descarga ZIP	Interrupción de red o contenido no disponible	Reintentar descarga y revisar estado de materiales

Cuadro 7.2: Matriz de incidencias frecuentes y resolución recomendada

7.9– Seguridad y protección de la información en el uso diario

La seguridad operativa depende tanto de mecanismos técnicos como de buenas prácticas de uso. Para preservar confidencialidad e integridad de datos académicos, se recomienda:

- no compartir contraseñas ni tokens de sesión;
- cerrar sesión en dispositivos compartidos;
- evitar cargas de archivos desde equipos no confiables;
- revisar periódicamente permisos y visibilidad de notas.

Estas medidas reducen riesgos de acceso no autorizado y publicación accidental de información académica sensible.

7.10– Checklist operativo por perfil

Como apoyo de uso rápido, se incluye un checklist resumido para cada rol:

- **Administrador:** alta de usuarios, asignación de roles, revisión de estructura de cursos/grupos.
- **Profesor:** configuración de actividad, publicación de entregable, corrección y publicación de nota.

- **Estudiante:** revisión de actividad visible, entrega en plazo, verificación de estado y feedback.

El uso sistemático de este checklist disminuye errores de operación y mejora la consistencia del proceso académico.

7.11– Buenas prácticas de uso

Para minimizar incidencias operativas:

- comprobar siempre fecha límite y tipo de entrega antes de enviar,
- revisar la vista de detalle tras cada acción crítica,
- mantener actualizado el perfil de usuario,
- evitar compartir credenciales,
- cerrar sesión en equipos compartidos.

7.12– Alternativa o complemento a la enseñanza virtual

La plataforma se centra en el ciclo de actividades, entregables y evaluación. En contextos donde no se requiere un LMS completo, puede funcionar como alternativa ligera, con menor complejidad operativa y una curva de aprendizaje más directa.

Cuando ya existe un entorno como Moodle, el sistema puede actuar como complemento especializado para la gestión de entregas, correcciones y publicación de notas, manteniendo el resto de la docencia (contenidos, foros, cuestionarios) en el LMS. La convivencia es viable mediante enlaces cruzados y coordinación de calendarios, evitando duplicidades en el trabajo docente.

7.13– Aspectos novedosos

El proyecto introduce capacidades que amplían la operativa habitual de un LMS:

- **Uso de APIs de OneDrive y correo:** la integración con OneDrive permite autorizar el acceso del usuario a su cuenta (OAuth2) y gestionar entregas en su almacenamiento en la nube, mientras que el canal de correo (actualmente operado mediante Brevo) soporta procesos como recuperación de contraseña y notificaciones de evaluación.
- **Importación desde OneDrive:** se habilita la incorporación de la estructura de carpetas del usuario para organizar entregables y mantener trazabilidad con su repositorio personal.

7.14– Desafíos técnicos encontrados en el proyecto

Los principales retos técnicos identificados durante el desarrollo y el despliegue han sido:

- incompatibilidad de ESLint 9 con configuraciones clásicas, resuelta con migración a formato *flat config*;
- restricciones de Render en servicios Static Site y ajustes del blueprint para sincronización correcta;

- restricciones de correo saliente en PaaS, mitigadas usando proveedores HTTP (Brevo) para correos transaccionales;
- migración de proveedor de correo transaccional: inicialmente se utilizó SendGrid, pero al finalizar su plan gratuito se migró a Brevo. Se evaluó Resend como alternativa, pero su plan gratuito requiere dominio verificado para enviar a destinatarios arbitrarios. El diseño del `EmailService` con patrón de proveedor desacoplado permite intercambiar proveedores sin modificar lógica de negocio;
- conflictos y deprecaciones en SonarCloud, solventados migrando al plugin de Maven;
- tiempos excesivos en OWASP Dependency Check sin API Key de NVD, mitigados con credenciales dedicadas;
- errores CORS en despliegue (403) al no incluir el origen del frontend en la configuración;
- inactividad del backend en el tier gratuito, mitigada mediante *keep-alive* automatizado.

7.15– Análisis de la evolución del proyecto

El historial de commits con prefijo `feat` refleja una evolución en fases (36 registros):

- **Arranque y base:** commit inicial, definición de estructura y primeros flujos de CI.
- **Infraestructura:** despliegue continuo, Render, workflows de mantenimiento y seguridad (Dependabot, Sonar, auditorías).
- **Funcionalidad nuclear:** JWT, recuperación de contraseña, mejoras de administración, integración OneDrive, descargas y panel de evaluaciones.
- **Calidad y robustez:** ampliación de test, mutación, cobertura y eliminación de *hotspots*.
- **Documentación:** avances y consolidación de la memoria del TFG.

7.16– Conclusiones del capítulo

El manual de usuario consolida el uso operativo del sistema y garantiza que los tres perfiles principales disponen de un procedimiento claro, coherente con las reglas de negocio y alineado con los requisitos definidos.

Su existencia facilita adopción, reduce errores de uso y mejora la calidad del servicio académico prestado por la plataforma.

Conclusiones y desarrollos futuros

8.1– Síntesis final del trabajo

El desarrollo realizado ha permitido construir un sistema web de gestión de entregables académicos con una orientación claramente profesional, integrando análisis formal de requisitos, diseño de arquitectura, implementación por capas, verificación en profundidad y despliegue continuo.

El resultado no se limita a un prototipo funcional, sino a una solución trazable y mantenible, alineada con el problema de negocio detectado en el contexto docente: mejorar la gestión de actividades, entregas, evaluaciones y feedback frente a herramientas generalistas con menor adecuación a la operativa diaria.

8.2– Grado de cumplimiento de objetivos

El objetivo general planteado en la memoria (diseñar e implementar una plataforma de gestión de entregables usable, segura y evolutiva) puede considerarse **cumplido**.

Respecto a los objetivos específicos, el balance es el siguiente:

- **Modelado del dominio académico:** cumplido. Se han implementado entidades, relaciones y reglas de negocio acordes al ERS/DAS.
- **Soporte completo del flujo de entrega/evaluación:** cumplido. El sistema cubre ciclo de vida de entregables, entregas, calificación y feedback.
- **Control de acceso por roles y contexto:** cumplido. Se ha materializado autorización con verificaciones por perfil y recurso objetivo.
- **Calidad técnica y verificación:** cumplido. Se dispone de estrategia de pruebas en capas con métricas elevadas de cobertura y mutación.
- **Despliegue y operación automatizada:** cumplido. Se han definido pipelines CI/CD, controles de seguridad y configuración por entornos.

8.3– Aportaciones principales

Las aportaciones más relevantes del trabajo son:

1. **Aportación funcional:** una plataforma centrada en la operativa docente, con flujos diferenciados para administración, profesorado y alumnado.
2. **Aportación técnica:** arquitectura modular SPA + API, persistencia versionada con Flyway, seguridad JWT y soporte de integraciones cloud configurables.
3. **Aportación de calidad:** incorporación de testing multinivel y mutación como criterio objetivo de robustez.
4. **Aportación de operación:** canal CI/CD reproducible con validaciones continuas de calidad y seguridad.

8.4– Resultados y evidencia

La evidencia disponible en el proyecto permite sostener de forma objetiva la madurez alcanzada:

- cobertura alta en frontend con umbrales exigentes en líneas, funciones, sentencias y ramas;
- resultados favorables en mutación (frontend y backend), lo que indica que los tests detectan alteraciones de comportamiento;
- validación de flujos de usuario críticos mediante E2E;
- automatización de verificaciones en pipelines de integración continua.

En conjunto, estos resultados reducen el riesgo de regresión y proporcionan una base fiable para mantenimiento y evolución del sistema.

Las decisiones de arquitectura, pruebas y seguridad se han contrastado con documentación técnica oficial de las tecnologías empleadas y recomendaciones de referencia en seguridad web (VMware, 2026; Meta, 2026; Redgate, 2026; Contributors, 2026; Microsoft, 2026; Foundation, 2021).

8.5– Evaluación del impacto por tipo de usuario

El valor del proyecto se aprecia de forma diferente según el perfil de uso. Por ello, se realiza una evaluación diferenciada:

- **Profesorado:** disminución de tareas manuales repetitivas, mejor visibilidad del estado de entregas y consolidación del flujo de corrección.
- **Estudiantado:** proceso de entrega más guiado, mejor comprensión del estado de su trabajo y acceso más claro al feedback.
- **Administración:** mayor control sobre estructura académica y roles, con operaciones de gobierno centralizadas.

Desde esta perspectiva, el sistema no solo aporta funcionalidad técnica, sino también una mejora organizativa en la operativa académica.

8.6– Validación del enfoque de ingeniería adoptado

Una conclusión relevante del trabajo es que el enfoque metodológico aplicado (requisitos formalizados, arquitectura modular, pruebas en capas y CI/CD) ha sido adecuado para un proyecto de este alcance.

Los principales indicadores de validez del enfoque son:

1. coherencia entre objetivos, requisitos, implementación y verificación;
2. estabilidad de las iteraciones sin desviaciones críticas de calendario;
3. capacidad de incorporar mejoras sin ruptura estructural del sistema.

Esto refuerza la idea de que, en proyectos software de carácter académico-profesional, la disciplina metodológica es un factor diferencial tan importante como la selección tecnológica.

8.7– Limitaciones del trabajo

Como en todo proyecto con alcance temporal acotado, se identifican limitaciones que delimitan el estado actual:

- no todas las funcionalidades *Should/Could* del backlog están cerradas al 100 % en el corte del TFG;
- ciertas capacidades avanzadas (p. ej., autenticación federada completa, 2FA integral u observabilidad avanzada) se mantienen como evolución posterior;
- la estrategia E2E predominante en el ciclo actual es mockeada, por lo que conviene reforzar escenarios con backend real en staging.

Estas limitaciones no invalidan el cumplimiento del objetivo principal, pero orientan de forma clara la hoja de ruta futura.

8.8– Lecciones aprendidas

Durante la ejecución del proyecto se consolidan varios aprendizajes de interés para trabajos similares:

- diseñar la trazabilidad desde el inicio (requisito-backlog-código-prueba) simplifica justificación técnica y auditoría;
- la calidad debe construirse iterativamente, no solo en la fase final de cierre;
- desacoplar configuración por entorno reduce fricción en migraciones de infraestructura;
- integrar seguridad y permisos en la lógica de dominio evita defectos estructurales costosos.

8.9– Riesgos residuales y deuda técnica controlada

En el estado actual se identifican riesgos residuales, asumidos de forma explícita y controlada:

- cobertura funcional incompleta en funcionalidades *Should/Could*;
- necesidad de ampliar E2E contra backend real para ciertos escenarios;
- dependencia parcial de configuraciones externas en integraciones cloud.

La deuda técnica asociada se considera **controlada**, porque está identificada, priorizada y conectada con una hoja de ruta de evolución.

8.10– Desarrollos futuros

Se proponen líneas de evolución priorizadas por impacto y viabilidad.

8.10.1. Corto plazo

- cerrar PBIs pendientes de usabilidad y reporting docente;
- ampliar E2E con entorno semirreal y pruebas de regresión ampliadas;
- reforzar auditoría de permisos y evidencias de seguridad en reproducción.

8.10.2. Medio plazo

- autenticación federada institucional completa y doble factor transversal;
- observabilidad avanzada (métricas de negocio, trazas distribuidas y alertado);
- funcionalidades analíticas para seguimiento de actividad y carga de corrección.

8.10.3. Largo plazo

- incorporación de módulos de apoyo docente (detección de patrones de entrega, recomendación de revisión, etc.);
- estudio de interoperabilidad con LMS institucionales;
- escalado multi-institución con políticas avanzadas de gobierno de datos.

8.11– Hoja de ruta priorizada de continuidad

Para facilitar la continuidad del producto, se sintetiza una priorización por horizonte temporal y retorno esperado.

Horizonte	Iniciativas principales	Impacto esperado
Corto plazo	cierre de PBIs pendientes, ampliación de E2E y refuerzo de auditoría de permisos	mejora inmediata de robustez operativa
Medio plazo	federación de identidad, 2FA y observabilidad avanzada	aumento de seguridad y gobierno técnico
Largo plazo	interoperabilidad institucional y módulos avanzados de apoyo docente	escalabilidad funcional y adopción ampliada

Cuadro 8.1: Hoja de ruta priorizada para evolución del sistema

8.12– Reflexión final

El trabajo realizado demuestra que, incluso en un marco temporal exigente, es posible construir una solución académica con criterios de ingeniería profesionales si se mantiene una estrategia de trazabilidad, priorización de alcance y verificación continua.

La principal fortaleza del resultado no reside únicamente en las funcionalidades implementadas, sino en la consistencia entre problema, decisiones adoptadas y evidencia de calidad.

8.13– Cierre

El TFG demuestra que es posible construir, con un enfoque de ingeniería disciplinado, una solución académica de valor real para usuarios finales, sin renunciar a criterios de calidad, seguridad y mantenibilidad.

La plataforma resultante ofrece una base sólida para continuidad, adopción progresiva y extensión funcional, y cumple con los objetivos académicos y técnicos definidos al inicio del proyecto.

Apéndices

A.1– Introducción

Este bloque recoge información complementaria de carácter operativo que no forma parte del hilo principal de diseño e implementación, pero facilita reproducción, mantenimiento y auditoría del proyecto.

A.2– Anexo A. Manual de instalación

A.2.1. Requisitos previos

Para ejecutar la solución en entorno local se requiere:

- JDK 21 y Maven 3.9+ para backend.
- Node.js 18+ y npm 9+ para frontend.
- PostgreSQL 14+ disponible en local o contenedor.

A.2.2. Preparación del entorno

1. Clonar el repositorio y situarse en la raíz del proyecto.
2. Verificar credenciales y variables según el Anexo B.

A.2.3. Base de datos

Para entorno local se recomienda PostgreSQL con la base `tf gdb` y el usuario `tf g`. El repositorio incluye un script de apoyo `backend/run-postgres.ps1` que comprueba el servicio y arranca el backend. En caso de preferir contenedor, puede utilizarse `docker-compose` (ver subsección correspondiente).

A.2.4. Arranque del backend

Pasos de ejecución recomendados:

```
cd backend
./mvnw clean install
./mvnw spring-boot:run
```

Si se desea forzar configuración local con PostgreSQL, puede activarse el perfil `local` mediante `SPRING_PROFILES_ACTIVE=local`. Para pruebas rápidas sin base externa, está disponible el perfil `h2`.

A.2.5. Arranque del frontend

```
cd frontend
npm install
npm run dev
```

Por defecto el frontend consumirá la API local. Si es necesario, ajustar `VITE_API_BASE_URL` según el entorno.

A.2.6. Ejecución con contenedores

Como alternativa reproducible, la solución puede ejecutarse con `docker-compose`, levantando base de datos y backend con healthchecks. Esta opción reduce diferencias entre máquinas de desarrollo.

A.2.7. Build de producción

Para generar artefactos de despliegue:

```
cd backend
./mvnw clean package -DskipTests

cd frontend
npm run build
```

A.3– Anexo B. Configuración por entornos

A.3.1. Variables críticas de backend

Las variables mínimas para funcionamiento seguro en entornos no locales son:

- `DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD`,
- `JWT_SECRET`,
- `SPRING_PROFILES_ACTIVE`,
- `APP_FRONTEND_BASE_URL`.

En producción se habilitan además variables para servicios externos como almacenamiento cloud y correo transaccional. En el despliegue sobre Render, el correo se configura mediante Brevo usando API HTTP, por lo que deben definirse `BREVO_API_KEY` y `BREVO_FROM_EMAIL` con un remitente verificado en Brevo. Esta elección evita depender de conexiones SMTP salientes, que pueden estar restringidas por el PaaS.

A.3.2. Variables críticas de frontend

La variable principal de configuración de cliente es `VITE_API_BASE_URL`, que define el endpoint de API consumido.

A.4– Anexo C. Operación y verificación

A.4.1. Comprobaciones de salud

El sistema expone endpoints de salud para diferenciación operativa:

- liveness: valida disponibilidad del proceso,
- readiness: valida disponibilidad real de dependencias (base de datos).

Esta separación permite detectar de forma temprana incidencias de conectividad y reducir falsos positivos en monitorización.

A.4.2. Validación post-despliegue

Tras cada despliegue se recomienda ejecutar al menos:

1. comprobación de endpoints de salud,
2. login con usuario de prueba,
3. operación de lectura y escritura en flujo académico,
4. verificación de subida/descarga de material,
5. prueba de correo transaccional (recuperación de contraseña o notificación) mediante Brevo.

A.5– Anexo D. Declaración de uso de IA generativa

En este TFG se ha utilizado IA generativa como herramienta de apoyo en tareas acotadas y verificables, con el objetivo de mejorar claridad documental, revisar estructura y acelerar actividades de bajo riesgo. La IA no ha sustituido el criterio técnico ni ha generado decisiones de diseño o implementación sin validación humana.

A.5.1. Alcance y criterios de uso

El uso de IA se limitó a los siguientes casos:

- apoyo de redacción y reformulación de texto técnico;
- revisión de estructura y coherencia de secciones;
- generación de listados de verificación y casos borde;
- microcopy y textos de ayuda para interfaz;
- aclaraciones puntuales de sintaxis o formatos (por ejemplo, JSON o $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$).

Se excluyeron explícitamente usos como:

- decisiones arquitectónicas finales sin justificación propia;
- código productivo incorporado sin revisión manual;
- resultados empíricos o métricas no verificadas;
- contenido no contrastado con el repositorio y la evidencia de pruebas.

A.5.2. Proceso de control y verificación

Cada propuesta generada por IA se trató como borrador. El proceso seguido fue:

1. formular la consulta indicando el contexto del proyecto,
2. evaluar si la respuesta era coherente con requisitos y código,
3. ajustar manualmente terminología y rigor académico,
4. contrastar con artefactos del repositorio (memoria, código, tests),
5. integrar la versión final ya revisada.

A.5.3. Ejemplos concretos de consultas y respuestas

Los siguientes ejemplos resumen consultas reales y la respuesta resumida que se utilizó como punto de partida. En todos los casos se realizó revisión manual antes de incorporar el contenido a la memoria o al producto.

1. **Redacción académica de objetivos.** *Consulta:* *Reescribe este párrafo de objetivos para un tono académico, evitando repeticiones y manteniendo el significado.* **Respuesta resumida:** propuesta de redacción más concisa con estructura en dos frases y vocabulario técnico coherente. **Aplicación:** se ajustaron términos a la nomenclatura del ERS y se incorporó solo tras validar la trazabilidad con requisitos.
2. **Estructura de sección comparativa.** *Consulta:* *Sugiere un esquema para un análisis de competidores en una memoria de TFG de software.* **Respuesta resumida:** se propuso dividir por tipologías de competidor, incluir criterios de comparación y cerrar con diferenciadores propios. **Aplicación:** el esquema se adaptó al contenido real del proyecto y a las evidencias técnicas implementadas.
3. **Validación de estructuras ZIP.** *Consulta:* *Propón ejemplos de reglas para validar estructura interna de un ZIP con archivos obligatorios y carpetas esperadas.* **Respuesta resumida:** lista de casos con archivos obligatorios, extensiones permitidas y modos estricto/mínimo requerido. **Aplicación:** se contrastó con la lógica del servicio de validación y se redactó la explicación en la memoria con terminología consistente.
4. **Textos de ayuda en interfaz de entrega.** *Consulta:* *Sugiere mensajes breves y claros para indicar que una entrega requiere un ZIP con nombre y estructura concretos.* **Respuesta resumida:** propuestas de microcopy con tono informativo y referencias explícitas a nombre esperado, tamaño y estructura. **Aplicación:** los textos se ajustaron manualmente para alinearlos con el vocabulario usado en la UI del proyecto.
5. **Casos borde para pruebas.** *Consulta:* *Enumera casos borde para pruebas de subida de ZIP en un sistema de entregables.* **Respuesta resumida:** listado de escenarios (nombre incorrecto, estructura incompleta, archivos extra, ZIP corrupto, tamaño excedido). **Aplicación:** se utilizó como checklist de verificación y se descartaron casos no aplicables al alcance real.

Este anexo se incluye para garantizar transparencia metodológica y trazabilidad del proceso de elaboración de la memoria.

A.6– Anexo E. Documentación complementaria disponible

Como soporte adicional al contenido principal de la memoria, el repositorio incorpora documentos técnicos de operación, seguridad, testing y despliegue que permiten ampliar detalle en revisiones futuras.

A.7– Anexo F. Referencias técnicas de apoyo

Para el diseño, implementación y validación de la solución se han utilizado como apoyo las guías oficiales de los principales frameworks y herramientas del proyecto, incluyendo Spring Boot, React, Flyway, Vitest, Playwright y OWASP.

A.8– Cierre de apéndices

Los anexos anteriores consolidan la dimensión operativa y metodológica del TFG, aportando una base práctica para reproducibilidad y continuidad del proyecto más allá del alcance temporal de la convocatoria.

Referencias

- Contributors, V. (2026). *Vitest documentation*. Descargado de <https://vitest.dev/> (Fecha de consulta: 8 de abril de 2026)
- Foundation, O. (2021). *Owasp top 10: The ten most critical web application security risks*. Descargado de <https://owasp.org/www-project-top-ten/> (Fecha de consulta: 8 de abril de 2026)
- Meta. (2026). *React documentation*. Descargado de <https://react.dev/> (Fecha de consulta: 8 de abril de 2026)
- Microsoft. (2026). *Playwright documentation*. Descargado de <https://playwright.dev/> (Fecha de consulta: 8 de abril de 2026)
- Redgate. (2026). *Flyway documentation*. Descargado de <https://documentation.red-gate.com/flyway> (Fecha de consulta: 8 de abril de 2026)
- VMware. (2026). *Spring boot reference documentation*. Descargado de <https://docs.spring.io/spring-boot/documentation.html> (Fecha de consulta: 8 de abril de 2026)